Name: Darshan Prajapati
Roll no: 24
MCA 3

P1

Q1. Read an image using OpenCV and display its width, height, and number of channels

```
import matplotlib.pyplot as plt
# %pip install opencv-python
import cv2
```

```
# read the image
img = cv2.imread("image.jpg")

if img is None:
    print("Image not found")
else:
    print("Image loaded successfully")
```

```
Image loaded successfully
```

```
# Get height, width, and number of channels
height,width,channels = img.shape
```

```
# Display information
print("Width:", width)
print("Height:", height)
print("Number of Channels:", channels)
```

```
Width: 4702
Height: 3394
Number of Channels: 3
```

```
# Display image
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



Q2. Read an image and save it in another image format

```
import cv2
import matplotlib.pyplot as plt
```

```
# Read the image
img = cv2.imread("image.jpg")
if img is None:
    print("Image not found!")
else:
    print("Image Loaded Successfully")
```

```
Image Loaded Successfully
```

```
# Save the image in PNG format
cv2.imwrite("image.png", img)

print("Image saved successfully")
```

```
Image saved successfully
```

Q3. Resize an image to 224 x 224 pixels and display the original and resized images

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("image.jpg")

if img is None:
    print("Image not found!")
else:
    print("Image Loaded Successfully")
```

```
Image Loaded Successfully
```

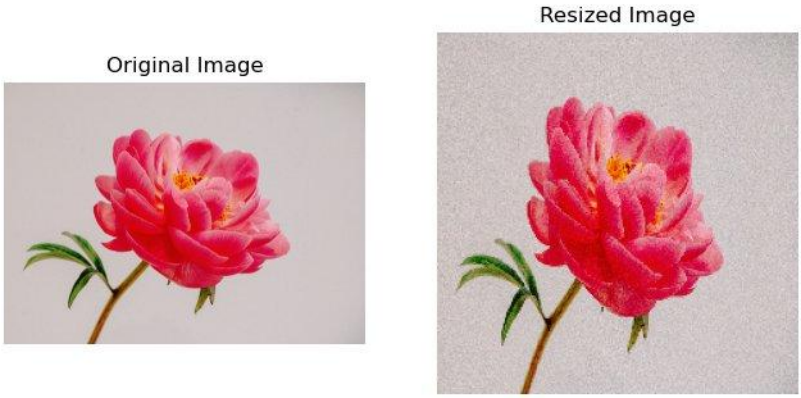
```
# Resize the image
resized_img = cv2.resize(img, (224, 224))

# Display original and resized images
plt.figure(figsize=(8, 4))

# Original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# Resized image
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB))
plt.title("Resized Image")
plt.axis("off")

plt.show()
```



Q4. Resize an image to 50% of its original dimensions

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("image.jpg")

# Get original dimensions
height, width = img.shape[:2]

# Calculate 50% dimensions
new_width = int(width * 0.5)
new_height = int(height * 0.5)

# Resize the image
resized_img = cv2.resize(img, (new_width, new_height))

# Display original and resized images
plt.figure(figsize=(8, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB))
plt.title("50% Resized")
plt.axis("off")

plt.show()
```



Q5. Crop a rectangular region from an image using specified starting and ending coordinates

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("image.jpg")

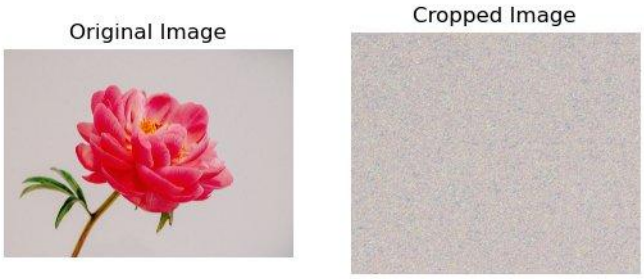
# Starting and ending coordinates
x1, y1 = 100, 50
x2, y2 = 400, 300

# Crop the image
cropped = img[y1:y2, x1:x2]

# Display original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# Display cropped image
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(cropped, cv2.COLOR_BGR2RGB))
plt.title("Cropped Image")
plt.axis("off")

plt.show()
```



Q.6 Extract the central region of an image using cropping

```
import cv2
import matplotlib.pyplot as plt

# Read the image
```

```
img = cv2.imread("image.jpg")

# Get image dimensions
height, width = img.shape[:2]

# Size of central region
crop_width = 1000
crop_height = 1000

# Find starting coordinates
x1 = (width - crop_width) // 2
y1 = (height - crop_height) // 2

# Find ending coordinates
x2 = x1 + crop_width
y2 = y1 + crop_height

# Crop the central region
cropped = img[y1:y2, x1:x2]

# Display images
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(cropped, cv2.COLOR_BGR2RGB))
plt.title("Central Region")
plt.axis("off")

plt.show()
```



Q.7 Convert a color image from BGR/RGB to grayscale

```
import cv2
import matplotlib.pyplot as plt

# Read the color image
img = cv2.imread("image.jpg")

# Convert BGR image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display original image
plt.subplot(1, 2, 1)
```

```
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Color Image")
plt.axis("off")

# Display grayscale image
plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.show()
```



Q.8 Convert an image from BGR/RGB to HSV color space and display the result

```
import cv2
import matplotlib.pyplot as plt

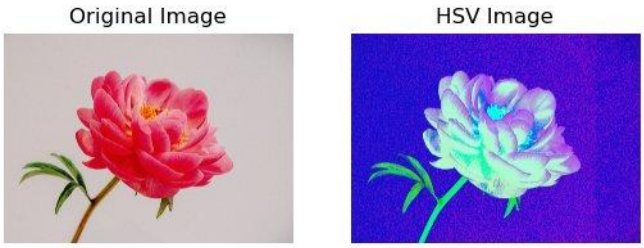
# Read the image (OpenCV reads it in BGR format)
img = cv2.imread("image.jpg")

# Convert BGR to HSV
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Display original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# Display HSV image
plt.subplot(1, 2, 2)
plt.imshow(hsv_img)
plt.title("HSV Image")
plt.axis("off")

plt.show()
```



Q.9 Convert an image from BGR/RGB to LAB color space and display the result

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("image.jpg")

# Convert BGR to LAB
lab_img = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)

# Display original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# # Convert LAB back to RGB for proper display
# lab_display = cv2.cvtColor(lab_img, cv2.COLOR_LAB2RGB)

# Display LAB image
plt.subplot(1, 2, 2)
plt.imshow(lab_img)
plt.title("LAB Image")
plt.axis("off")

plt.show()
```



Q.10 Add salt-and-pepper noise to an image and remove the noise using a median filter

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Read the image
img = cv2.imread("Test.jpg",cv2.IMREAD_GRAYSCALE)

# Convert BGR to RGB for processing/display
img_rgb = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)

# Create a copy of the image
noisy_img = img_rgb.copy()

# Number of noisy pixels
num_pixels = 5000

# Get image height and width
height, width = img_rgb.shape[:2]
```

```
# Add salt noise (white pixels)
for i in range(num_pixels // 2):
    x = np.random.randint(0, width)
    y = np.random.randint(0, height)
    noisy_img[y, x] = [255, 255, 255]

# Add pepper noise (black pixels)
for i in range(num_pixels // 2):
    x = np.random.randint(0, width)
    y = np.random.randint(0, height)
    noisy_img[y, x] = [0, 0, 0]

# Remove noise using median filter
denoised_img = cv2.medianBlur(noisy_img, 5)
```

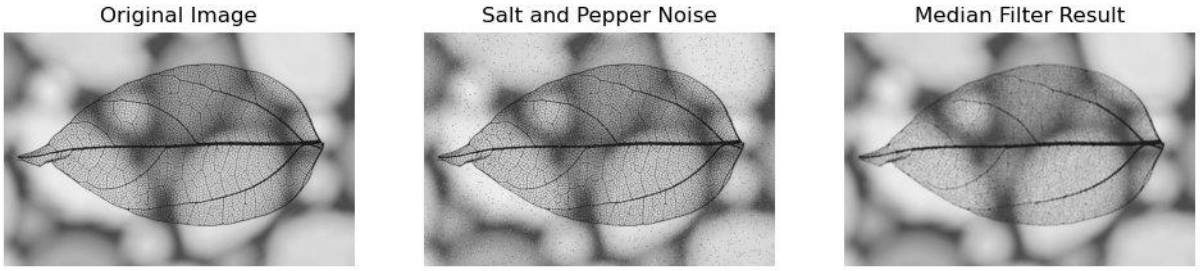
```
# Display images
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(noisy_img)
plt.title("Salt and Pepper Noise")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(denoised_img)
plt.title("Median Filter Result")
plt.axis("off")

plt.show()
```




```
import cv2
import random

# Read image in grayscale
image = cv2.imread("Test.jpg", cv2.IMREAD_GRAYSCALE)

if image is None:
    raise ValueError("Could not load image")

# Add salt and pepper noise
def add_salt_and_pepper_noise(image, noise_prob):
```

```
    noisy_image = image.copy()
    height, width = noisy_image.shape

    for i in range(height):
        for j in range(width):
            r = random.random()

            if r < noise_prob / 2:
                noisy_image[i, j] = 255       # Salt
            elif r < noise_prob:
                noisy_image[i, j] = 0        # Pepper

    return noisy_image


# Add 5% noise
noisy_image = add_salt_and_pepper_noise(image, 0.05)

# Remove noise using median filter
filtered_image = cv2.medianBlur(noisy_image, 5)

# Display
cv2.imshow("Original Image", image)
cv2.imshow("Noisy Image", noisy_image)
cv2.imshow("Filtered Image", filtered_image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Q.11 Apply a mean filter to an image and observe its effect on image quality

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("Test.jpg")

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

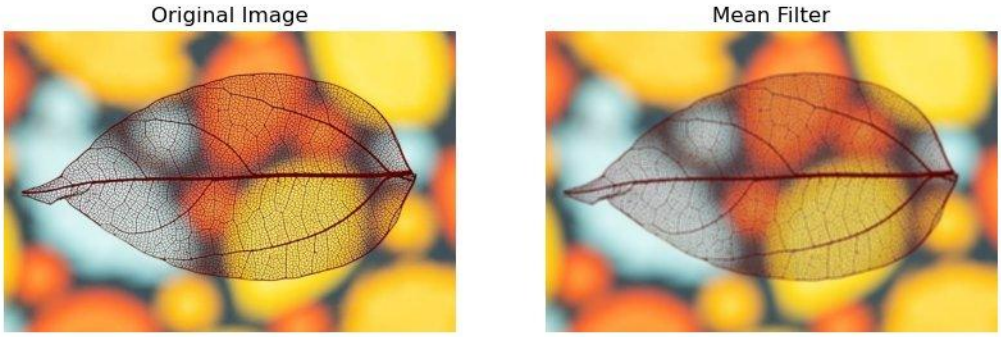
# Apply mean filter
mean_filtered = cv2.blur(img_rgb, (5, 5))

# Display images
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(mean_filtered)
plt.title("Mean Filter")
plt.axis("off")

plt.show()
```



Q.12 Apply Gaussian smoothing using 3x3, 5x5, and 7x7 kernels and compare the results

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("Test.jpg")

# Convert BGR to RGB for displaying
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Apply Gaussian smoothing with different kernel sizes
gaussian_3 = cv2.GaussianBlur(img_rgb, (3, 3), 0)
gaussian_5 = cv2.GaussianBlur(img_rgb, (5, 5), 0)
gaussian_7 = cv2.GaussianBlur(img_rgb, (7, 7), 0)



# Display the images
plt.figure(figsize=(12, 8))

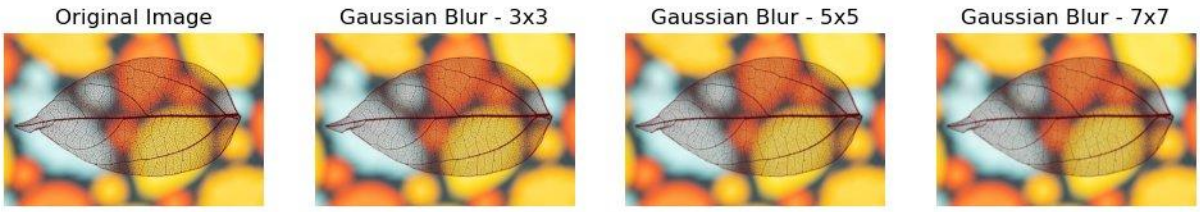
# Original image
plt.subplot(1, 4, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# 3x3 Gaussian filter
plt.subplot(1, 4, 2)
plt.imshow(gaussian_3)
plt.title("Gaussian Blur - 3x3")
plt.axis("off")

# 5x5 Gaussian filter
plt.subplot(1, 4, 3)
plt.imshow(gaussian_5)
plt.title("Gaussian Blur - 5x5")
plt.axis("off")

# 7x7 Gaussian filter
plt.subplot(1, 4, 4)
plt.imshow(gaussian_7)
plt.title("Gaussian Blur - 7x7")
plt.axis("off")

plt.show()
```



Q.13 Apply bilateral filtering to an image and observe how it affects noise and edges

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("Dog.jpg")

# Convert BGR to RGB for display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

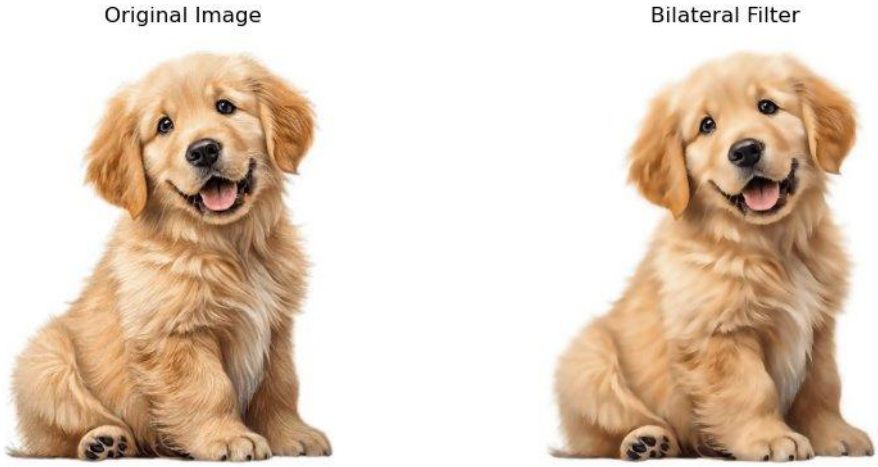
# Apply bilateral filter
bilateral_img = cv2.bilateralFilter(img_rgb, 9, 75, 75)

# Display original and filtered images
plt.figure(figsize=(10, 5))

# Original image
plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Bilateral filtered image
plt.subplot(1, 2, 2)
plt.imshow(bilateral_img)
plt.title("Bilateral Filter")
plt.axis("off")

plt.show()
```



Q.14 Apply Gaussian, median, and bilateral filters to the same noisy image and compare their results

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("test.jpg")

# Convert BGR to RGB for display
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Create a copy to add noise
noisy_img = img_rgb.copy()

# Get image dimensions
height, width = noisy_img.shape[:2]

# Number of noisy pixels
num_pixels = 5000

# Add salt-and-pepper noise
for i in range(num_pixels):

    # Generate random coordinates
    x = np.random.randint(0, width)
    y = np.random.randint(0, height)

    # Add salt or pepper noise
    if i % 2 == 0:
        noisy_img[y, x] = [255, 255, 255]  # Salt (White)
    else:
        noisy_img[y, x] = [0, 0, 0]        # Pepper (Black)


# Apply Gaussian filter
gaussian = cv2.GaussianBlur(noisy_img, (5, 5), 0)

# Apply Median filter
median = cv2.medianBlur(noisy_img, 5)

# Apply Bilateral filter
bilateral = cv2.bilateralFilter(noisy_img, 9, 75, 75)



# Display all images
plt.figure(figsize=(15, 10))

# Original image
plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Noisy image
plt.subplot(2, 3, 2)
plt.imshow(noisy_img)
```

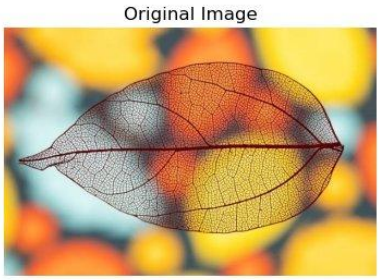
```
plt.title("Noisy Image")
plt.axis("off")

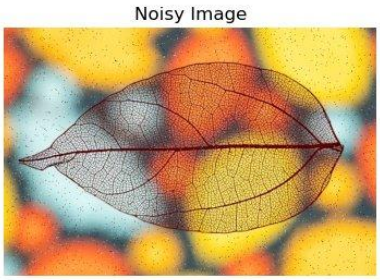
# Gaussian filter
plt.subplot(2, 3, 3)
plt.imshow(gaussian)
plt.title("Gaussian Filter")
plt.axis("off")

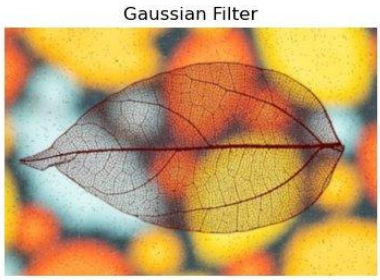
# Median filter
plt.subplot(2, 3, 4)
plt.imshow(median)
plt.title("Median Filter")
plt.axis("off")

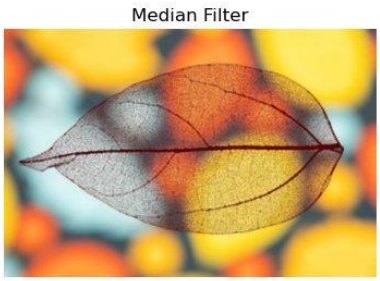
# Bilateral filter
plt.subplot(2, 3, 5)
plt.imshow(bilateral)
plt.title("Bilateral Filter")
plt.axis("off")

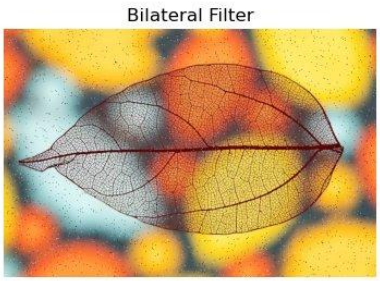
plt.show()
```











Q.15 Sharpen an image using a Laplacian-based sharpening technique

```
import cv2
import matplotlib.pyplot as plt
```

```
# Read the image
img = cv2.imread("coin.jpg")

# Convert BGR to RGB for display
```

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply Laplacian operator
laplacian = cv2.Laplacian(gray, cv2.CV_64F)

# Convert Laplacian to unsigned 8-bit format
laplacian_abs = cv2.convertScaleAbs(laplacian)

# Sharpen the image
sharpened = cv2.subtract(img_rgb, cv2.cvtColor(laplacian_abs, cv2.COLOR_GRAY2RGB))
```

```
# Display images
plt.figure(figsize=(12, 4))

# Original image
plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Laplacian edges
plt.subplot(1, 3, 2)
plt.imshow(laplacian_abs, cmap="gray")
plt.title("Laplacian Edges")
plt.axis("off")

# Sharpened image
plt.subplot(1, 3, 3)
plt.imshow(sharpened)
plt.title("Sharpened Image")
plt.axis("off")

plt.show()
```



Q.16 Apply unsharp masking

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("coin.jpg")

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# Create a blurred version
blurred = cv2.GaussianBlur(img_rgb, (5, 5), 0)

# Apply unsharp masking
sharpened = cv2.addWeighted(
    img_rgb, 1.5,
    blurred, -0.5,
    0
)

# Display images
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(blurred)
plt.title("Blurred Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(sharpened)
plt.title("Unsharp Masking")
plt.axis("off")

plt.show()
```



Q.17 Perform contrast stretching on a low-contrast grayscale image

```
import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("dog.jpg", cv2.IMREAD_GRAYSCALE)

# Find minimum and maximum pixel values
min_val = img.min()
max_val = img.max()

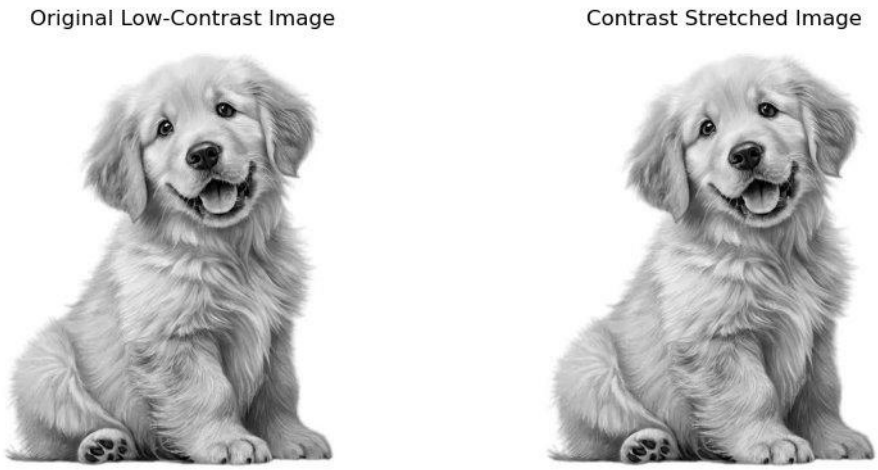
# Perform contrast stretching
stretched = ((img - min_val) / (max_val - min_val) * 255).astype("uint8")

# Display original and stretched images
plt.figure(figsize=(10, 5))
```

```
plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Low-Contrast Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(stretched, cmap="gray")
plt.title("Contrast Stretched Image")
plt.axis("off")

plt.show()
```



Q.18 Apply histogram equalization and display the image and its histogram before and after processing

```
import cv2
import matplotlib.pyplot as plt
```

```
# Read image in grayscale
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Apply histogram equalization
equalized = cv2.equalizeHist(img)
```

```
# Create figure
plt.figure(figsize=(12, 8))

# Original image
plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

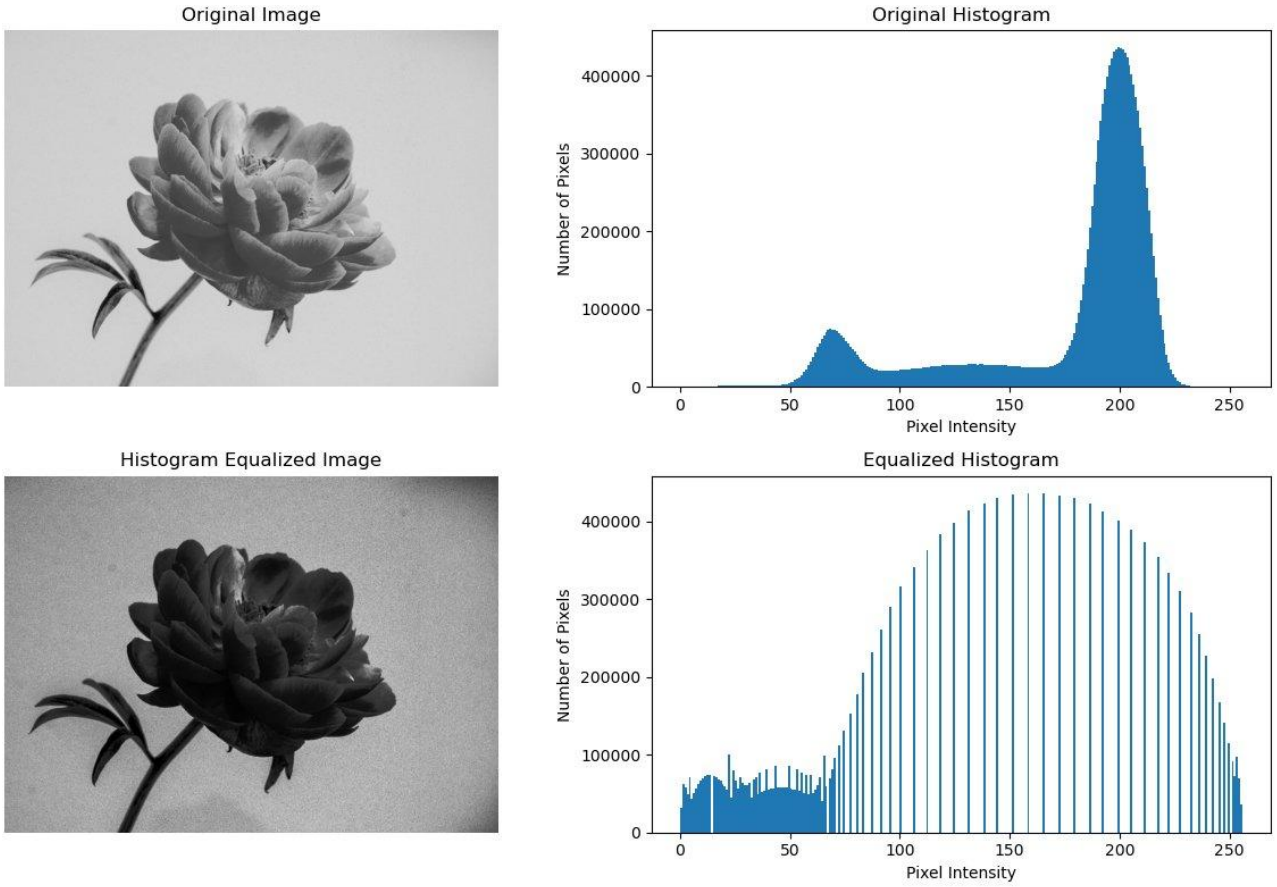
# Original histogram
plt.subplot(2, 2, 2)
plt.hist(img.ravel(), bins=256, range=[0, 256])
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
```

```
plt.ylabel("Number of Pixels")

# Equalized image
plt.subplot(2, 2, 3)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized Image")
plt.axis("off")

# Equalized histogram
plt.subplot(2, 2, 4)
plt.hist(equalized.ravel(), bins=256, range=[0, 256])
plt.title("Equalized Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Number of Pixels")

plt.tight_layout()
plt.show()
```



Q.19 Apply CLAHE to a low-contrast image and compare the result with standard histogram equalization

```
import cv2
import matplotlib.pyplot as plt

# Read the image in grayscale
image = cv2.imread("image.jpg")
```

```
normalized_image = image.astype(np.float32) / 255.0

print("--- Before Normalization ---")
print(f"Data Type: {image.dtype}")
print(f"Min Value: {image.min()}, Max Value: {image.max()}")

print("\n--- After Normalization ---")
print(f"Data Type: {normalized_image.dtype}")
print(f"Min Value: {normalized_image.min():.2f}, Max Value: {normalized_image.max():.2f}")
# Display images
plt.figure(figsize=(15, 5))

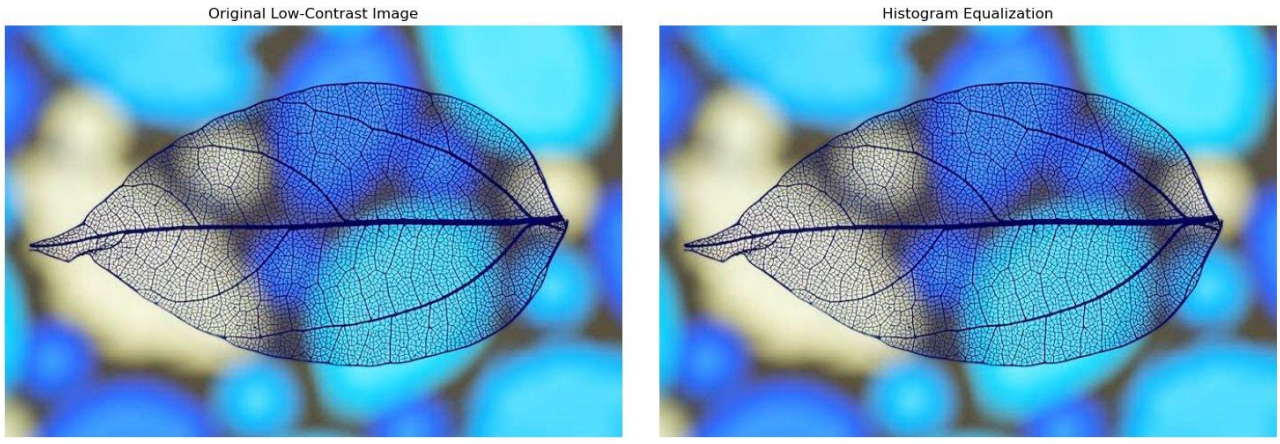
# Original image
plt.subplot(1, 2, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Low-Contrast Image")
plt.axis("off")

# Standard histogram equalization
plt.subplot(1, 2, 2)
plt.imshow(normalized_image, cmap="gray")
plt.title("Histogram Equalization")
plt.axis("off")


plt.tight_layout()
plt.show()
```

```
--- Before Normalization ---
Data Type: uint8
Min Value: 0, Max Value: 255

--- After Normalization ---
Data Type: float32
Min Value: 0.00, Max Value: 1.00
```



Q20. Normalize an image from the pixel range 0-255 to 0-1

```
import cv2
```

```
# Read image
img = cv2.imread("image.jpg")

# Normalize pixel values from 0-255 to 0-1
normalized_img = img / 255.0

# Check ranges
print("Original range:", img.min(), "to", img.max())
print("Normalized range:",normalized_img.min(),"to",normalized_img.max())
```

```
Original range: 0 to 255
Normalized range: 0.0 to 1.0
```

Q.21 Standardize the pixel values of a grayscale image using its mean and standard deviation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image in grayscale
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Convert image to float
img_float = img.astype(np.float32)

# Calculate mean and standard deviation
mean = img_float.mean()
std = img_float.std()

# Standardize the image
standardized_img = (img_float - mean) / std

# Print values
print("Original Mean:", mean)
print("Original Standard Deviation:", std)

print("Standardized Mean:", standardized_img.mean())
print("Standardized Standard Deviation:", standardized_img.std())

# Display images
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(standardized_img, cmap="gray")
plt.title("Standardized Image")
plt.axis("off")

plt.show()
```

```
Original Mean: 176.18396
```

```
Original Standard Deviation: 45.599598
Standardized Mean: 2.7962847e-06
Standardized Standard Deviation: 0.9999998
```



Q.22 Apply global thresholding to a grayscale image using a threshold value of 127

```
import cv2
import matplotlib.pyplot as plt

# Read the image in grayscale
img = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE)

# Apply global thresholding
_, threshold_img = cv2.threshold(
    img,
    127,
    255,
    cv2.THRESH_BINARY
)

# Display images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(threshold_img, cmap="gray")
plt.title("Thresholded Image")
plt.axis("off")

plt.show()
```





Q.23 Apply Otsu's thresholding method to automatically determine a suitable threshold for an image

```
import cv2
import matplotlib.pyplot as plt
```

```
# read the image in grayscale
img = cv2.imread("dog.jpg",cv2.IMREAD_GRAYSCALE)

# Apply Otsu's thresholding
threshold_value , otsu_img = cv2.threshold(img,0,255,cv2.THRESH_BINARY+cv2.THRESH_OTSU)



# Print automatically selected threshold
print("Otsu's Threshold Value:", threshold_value)
```

```
Otsu's Threshold Value: 194.0
```

```
# Display images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(otsu_img, cmap="gray")
plt.title("Otsu Thresholding")
plt.axis("off")

plt.show()
```





Q.24 Apply adaptive mean and adaptive Gaussian thresholding to an image and compare the results

```
import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("dog.jpg", cv2.IMREAD_GRAYSCALE)

# Adaptive Mean Thresholding
adaptive_mean = cv2.adaptiveThreshold(
    img,
    255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

# Adaptive Gaussian Thresholding
adaptive_gaussian = cv2.adaptiveThreshold(
    img,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

# Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(adaptive_mean, cmap="gray")
plt.title("Adaptive Mean")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(adaptive_gaussian, cmap="gray")
plt.title("Adaptive Gaussian")
```

```
plt.axis("off")

plt.tight_layout()
plt.show()
```



Q.25 Convert a grayscale image into a binary image and analyze the effect of different threshold values

```
import cv2
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("dog.jpg", cv2.IMREAD_GRAYSCALE)

# Apply different threshold values
_, threshold_50 = cv2.threshold(
    img, 50, 255, cv2.THRESH_BINARY
)

_, threshold_100 = cv2.threshold(
    img, 100, 255, cv2.THRESH_BINARY
)

_, threshold_150 = cv2.threshold(
    img, 150, 255, cv2.THRESH_BINARY
)

_, threshold_200 = cv2.threshold(
    img, 200, 255, cv2.THRESH_BINARY
)



# Display results
plt.figure(figsize=(12, 8))

# Original grayscale image
plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Grayscale")
plt.axis("off")

# Threshold = 50
```

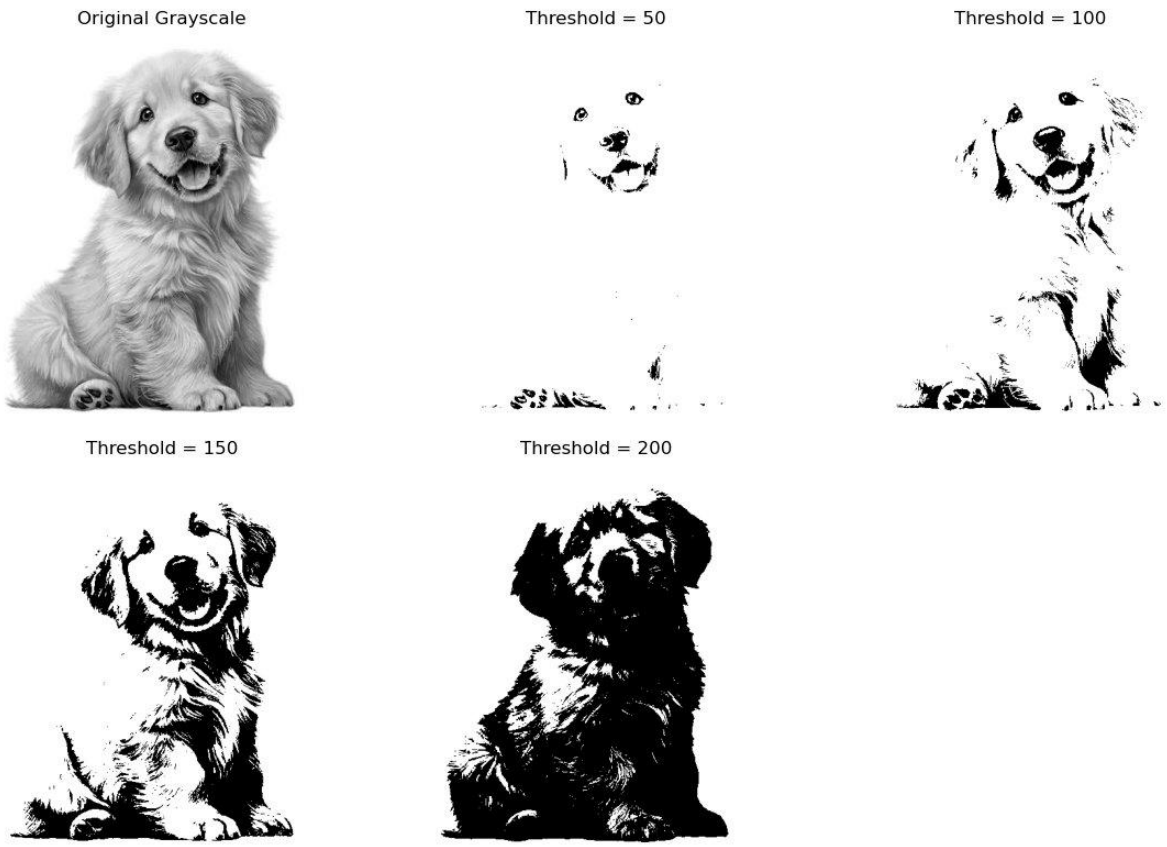
```
plt.subplot(2, 3, 2)
plt.imshow(threshold_50, cmap="gray")
plt.title("Threshold = 50")
plt.axis("off")

# Threshold = 100
plt.subplot(2, 3, 3)
plt.imshow(threshold_100, cmap="gray")
plt.title("Threshold = 100")
plt.axis("off")

# Threshold = 150
plt.subplot(2, 3, 4)
plt.imshow(threshold_150, cmap="gray")
plt.title("Threshold = 150")
plt.axis("off")

# Threshold = 200
plt.subplot(2, 3, 5)
plt.imshow(threshold_200, cmap="gray")
plt.title("Threshold = 200")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Q.26 Apply erosion and dilation to a binary image and display the results

```
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("dog.jpg", cv2.IMREAD_GRAYSCALE)

# Convert grayscale image to binary
_, binary_img = cv2.threshold(img,127,255,cv2.THRESH_BINARY)

# Create kernel
kernel = np.ones((5, 5), np.uint8)

# Apply erosion
erosion = cv2.erode(binary_img,kernel,iterations=1)

# Apply dilation
dilation = cv2.dilate(binary_img,kernel,iterations=1)

# Display results
plt.figure(figsize=(15, 5))

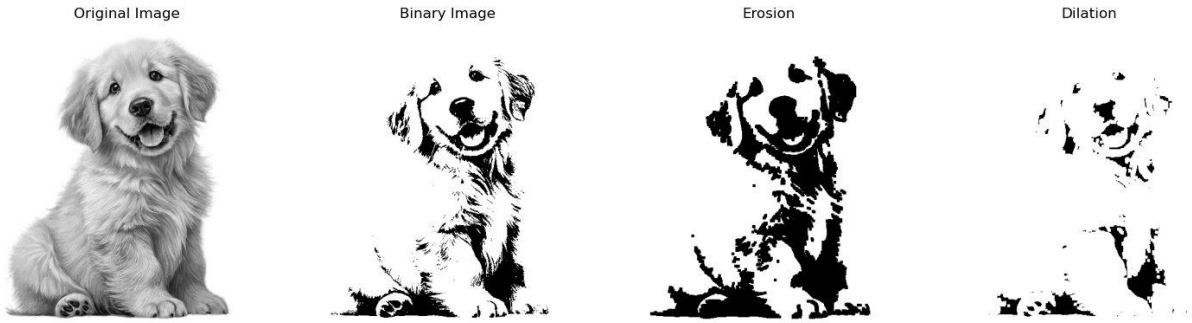
# Original grayscale image
plt.subplot(1, 4, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

# Binary image
plt.subplot(1, 4, 2)
plt.imshow(binary_img, cmap="gray")
plt.title("Binary Image")
plt.axis("off")

# Erosion
plt.subplot(1, 4, 3)
plt.imshow(erosion, cmap="gray")
plt.title("Erosion")
plt.axis("off")

# Dilation
plt.subplot(1, 4, 4)
plt.imshow(dilation, cmap="gray")
plt.title("Dilation")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Q.27 Apply morphological opening and closing to a binary image containing small noise and gaps

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("dog.jpg", cv2.IMREAD_GRAYSCALE)

# Convert grayscale image to binary
_, binary_img = cv2.threshold(
    img,
    127,
    255,
    cv2.THRESH_BINARY
)

# Create a kernel
kernel = np.ones((5, 5), np.uint8)

# Apply morphological opening
opening = cv2.morphologyEx(
    binary_img,
    cv2.MORPH_OPEN,
    kernel
)

# Apply morphological closing
closing = cv2.morphologyEx(
    binary_img,
    cv2.MORPH_CLOSE,
    kernel
)

# Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(binary_img, cmap="gray")
plt.title("Original Binary Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(opening, cmap="gray")
plt.title("Opening")
```

```
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(closing, cmap="gray")
plt.title("Closing")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Q.28 Apply Sobel, Prewitt, Laplacian, and Canny edge detection to the same image and compare the detected edges

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("a.jpg", cv2.IMREAD_GRAYSCALE)

# -------------------------
# 1. Sobel Edge Detection
# -------------------------

sobel_x = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)
sobel_y = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)

# Combine X and Y edges
sobel = cv2.magnitude(sobel_x, sobel_y)
sobel = cv2.convertScaleAbs(sobel)



# -------------------------
# 2. Prewitt Edge Detection
# -------------------------

prewitt_x_kernel = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
])
```

```
prewitt_y_kernel = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
])

prewitt_x = cv2.filter2D(img,cv2.CV_64F,prewitt_x_kernel)
prewitt_y = cv2.filter2D(img,cv2.CV_64F,prewitt_y_kernel)

# Combine X and Y edges
prewitt = cv2.magnitude(prewitt_x,prewitt_y)
prewitt = cv2.convertScaleAbs(prewitt)



# -------------------------
# 3. Laplacian Edge Detection
# -------------------------

laplacian = cv2.Laplacian(img,cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)



# -------------------------
# 4. Canny Edge Detection
# -------------------------

canny = cv2.Canny(img,100,200)
```

```
# -------------------------
# Display Results
# -------------------------

plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(sobel, cmap="gray")
plt.title("Sobel Edges")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(prewitt, cmap="gray")
plt.title("Prewitt Edges")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian Edges")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(canny, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



Q.29 Detect the edges of an object using Canny edge detection and display the resulting edge image

```
import cv2
import matplotlib.pyplot as plt

# Read the image
img = cv2.imread("dog.jpg")

# Convert BGR to RGB for displaying
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert the image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply Gaussian blur to reduce noise
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

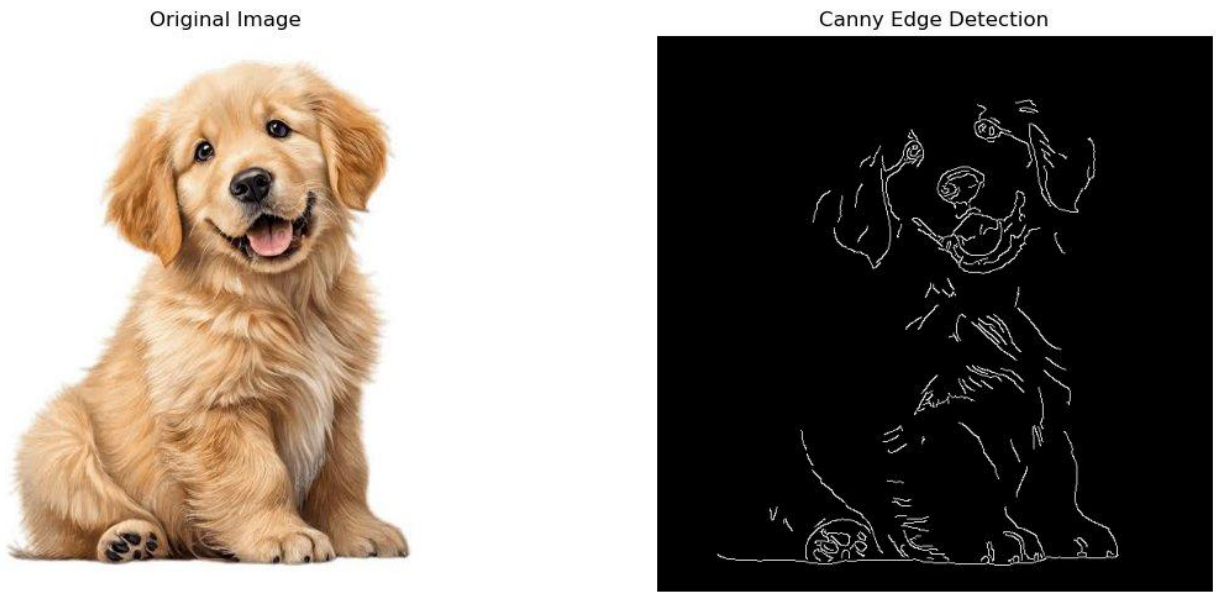
# Apply Canny edge detection
edges = cv2.Canny(blurred, 100, 200)

# Display original and edge image
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
```

```
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Q.30 Perform background removal using grayscale conversion, thresholding, and a binary mask, then display the extracted foreground object

```
import cv2
import matplotlib.pyplot as plt

# Read the original image
img = cv2.imread("a.jpg")

# Convert BGR to RGB for displaying
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Create a binary mask
_, mask = cv2.threshold(gray,127,255,cv2.THRESH_BINARY)

# Extract foreground using the binary mask
foreground = cv2.bitwise_and(img,img,mask=mask)

# Convert extracted foreground to RGB
foreground_rgb = cv2.cvtColor(foreground,cv2.COLOR_BGR2RGB)
```

```
# Display results
plt.figure(figsize=(15, 5))

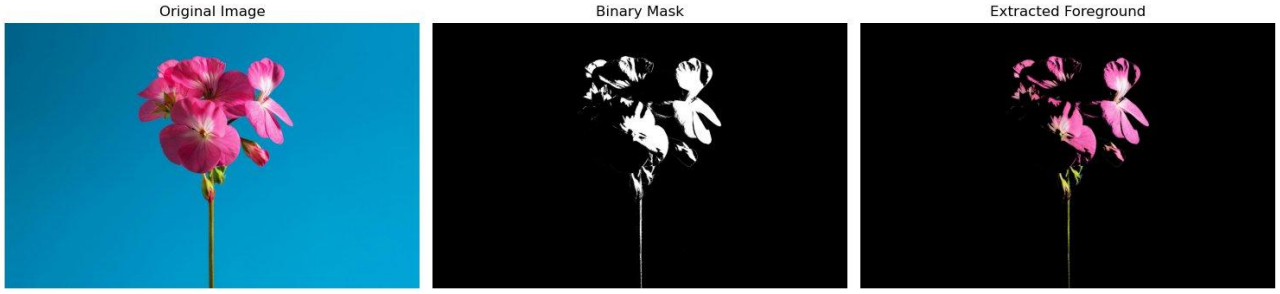
# Original image
```

```
plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Binary mask
plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(mask,cv2.COLOR_GRAY2RGB))
plt.title("Binary Mask")
plt.axis("off")

# Extracted foreground
plt.subplot(1, 3, 3)
plt.imshow(foreground_rgb)
plt.title("Extracted Foreground")
plt.axis("off")

plt.tight_layout()
plt.show()
```



P2

Question 1 –

Canny Edge Detection Write a Python program to: • Read an input image. • Convert it to grayscale. • Apply Gaussian smoothing. • Apply Canny edge detection. • Display the original and edge-detected images. Try different lower and upper threshold values and observe the changes.

```
import cv2
import matplotlib.pyplot as plt
```

```
# Step 1: Read the image
image = cv2.imread("dog.jpg")

# Step 2: Convert the image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Apply Gaussian smoothing
blur = cv2.GaussianBlur(gray, (9, 9), 0)

# Step 4: Apply Canny Edge Detection
edges = cv2.Canny(blur, 30, 100)
```

```
plt.figure(figsize=(10,5))

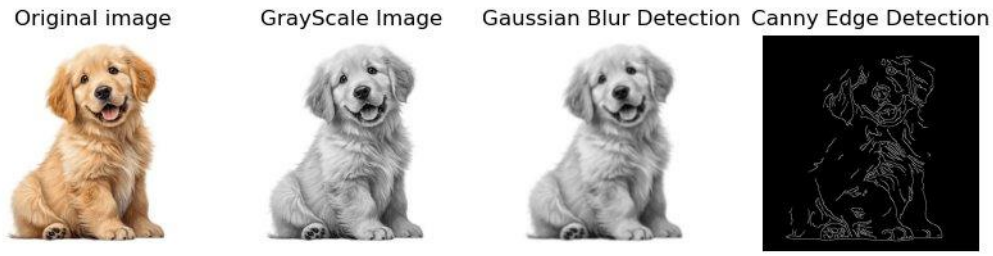
plt.subplot(1,4,1)
plt.imshow(cv2.cvtColor(image,cv2.COLOR_BGR2RGB))
plt.title("Original image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(gray,cmap="gray")
plt.title("GrayScale Image")
plt.axis("off")

plt.subplot(1, 4,3)
plt.imshow(blur,cmap="gray")
plt.title("Gaussian Blur Detection")
plt.axis("off")

plt.subplot(1,4,4)
plt.imshow(edges,cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

plt.show()
```



Q2

Question 2 – Laplacian of Gaussian (LoG) Write a Python program to: • Read an image. • Convert it to grayscale. • Apply Gaussian Blur. • Apply the Laplacian operator. • Display the detected edges. Change the Gaussian kernel size and observe the effect

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Read the image
image = cv2.imread("dog.jpg")

# Step 2: Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Apply Gaussian Blur
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Step 4: Apply Laplacian operator
laplacian = cv2.Laplacian(blur, cv2.CV_64F)

# Convert result to uint8 so that it can be displayed
laplacian = cv2.convertScaleAbs(laplacian)
```

```
plt.figure(figsize=(10,5))

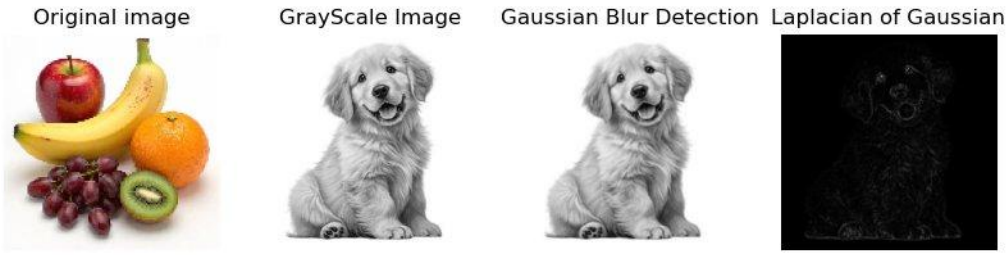
plt.subplot(1,4,1)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))
plt.title("Original image")
plt.axis("off")

plt.subplot(1, 4, 2)
plt.imshow(gray,cmap="gray")
plt.title("GrayScale Image")
plt.axis("off")

plt.subplot(1, 4,3)
plt.imshow(blur,cmap="gray")
plt.title("Gaussian Blur Detection")
plt.axis("off")

plt.subplot(1,4,4)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian of Gaussian")
plt.axis("off")

plt.show()
```



Question 3 – Difference of Gaussian (DoG)

Write a Python program to: • Apply Gaussian blur using two different sigma values. • Subtract the two blurred images. • Display the resulting DoG image. • Compare the DoG result with the original image.

```
import cv2
import matplotlib.pyplot as plt
```

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Read the image
image = cv2.imread("dog.jpg")

# Step 2: Convert the image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Apply Gaussian Blur with two different sigma values
blur1 = cv2.GaussianBlur(gray, (5, 5), 1)
blur2 = cv2.GaussianBlur(gray, (5, 5), 2)

# Step 4: Subtract the two blurred images
dog = blur1.astype(float) - blur2.astype(float)

# Step 5: Convert DoG result to displayable format
dog = cv2.convertScaleAbs(dog)
```

```
# Step 6: Display original and DoG images
plt.figure(figsize=(10, 5))

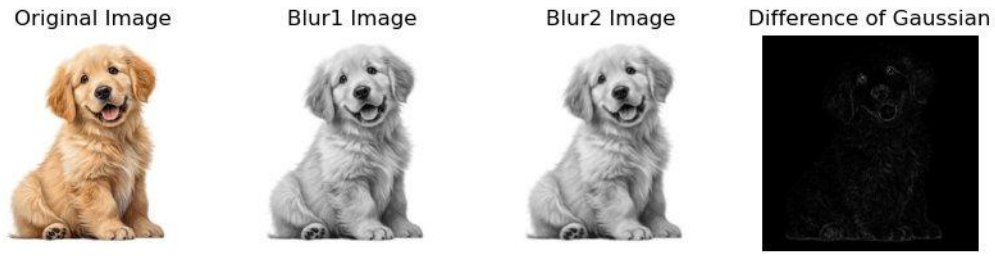
# Original image
plt.subplot(1, 4, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,4,2)
plt.title("Blur1 Image")
plt.imshow(blur1,cmap="gray")
plt.axis("off")

plt.subplot(1,4,3)
plt.title("Blur2 Image")
plt.imshow(blur2,cmap="gray")
plt.axis("off")
```

```
# DoG image
plt.subplot(1, 4,4)
plt.imshow(dog, cmap="gray")
plt.title("Difference of Gaussian")
plt.axis("off")

plt.show()
```



Question 4 – Comparison of Edge Detectors

Apply Canny, LoG, and DoG to the same image. Display all results and compare: • Edge quality • Noise • Number of detected edges • Computational requirements

```
import cv2
import matplotlib.pyplot as plt
import time

# -----------------------------------
# Step 1: Read the image
# -----------------------------------

image = cv2.imread("dog.jpg")

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# -----------------------------------
# Step 2: Canny Edge Detection
# -----------------------------------

start = time.time()

# Apply Gaussian Blur
canny_blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Apply Canny
canny = cv2.Canny(canny_blur, 50, 150)

canny_time = time.time() - start



# -----------------------------------
# Step 3: Laplacian of Gaussian (LoG)
# -----------------------------------

start = time.time()
```

```
# Apply Gaussian Blur
log_blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Apply Laplacian
log = cv2.Laplacian(log_blur, cv2.CV_64F)

# Convert result to displayable format
log = cv2.convertScaleAbs(log)

log_time = time.time() - start



# ----------------------------------
# Step 4: Difference of Gaussian (DoG)
# ----------------------------------

start = time.time()

# Apply Gaussian Blur with sigma = 1
dog_blur1 = cv2.GaussianBlur(gray, (5, 5), 1)

# Apply Gaussian Blur with sigma = 2
dog_blur2 = cv2.GaussianBlur(gray, (5, 5), 2)

# Subtract the two blurred images
dog = dog_blur1.astype(float) - dog_blur2.astype(float)

# Convert result to displayable format
dog = cv2.convertScaleAbs(dog)

dog_time = time.time() - start
```

```
# ----------------------------------
# Step 5: Display all results
# ----------------------------------

plt.figure(figsize=(12, 8))

# Original Image
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

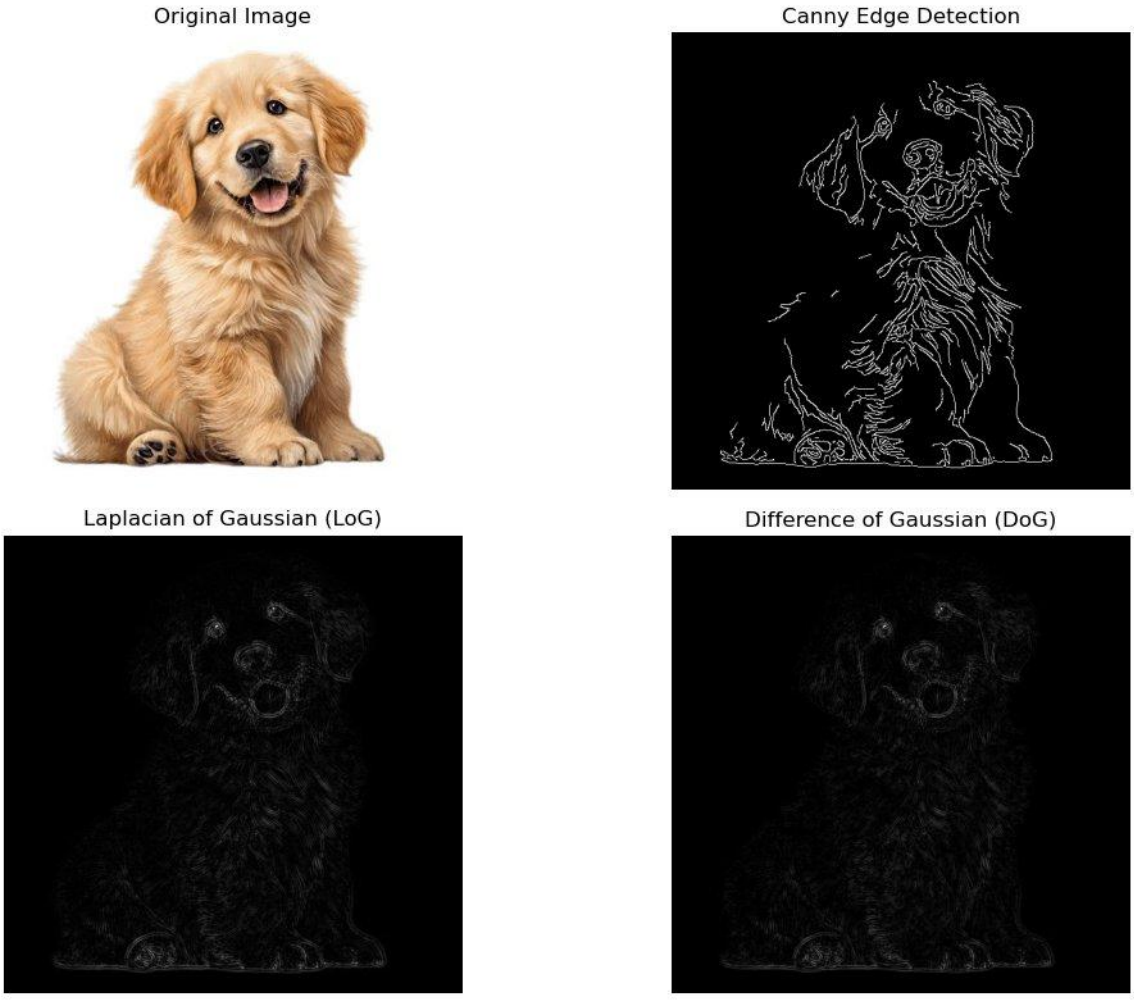
# Canny
plt.subplot(2, 2, 2)
plt.imshow(canny, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

# LoG
plt.subplot(2, 2, 3)
plt.imshow(log, cmap="gray")
plt.title("Laplacian of Gaussian (LoG)")
plt.axis("off")

# DoG
```

```
plt.subplot(2, 2, 4)
plt.imshow(dog, cmap="gray")
plt.title("Difference of Gaussian (DoG)")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# ----------------------------------
# Step 6: Count detected edges
# ----------------------------------

canny_edges = cv2.countNonZero(canny)
log_edges = cv2.countNonZero(log)
dog_edges = cv2.countNonZero(dog)

print("Number of detected edges:")
print("Canny:", canny_edges)
print("LoG  :", log_edges)
print("DoG  :", dog_edges)

print("\nComputational time:")
print("Canny:", canny_time, "seconds")
print("LoG  :", log_time, "seconds")
```

```
print("DoG  :", dog_time, "seconds")
```

```
Number of detected edges:
Canny: 12455
LoG  : 138757
DoG  : 116969

Computational time:
Canny: 0.004315614700317383 seconds
LoG  : 0.0051152706146240234 seconds
DoG  : 0.010370016098022461 seconds
```

Question 5 – Hough Line Transform

Write a Python program to: • Detect edges using Canny. • Apply the Standard Hough Line Transform. • Detect straight lines. • Draw the detected lines on the original image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Step 1: Read the image
image = cv2.imread("input.jpg")

# Make a copy of the original image
result = image.copy()

# Step 2: Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Detect edges using Canny
edges = cv2.Canny(gray, 50, 150)

# Step 4: Apply Standard Hough Line Transform
lines = cv2.HoughLines(edges, 1, np.pi / 180, 150)

# Step 5: Draw the detected lines
if lines is not None:

    for line in lines:
        rho, theta = line[0]

        # Convert polar coordinates to two points
        a = np.cos(theta)
        b = np.sin(theta)

        x0 = a * rho
        y0 = b * rho

        x1 = int(x0 + 1000 * (-b))
        y1 = int(y0 + 1000 * (a))

        x2 = int(x0 - 1000 * (-b))
        y2 = int(y0 - 1000 * (a))

        # Draw the line
```

```
        cv2.line(result, (x1, y1), (x2, y2), (0, 0, 255), 2)
```

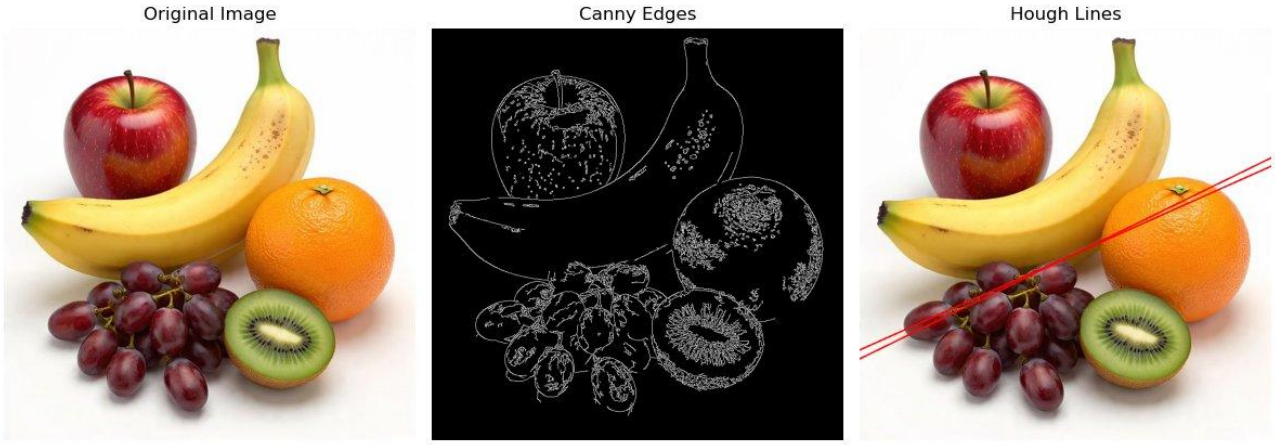
```
# Step 6: Display the results
plt.figure(figsize=(12, 5))

# Original image
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# Canny edges
plt.subplot(1, 3, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

# Detected lines
plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Hough Lines")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Question 6 – Probabilistic Hough Transform

Apply HoughLinesP() to an edge-detected image and draw the detected line segments. Change the threshold, minimum line length, and maximum line gap.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Step 1: Read the image
image = cv2.imread("input.jpg")

# Make a copy to draw lines
result = image.copy()
```

```
# Step 2: Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Detect edges using Canny
edges = cv2.Canny(gray,50,150)

# Step 4: Apply Probabilistic Hough Line Transform
lines = cv2.HoughLinesP(edges,1,np.pi/180,threshold=50,minLineLength=50,maxLineGap=10)

# Step 5: Draw detected line segments
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line
        cv2.line(result, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

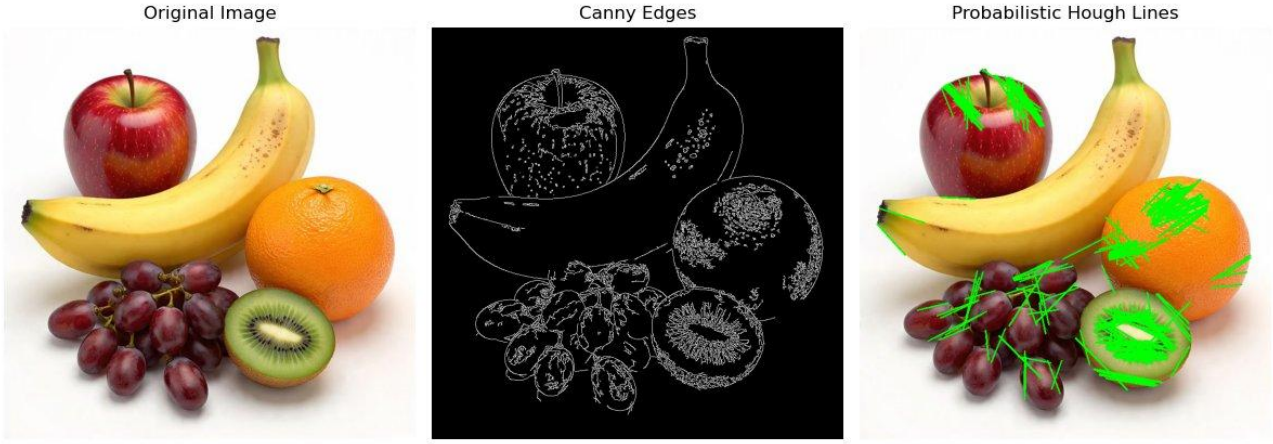
```
# Step 6: Display results
plt.figure(figsize=(12, 5))

# Original image
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# Canny edges
plt.subplot(1, 3, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

# Hough line segments
plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Probabilistic Hough Lines")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Question 7 – Hough Transform for Road/Lane Detection

Use a road image and: • Detect edges using Canny. • Apply the Probabilistic Hough Transform. • Detect lane lines. • Draw the detected lines on the road image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Read the road image
image = cv2.imread("road.jpg")

# Make a copy of the original image
result = image.copy()

# Step 2: Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Step 3: Apply Canny Edge Detection
edges = cv2.Canny(gray, 50, 150)

# Step 4: Apply Probabilistic Hough Transform
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    threshold=50,
    minLineLength=50,
    maxLineGap=100
)

# Step 5: Draw detected lines
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line.flatten()
        cv2.line(result, (x1, y1), (x2, y2), (0, 0, 255), 3)

# Step 6: Display the results
plt.figure(figsize=(15, 5))

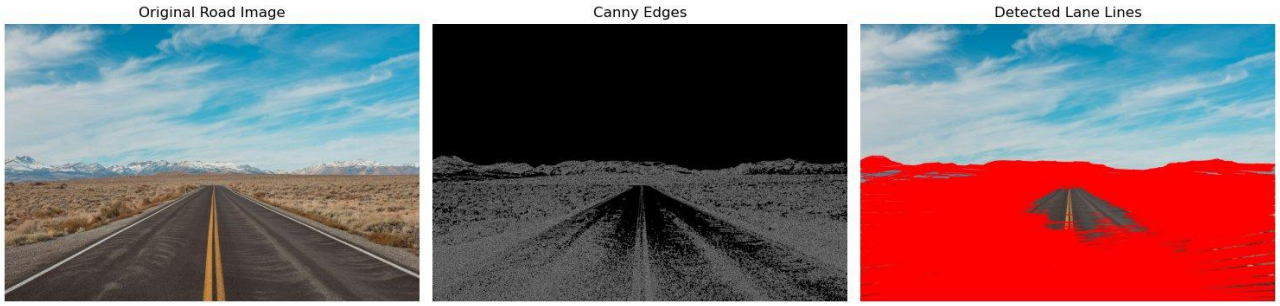
# Original road image
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Road Image")
plt.axis("off")

# Canny edges
plt.subplot(1, 3, 2)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

# Detected lane lines
plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Detected Lane Lines")
plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```




```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Read image
image = cv2.imread("road.jpg")
if image is None:
    raise FileNotFoundError("Could not find or open 'road.jpg'. Check the file path.")

result = image.copy()
height, width = image.shape[:2]

# 2. Grayscale + Gaussian Blur (reduces high-frequency noise)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# 3. Canny Edge Detection
edges = cv2.Canny(blurred, 50, 150)

# 4. Region of Interest (ROI) Mask
# Create a triangular mask covering only the road in front of the camera
mask = np.zeros_like(edges)
roi_vertices = np.array([
    [
        (int(width * 0.1), height),          # Bottom-left
        (int(width * 0.45), int(height * 0.6)), # Top-left apex
        (int(width * 0.55), int(height * 0.6)), # Top-right apex
        (int(width * 0.9), height)           # Bottom-right
    ]
], dtype=np.int32)

cv2.fillPoly(mask, roi_vertices, 255)
masked_edges = cv2.bitwise_and(edges, mask)

# 5. Probabilistic Hough Transform on masked edges
lines = cv2.HoughLinesP(
    masked_edges,
    rho=1,
    theta=np.pi / 180,
    threshold=50,
    minLineLength=100,  # Increased to reject short dashes/noise
    maxLineGap=50       # Bridges broken dashes into single lines
)

# 6. Draw detected lines
if lines is not None:
```

```
    for line in lines:
        x1, y1, x2, y2 = line.flatten()
        cv2.line(result, (x1, y1), (x2, y2), (0, 0, 255), 3)

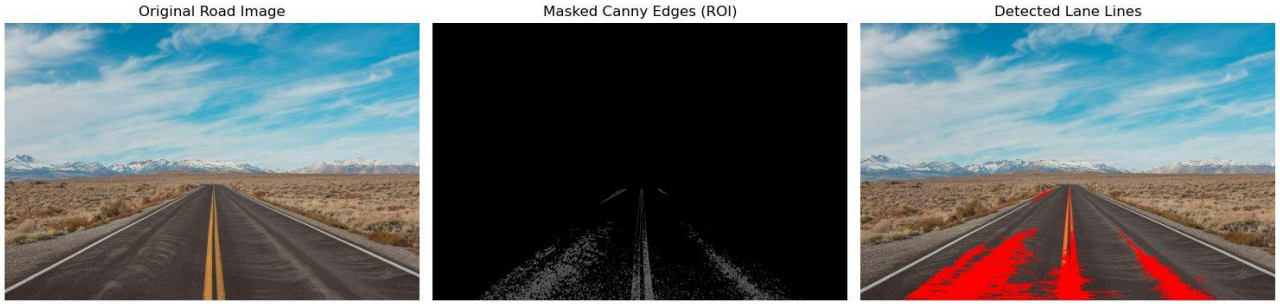
# 7. Display results
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Road Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(masked_edges, cmap="gray")
plt.title("Masked Canny Edges (ROI)")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Detected Lane Lines")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Question 8 – Harris Corner Detection

Write a Python program to: • Convert an image to grayscale. • Apply Harris Corner Detection. • Apply a suitable threshold. • Mark the detected corners on the image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Step 1: Read the image
image = cv2.imread("dog.jpg")

# Make a copy of the original image
result = image.copy()

# Step 2: Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Convert grayscale image to float32
gray = np.float32(gray)
```

```
# Step 3: Apply Harris Corner Detection
corners = cv2.cornerHarris(gray,blockSize=2,ksize=3,k=0.04)
corners = cv2.dilate(corners, None)

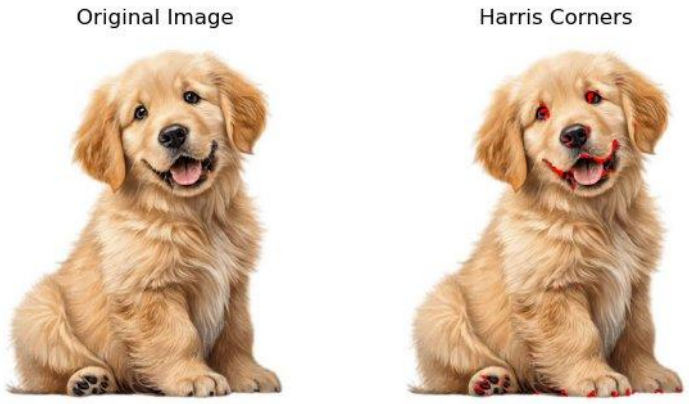
# Step 4: Apply threshold
threshold = 0.01 * corners.max()

# Mark detected corners
result[corners > threshold] = [0, 0, 255]
```

```
# Original image
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

# Image with detected corners
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Harris Corners")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Question 9 – Harris Corner Parameter Analysis

Apply Harris Corner Detection using different values of: • Block size • Sobel aperture size • Harris detector parameter k • Compare the detected corners.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt



# ------------------------------------
# Step 1: Read the image
# ------------------------------------

image = cv2.imread("input.jpg")

# Convert image to grayscale
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Harris requires float32
gray = np.float32(gray)



# ----------------------------------
# Function for Harris Corner Detection
# ----------------------------------

def detect_corners(block_size, sobel_size, k_value):

    # Apply Harris Corner Detection
    corners = cv2.cornerHarris(gray,block_size,sobel_size,k_value)
    corners = cv2.dilate(corners, None)

    # Make a copy of original image
    result = image.copy()

    # Apply threshold
    threshold = 0.01 * corners.max()

    # Mark corners in red
    result[corners > threshold] = [0, 0, 255]

    return result



# ----------------------------------
# Step 2: Different Block Sizes
# ----------------------------------

block1 = detect_corners(2, 3, 0.04)
block2 = detect_corners(5, 3, 0.04)
block3 = detect_corners(7, 3, 0.04)



# ----------------------------------
# Step 3: Different Sobel Sizes
# ----------------------------------

sobel1 = detect_corners(2, 3, 0.04)
sobel2 = detect_corners(2, 5, 0.04)
sobel3 = detect_corners(2, 7, 0.04)



# ----------------------------------
# Step 4: Different k Values
# ----------------------------------

k1 = detect_corners(2, 3, 0.02)
k2 = detect_corners(2, 3, 0.04)
k3 = detect_corners(2, 3, 0.06)



# ----------------------------------
# Step 5: Display Block Size Results
```

```
# ----------------------------------

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(block1, cv2.COLOR_BGR2RGB))
plt.title("Block Size = 2")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(block2, cv2.COLOR_BGR2RGB))
plt.title("Block Size = 5")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(block3, cv2.COLOR_BGR2RGB))
plt.title("Block Size = 7")
plt.axis("off")

plt.tight_layout()
plt.show()



# ----------------------------------
# Step 6: Display Sobel Size Results
# ----------------------------------

plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(sobel1, cv2.COLOR_BGR2RGB))
plt.title("Sobel Size = 3")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(sobel2, cv2.COLOR_BGR2RGB))
plt.title("Sobel Size = 5")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(sobel3, cv2.COLOR_BGR2RGB))
plt.title("Sobel Size = 7")
plt.axis("off")

plt.tight_layout()
plt.show()



# ----------------------------------
# Step 7: Display k Value Results
# ----------------------------------

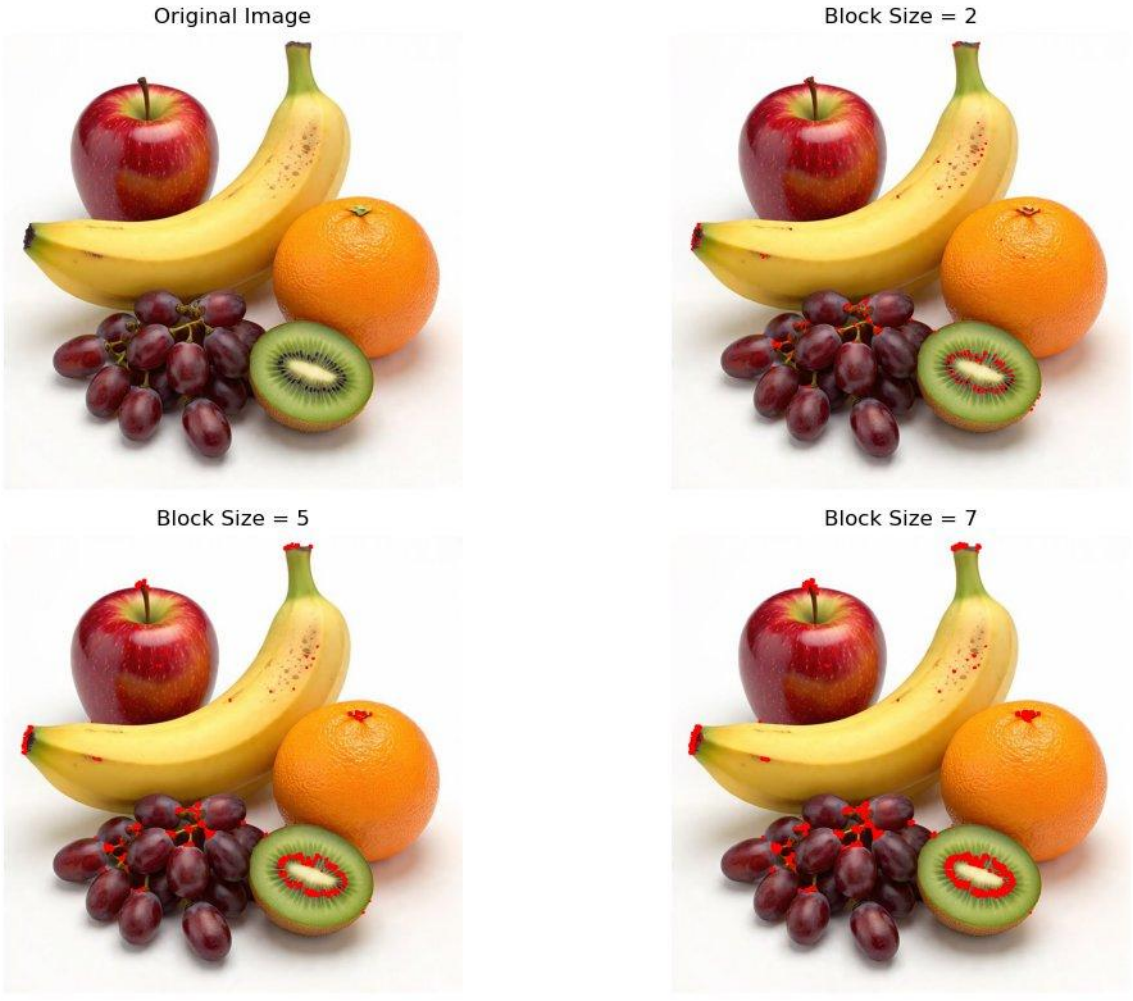
plt.figure(figsize=(12, 8))
```

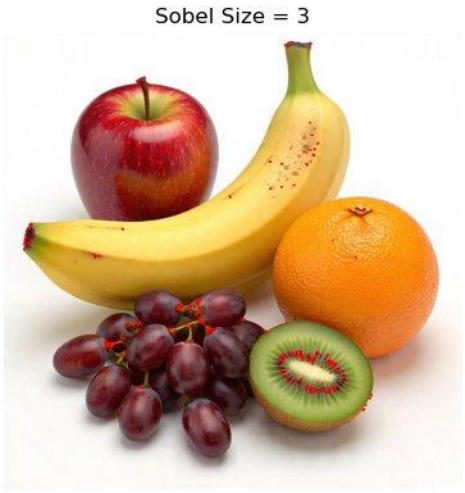
```
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(k1, cv2.COLOR_BGR2RGB))
plt.title("k = 0.02")
plt.axis("off")

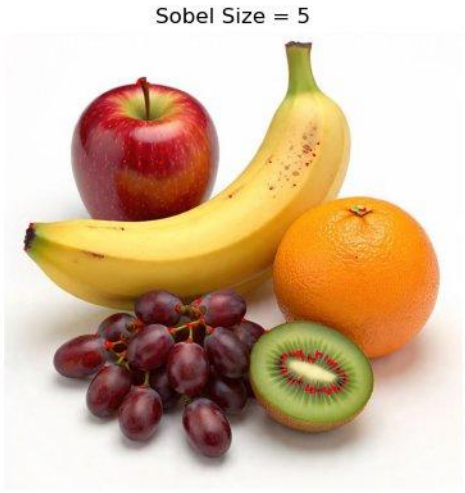
plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(k2, cv2.COLOR_BGR2RGB))
plt.title("k = 0.04")
plt.axis("off")

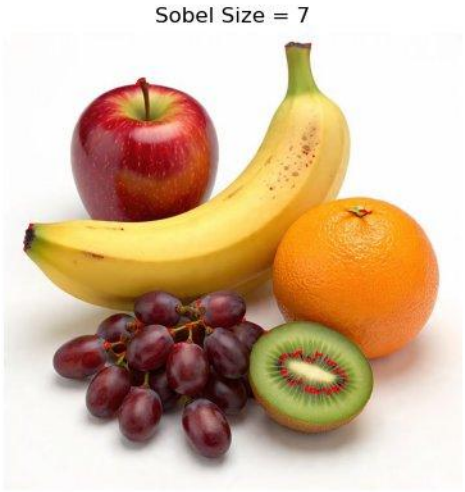
plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(k3, cv2.COLOR_BGR2RGB))
plt.title("k = 0.06")
plt.axis("off")

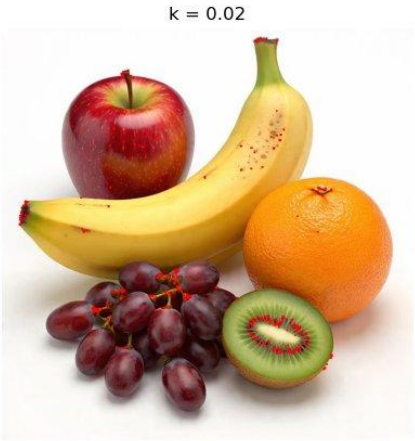
plt.tight_layout()
plt.show()
```

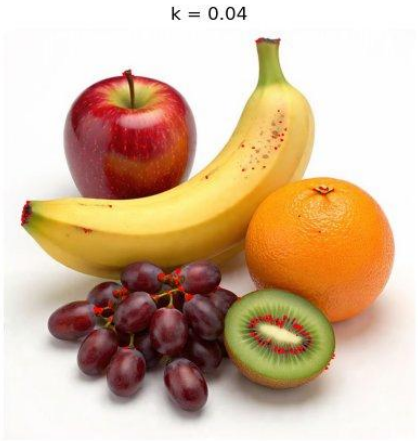


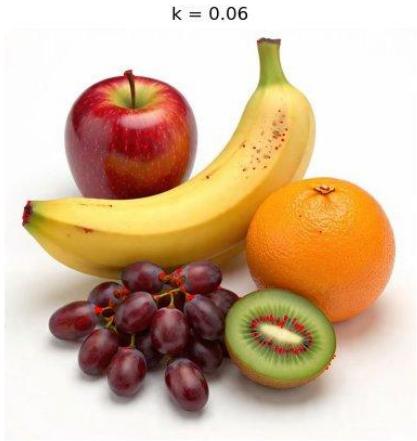












Question 10 – Harris Corner Detection on Shapes

Create an image containing squares, rectangles, and triangles. Apply Harris Corner Detection and mark the detected corners.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# ----------------------------------
# Step 1: Create a blank image
# ----------------------------------

image = np.zeros((500, 700, 3), dtype=np.uint8)

# Make background white
image[:] = 255

# ----------------------------------
# Step 2: Draw a square
# ----------------------------------
cv2.rectangle(image,(50, 100),(200, 250),(0, 0, 0),3)

# ----------------------------------
# Step 3: Draw a rectangle
# ----------------------------------
cv2.rectangle(image,(280, 100),(500, 250),(0, 0, 0),3)

# ----------------------------------
# Step 4: Draw a triangle
# ----------------------------------

triangle_points = np.array([
    [550, 250],
    [620, 100],
    [680, 250]
])

cv2.polylines(image,[triangle_points],True,(0, 0, 0),3)

# ----------------------------------
# Step 5: Convert to grayscale
# ----------------------------------

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Harris requires float32
gray = np.float32(gray)

# ----------------------------------
# Step 6: Apply Harris Corner Detection
# ----------------------------------

corners = cv2.cornerHarris(gray,blockSize=2,ksize=3,k=0.04)
corners = cv2.dilate(corners,None)

# ----------------------------------
# Step 7: Apply threshold
# ----------------------------------

threshold = 0.01 * corners.max()



# ----------------------------------
# Step 8: Mark detected corners
# ----------------------------------
```

```
result = image.copy()

result[corners > threshold] = [0, 0, 255]
```

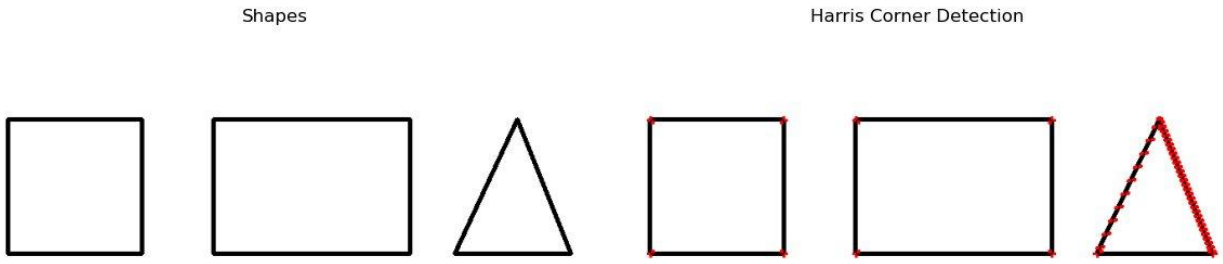
```
# ----------------------------------
# Step 9: Display results
# ----------------------------------

plt.figure(figsize=(12, 5))

# Original shapes
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Shapes")
plt.axis("off")

# Detected corners
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Harris Corner Detection")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Question 11 – Hessian-Based Interest Point Detection

Write a program to calculate the Hessian matrix/determinant at different image locations and identify strong interest points. Display the detected points on the image.

```
import cv2
import numpy as np
```

```
# Step 1: Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
```

```
    exit()

# Copy for drawing
output = img.copy()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = np.float32(gray)

# Step 3: Compute second-order derivatives (Hessian components)
Ixx = cv2.Sobel(gray, cv2.CV_64F, 2, 0, ksize=3)
Iyy = cv2.Sobel(gray, cv2.CV_64F, 0, 2, ksize=3)
Ixy = cv2.Sobel(gray, cv2.CV_64F, 1, 1, ksize=3)

# Step 4: Compute determinant of Hessian
detH = (Ixx * Iyy) - (Ixy ** 2)

# Step 5: Normalize for visualization
detH_norm = cv2.normalize(detH, None, 0, 255, cv2.NORM_MINMAX)
detH_norm = np.uint8(detH_norm)

# Step 6: Threshold to find strong interest points
threshold = 0.01 * detH.max()

for y in range(detH.shape[0]):
    for x in range(detH.shape[1]):
        if detH[y, x] > threshold:
            cv2.circle(output, (x, y), 2, (0, 0, 255), -1)
```

```
# Step 7: Display results
cv2.imshow("Original Image", img)
cv2.imshow("Hessian Determinant", detH_norm)
cv2.imshow("Interest Points", output)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Question 12 – Hessian-Affine Feature Detection

Detect interest regions using a Hessian-Affine based method or a suitable computer-vision library. Display the detected keypoints/regions and explain how the method provides robustness to scale and affine changes.

```
import cv2
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("input.jpg")

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and calculate descriptors
```

```
keypoints, descriptors = sift.detectAndCompute(gray, None)

# Draw detected keypoints
result = cv2.drawKeypoints(
    image,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

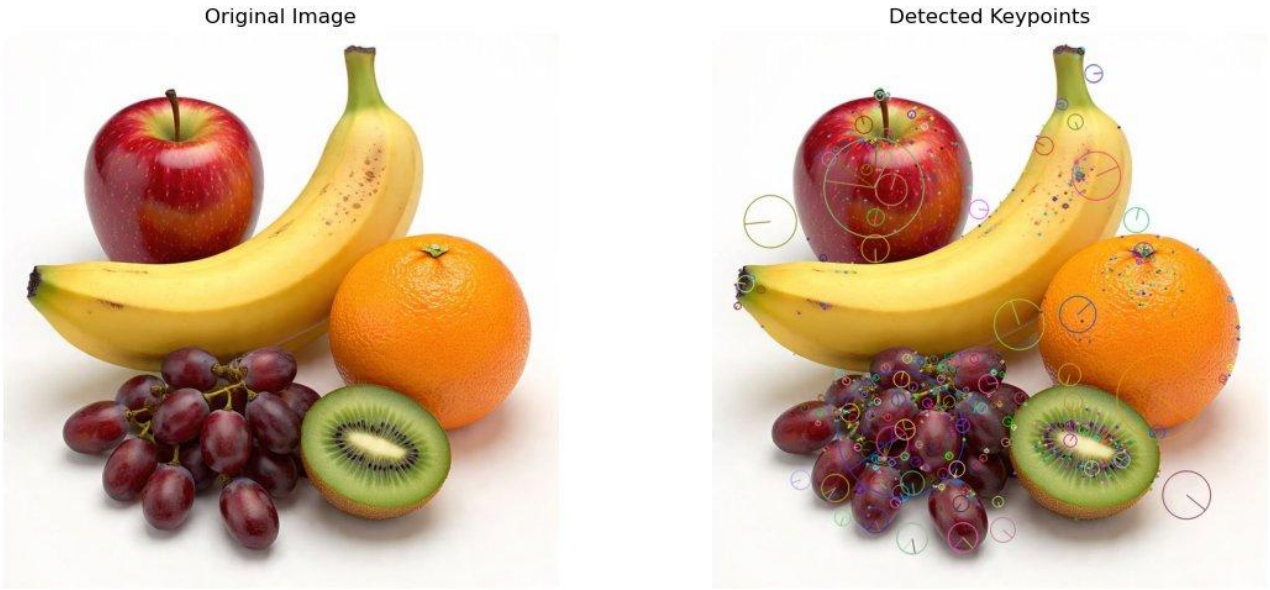
# Display results
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Detected Keypoints")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Number of detected keypoints:", len(keypoints))
```




```
Number of detected keypoints: 762
```

Question 13 – Image Gradient and Orientation

Use Sobel operators to calculate: • Gx and Gy • Gradient magnitude • Gradient orientation • Display the resulting images.

```
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("input.jpg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
Gx = cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)
Gy = cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)

magnitude = cv2.magnitude(Gx.astype(np.float32),Gy.astype(np.float32))
orientation = np.arctan2(Gy,Gx)*180/np.pi

Gx_display = cv2.convertScaleAbs(Gx)
Gy_display = cv2.convertScaleAbs(Gy)

magnitude_display = cv2.normalize(magnitude,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
orientation_display = cv2.normalize(orientation,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
```

```
# Display results
plt.figure(figsize=(12, 8))

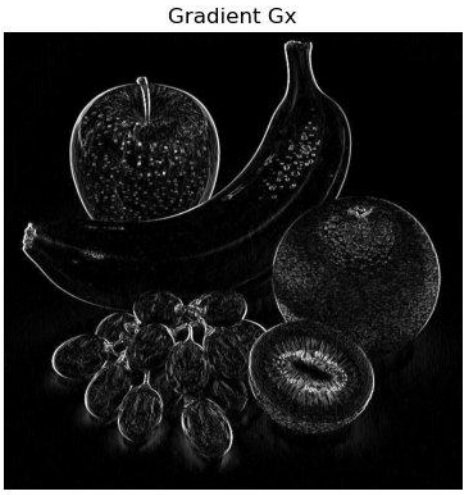
plt.subplot(2, 2, 1)
plt.imshow(Gx_display, cmap="gray")
plt.title("Gradient Gx")
plt.axis("off")

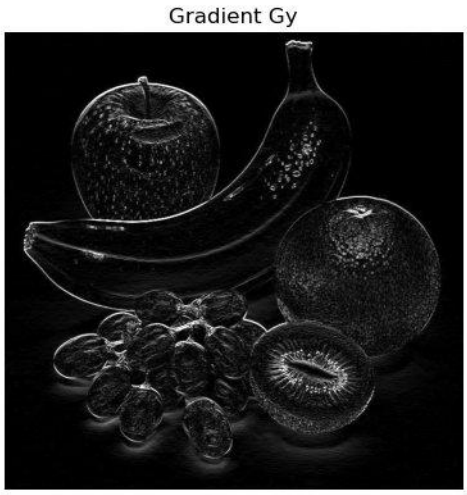
plt.subplot(2, 2, 2)
plt.imshow(Gy_display, cmap="gray")
plt.title("Gradient Gy")
plt.axis("off")

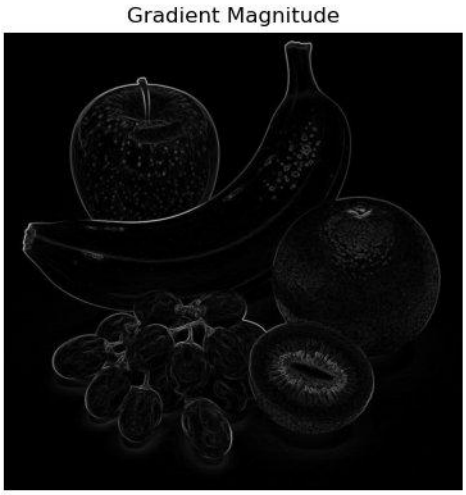
plt.subplot(2, 2, 3)
plt.imshow(magnitude_display, cmap="gray")
plt.title("Gradient Magnitude")
plt.axis("off")

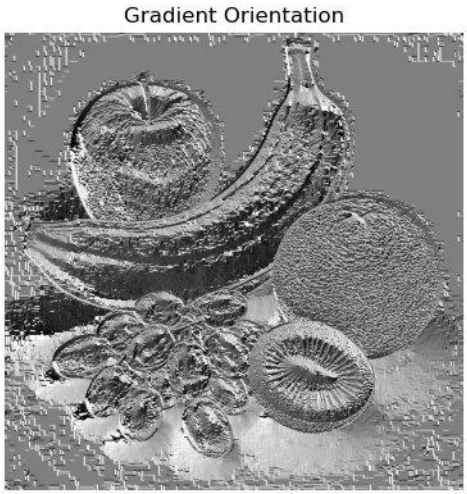
plt.subplot(2, 2, 4)
plt.imshow(orientation_display, cmap="gray")
plt.title("Gradient Orientation")
plt.axis("off")

plt.tight_layout()
plt.show()
```









Question 14 – Orientation Histogram

Divide an image into small cells and: • Calculate gradient magnitude and orientation. • Divide orientations into bins. • Construct an orientation histogram for each cell. • Display the histogram.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Read image
img = cv2.imread('road.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Step 3: Compute gradients using Sobel
Gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
Gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

```
# Step 4: Magnitude and Orientation
magnitude = np.sqrt(Gx**2 + Gy**2)
orientation = np.arctan2(Gy, Gx) * (180 / np.pi)

# Convert orientation to range 0–180
orientation[orientation < 0] += 180

# Step 5: Divide into cells (e.g., 8x8)
cell_size = 8
h, w = gray.shape

# Number of bins
bins = 9
bin_width = 180 // bins

histograms = []

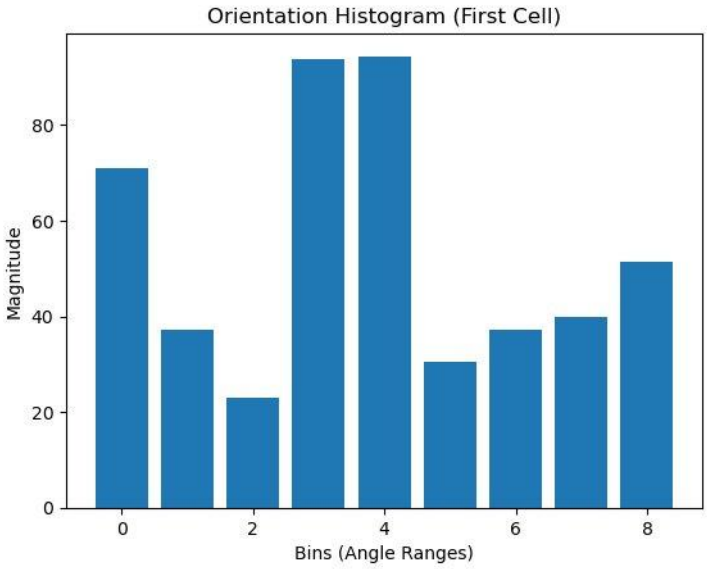
# Step 6: Compute histogram for each cell
for i in range(0, h, cell_size):
    for j in range(0, w, cell_size):
        cell_mag = magnitude[i:i+cell_size, j:j+cell_size]
        cell_ori = orientation[i:i+cell_size, j:j+cell_size]

        hist = np.zeros(bins)

        for x in range(cell_mag.shape[0]):
            for y in range(cell_mag.shape[1]):
                angle = cell_ori[x, y]
                mag = cell_mag[x, y]
                bin_idx = int(angle // bin_width)
                if bin_idx >= bins:
                    bin_idx = bins - 1
                hist[bin_idx] += mag

        histograms.append(hist)

# Step 7: Display histogram of first cell
plt.bar(range(bins), histograms[0])
plt.title("Orientation Histogram (First Cell)")
plt.xlabel("Bins (Angle Ranges)")
plt.ylabel("Magnitude")
plt.show()
```



Question 15 – SIFT Keypoint Detection

Write a Python program to: • Read an image. • Create a SIFT detector. • Detect keypoints. • Extract descriptors. • Draw the keypoints on the image. • Display the number of detected keypoints.

```
import cv2

# Step 1: Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Step 3: Create SIFT detector
sift = cv2.SIFT_create()

# Step 4: Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

# Step 5: Draw keypoints on image
output = cv2.drawKeypoints(
    img, keypoints, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

# Step 6: Print number of keypoints
print("Number of keypoints detected:", len(keypoints))

# Step 7: Display result
cv2.imshow("SIFT Keypoints", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Number of keypoints detected: 762
```

Question 16 – SIFT Feature Matching

Take two images containing the same object. Use SIFT to:

- Detect keypoints.
- Extract descriptors.
- Match descriptors using BFMatcher.
- Display the matched keypoints.

```
import cv2

# Step 1: Read two images
img1 = cv2.imread('input.jpg')   # Query image
img2 = cv2.imread('road.jpg')   # Train image

if img1 is None or img2 is None:
    print("Error: One or both images not found!")
    exit()

# Step 2: Convert to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Step 3: Create SIFT detector
sift = cv2.SIFT_create()

# Step 4: Detect keypoints and descriptors
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

# Step 5: BFMatcher (Brute Force Matcher)
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)

# Step 6: Match descriptors
matches = bf.match(des1, des2)

# Step 7: Sort matches by distance (best matches first)
matches = sorted(matches, key=lambda x: x.distance)

# Step 8: Draw top matches
matched_img = cv2.drawMatches(
    img1, kp1,
```

```
    img2, kp2,
    matches[:50],   # top 50 matches
    None,
    flags=2
)

# Step 9: Display results
print("Number of keypoints in Image 1:", len(kp1))
print("Number of keypoints in Image 2:", len(kp2))
print("Total Matches:", len(matches))

cv2.imshow("SIFT Matches", matched_img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Number of keypoints in Image 1: 762
Number of keypoints in Image 2: 72645
Total Matches: 576
```

Question 17 – SIFT and Scale Variation

Resize the same image to different scales and apply SIFT to each image. Compare the detected

keypoints and explain why SIFT is useful for scale changes.

```
import cv2

# Step 1: Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Create scaled versions
small = cv2.resize(img, None, fx=0.5, fy=0.5)
large = cv2.resize(img, None, fx=1.5, fy=1.5)

# Step 3: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray_small = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
gray_large = cv2.cvtColor(large, cv2.COLOR_BGR2GRAY)

# Step 4: Create SIFT detector
sift = cv2.SIFT_create()

# Step 5: Detect keypoints
kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(gray_small, None)
kp3, des3 = sift.detectAndCompute(gray_large, None)
```

```
# Step 6: Draw keypoints
out1 = cv2.drawKeypoints(img, kp1, None)
out2 = cv2.drawKeypoints(small, kp2, None)
out3 = cv2.drawKeypoints(large, kp3, None)

# Step 7: Print number of keypoints
print("Original image keypoints:", len(kp1))
print("Small image keypoints:", len(kp2))
print("Large image keypoints:", len(kp3))

# Step 8: Display results
cv2.imshow("Original", out1)
cv2.imshow("Scaled Down (0.5x)", out2)
cv2.imshow("Scaled Up (1.5x)", out3)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Original image keypoints: 762
Small image keypoints: 273
Large image keypoints: 1690
```

Question 18 – SURF Feature Detection

Use a suitable OpenCV/contributed implementation to:

- Detect SURF keypoints.
- Extract descriptors.
- Draw the keypoints.
- Report the number of detected keypoints.

```
import cv2

img = cv2.imread('input.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Use SIFT instead of SURF
sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)

output = cv2.drawKeypoints(img, keypoints, None)
```

```
print("Number of keypoints:", len(keypoints))

cv2.imshow("Keypoints", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Number of keypoints: 762
```

Question 19 – SIFT vs SURF

Apply SIFT and SURF to the same image and compare:

- Number of keypoints
- Descriptor size
- Processing time
- Scale robustness
- Rotation robustness
- Present your observations in a table.

```
import cv2
import time

# Read image
img = cv2.imread('input.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -------------------------
# 🔹 SIFT
# -------------------------
start = time.time()

sift = cv2.SIFT_create()
kp_sift, des_sift = sift.detectAndCompute(gray, None)

sift_time = time.time() - start
```

```
# -------------------------
#  🔹 SURF (Try-Except)
# -------------------------
try:
    start = time.time()

    surf = cv2.xfeatures2d.SURF_create(400)
    kp_surf, des_surf = surf.detectAndCompute(gray, None)

    surf_time = time.time() - start
    surf_available = True

except:
    kp_surf, des_surf = [], None
    surf_time = 0
    surf_available = False

# -------------------------
#  🔹 Results
# -------------------------
print("SIFT Keypoints:", len(kp_sift))
print("SIFT Descriptor Size:", des_sift.shape if des_sift is not None else "None")
print("SIFT Time:", sift_time)

if surf_available:
    print("SURF Keypoints:", len(kp_surf))
    print("SURF Descriptor Size:", des_surf.shape)
    print("SURF Time:", surf_time)
else:
    print("SURF not available (patented / disabled)")

# Draw SIFT result
out_sift = cv2.drawKeypoints(img, kp_sift, None)

cv2.imshow("SIFT Keypoints", out_sift)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
SIFT Keypoints: 762
SIFT Descriptor Size: (762, 128)
SIFT Time: 0.21123862266540527
SURF not available (patented / disabled)
```

Question 20 – HOG Feature Extraction

Write a Python program to:

- Resize the image.
- Calculate image gradients.
- Divide the image into cells.
- Calculate orientation histograms.
- Create HOG features.
- Display the HOG visualization.

```
import cv2
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure

# Step 1: Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Resize image
img = cv2.resize(img, (128, 128))

# Step 3: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Step 4: Apply HOG
features, hog_image = hog(
    gray,
    orientations=9,           # number of bins
    pixels_per_cell=(8, 8),   # cell size
    cells_per_block=(2, 2),   # block size
    visualize=True,
    block_norm='L2-Hys'
)

# Step 5: Improve visualization
hog_image_rescaled = exposure.rescale_intensity(hog_image, in_range=(0, 10))
```

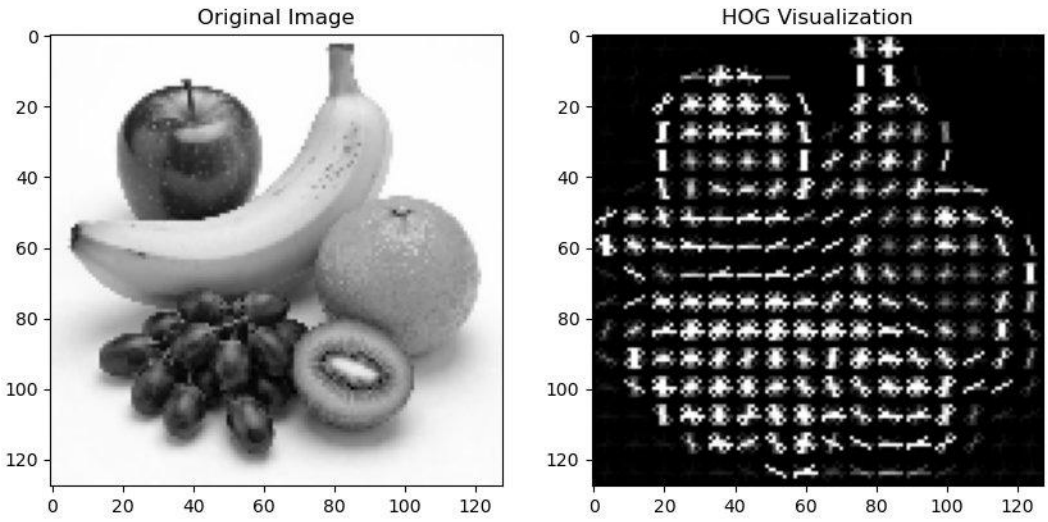
```
# Step 6: Display results
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")

plt.subplot(1,2,2)
plt.imshow(hog_image_rescaled, cmap='gray')
plt.title("HOG Visualization")

plt.show()

# Step 7: Print feature vector size
print("HOG feature vector length:", len(features))
```




```
HOG feature vector length: 8100
```

Question 21 – HOG for Human Detection

Use the OpenCV HOG descriptor and its pre-trained people detector to detect humans in an image.

Draw bounding boxes around detected persons.

```
import cv2
import matplotlib.pyplot as plt

# Read image
image = cv2.imread("people.jpg")
```

```
# Create HOG descriptor
hog = cv2.HOGDescriptor()

# Use OpenCV's pre-trained people detector
hog.setSVMDetector(
    cv2.HOGDescriptor_getDefaultPeopleDetector()
)

# Detect people
boxes, weights = hog.detectMultiScale(
    image,
    winStride=(8, 8),
    padding=(8, 8),
    scale=1.05
)

# Draw bounding boxes
for (x, y, w, h) in boxes:
    cv2.rectangle(
        image,
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        2
    )

# Display result
plt.figure(figsize=(12, 7))
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("HOG Human Detection")
plt.axis("off")
plt.show()

# Display number of detected people
print("Number of detected persons:", len(boxes))
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[12], line 8
      5 image = cv2.imread("people.jpg")
      7 # Create HOG descriptor
----> 8 hog = cv2.HOGDescriptor()
     10 # Use OpenCV's pre-trained people detector
     11 hog.setSVMDetector(
     12     cv2.HOGDescriptor_getDefaultPeopleDetector()
     13 )

AttributeError: module 'cv2' has no attribute 'HOGDescriptor'
```

Question 22 – GLOH Feature Descriptor

Implement or use an appropriate library implementation of GLOH to extract local image features.

Explain:

- Spatial regions
- Orientation bins
- Feature vector formation
- Difference between GLOH and SIFT

```
import cv2

# Step 1: Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Step 3: Use SIFT (approximation of GLOH)
sift = cv2.SIFT_create()

# Step 4: Detect keypoints and descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

# Step 5: Draw keypoints
output = cv2.drawKeypoints(img, keypoints, None)

# Step 6: Display
print("Number of keypoints:", len(keypoints))
print("Descriptor shape:", descriptors.shape)

cv2.imshow("GLOH (SIFT Approximation)", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```
Number of keypoints: 685
Descriptor shape: (685, 128)
```

Question 23 – SIFT vs GLOH

Extract SIFT and GLOH features from the same image and compare:

- Descriptor representation
- Feature dimension
- Computational cost
- Robustness
- Applications

```
import cv2
import time
import numpy as np

# Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -------------------------
#  🔹 SIFT
# -------------------------
start = time.time()
sift = cv2.SIFT_create()
kp_sift, des_sift = sift.detectAndCompute(gray, None)
sift_time = time.time() - start

print("SIFT Keypoints:", len(kp_sift))
print("SIFT Descriptor Shape:", des_sift.shape)
print("SIFT Time:", sift_time)



# -------------------------
#  🔹 GLOH (Approx Implementation)
# -------------------------
start = time.time()
```

```
# Step 1: Gradients
Gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
Gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

magnitude = np.sqrt(Gx**2 + Gy**2)
orientation = np.arctan2(Gy, Gx) * (180 / np.pi)
orientation[orientation < 0] += 360

# Step 2: Log-polar bins
num_radial = 3
num_angular = 8
num_bins = 16

h, w = gray.shape
cx, cy = w // 2, h // 2

hist = np.zeros((num_radial, num_angular, num_bins))

# Step 3: Fill histogram
for i in range(h):
    for j in range(w):
        dx = j - cx
        dy = i - cy

        r = np.sqrt(dx**2 + dy**2)
        theta = (np.arctan2(dy, dx) * 180 / np.pi) % 360

        r_bin = min(int(r / (max(h, w)/num_radial)), num_radial - 1)
        a_bin = int(theta / (360 / num_angular))

        o_bin = int(orientation[i, j] / (360 / num_bins))
        if o_bin >= num_bins:
            o_bin = num_bins - 1

        hist[r_bin, a_bin, o_bin] += magnitude[i, j]

# Step 4: Flatten feature vector
gloh_feature = hist.flatten()
gloh_time = time.time() - start

print("\nGLOH Feature Vector Length:", len(gloh_feature))
print("GLOH Time:", gloh_time)
```

```
SIFT Keypoints: 762
SIFT Descriptor Shape: (762, 128)
SIFT Time: 0.13205623626708984

GLOH Feature Vector Length: 384
GLOH Time: 9.411133527755737
```

Question 24 – Complete Feature Detection Pipeline

For a given image, apply the following pipeline:

• Image → Grayscale → Gaussian Blur → Canny → Harris → SIFT → HOG

• Display the output of each stage and briefly explain the purpose of each technique.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure

# Step 1: Read image
img = cv2.imread('input.jpg')

if img is None:
    print("Error: Image not found!")
    exit()

# Resize for consistency
img = cv2.resize(img, (256, 256))

# -------------------------
#  🔹 Grayscale
# -------------------------
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -------------------------
#  🔹 Gaussian Blur
# -------------------------
blur = cv2.GaussianBlur(gray, (5,5), 0)

# -------------------------
#  🔹 Canny Edge Detection
# -------------------------
canny = cv2.Canny(blur, 100, 200)

# -------------------------
#  🔹 Harris Corner Detection
# -------------------------
gray_float = np.float32(gray)
harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)

harris_img = img.copy()
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]

# -------------------------
#  🔹 SIFT
# -------------------------
```

```
sift = cv2.SIFT_create()
kp, des = sift.detectAndCompute(gray, None)
sift_img = cv2.drawKeypoints(img, kp, None)

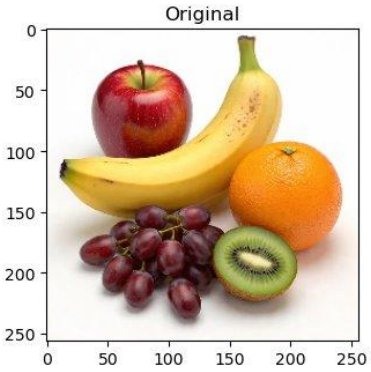
# -------------------------
# 🔹 HOG
# -------------------------
features, hog_image = hog(
    gray,
    orientations=9,
    pixels_per_cell=(8,8),
    cells_per_block=(2,2),
    visualize=True
)

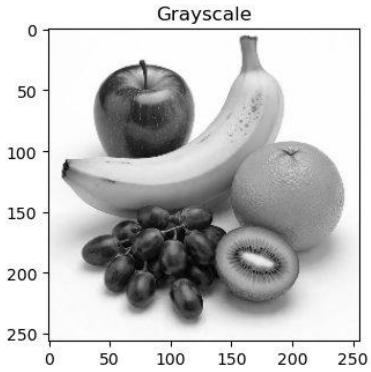
hog_image = exposure.rescale_intensity(hog_image, in_range=(0,10))

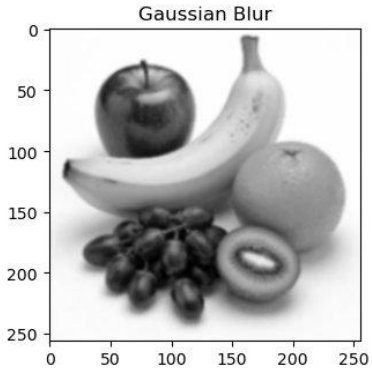
# -------------------------
# 🔹 Display all stages
# -------------------------
plt.figure(figsize=(12,10))

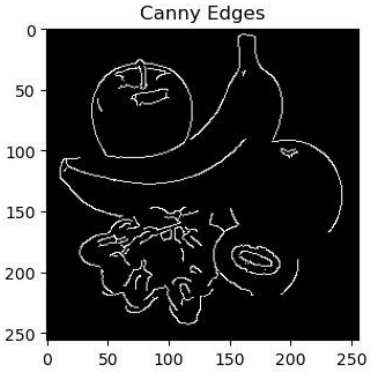
plt.subplot(3,3,1), plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)), plt.title("Original")
plt.subplot(3,3,2), plt.imshow(gray, cmap='gray'), plt.title("Grayscale")
plt.subplot(3,3,3), plt.imshow(blur, cmap='gray'), plt.title("Gaussian Blur")
plt.subplot(3,3,4), plt.imshow(canny, cmap='gray'), plt.title("Canny Edges")
plt.subplot(3,3,5), plt.imshow(cv2.cvtColor(harris_img, cv2.COLOR_BGR2RGB)), plt.title("Harris
Corners")
plt.subplot(3,3,6), plt.imshow(cv2.cvtColor(sift_img, cv2.COLOR_BGR2RGB)), plt.title("SIFT
Keypoints")
plt.subplot(3,3,7), plt.imshow(hog_image, cmap='gray'), plt.title("HOG")

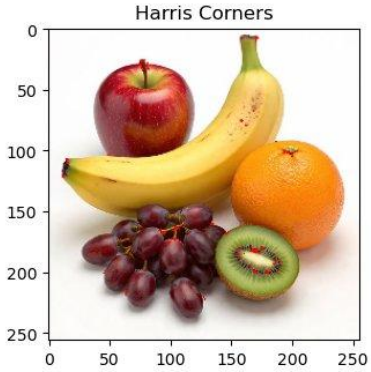
plt.tight_layout()
plt.show()
```

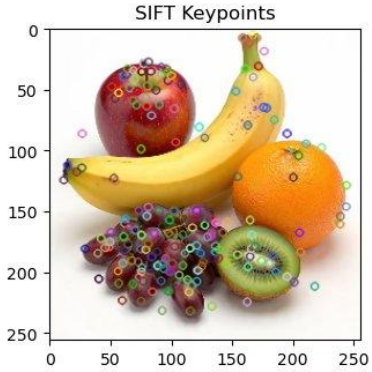

Original

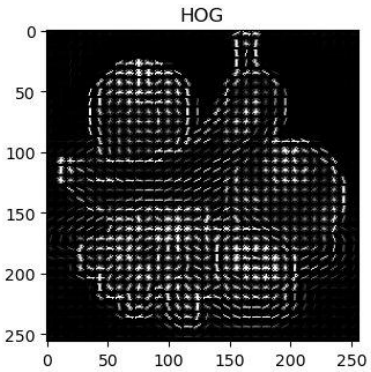

Grayscale


Gaussian Blur


Canny Edges


Harris Corners


SIFT Keypoints


HOG

Question 25 – Comparative Study of Computer Vision Features

For the same image, apply the following techniques and prepare a comparison based on type, what is

detected/described, scale invariance, rotation invariance, and main application:

- Canny
- LoG
- DoG
- Hough Transform
- Harris
- Hessian-Affine
- SIFT
- SURF
- HOG
- GLOH

```
import cv2
import numpy as np
import time
from skimage.feature import hog
from skimage import exposure

# Read image
img = cv2.imread('input.jpg')
```

```
if img is None:
    print("Error: Image not found!")
    exit()

img = cv2.resize(img, (256, 256))
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

results = []

# -------------------------
# 🔹 CANNY
# -------------------------
start = time.time()
canny = cv2.Canny(gray, 100, 200)
results.append(("Canny", "Edge detector", "Edges", "No", "No", "Edge detection", time.time()-
start))

# -------------------------
# 🔹 LoG
# -------------------------
start = time.time()
blur = cv2.GaussianBlur(gray, (5,5), 0)
log = cv2.Laplacian(blur, cv2.CV_64F)
results.append(("LoG", "Edge detector", "Edges", "Limited", "No", "Edge detection", time.time()-
start))

# -------------------------
# 🔹 DoG
# -------------------------
start = time.time()
g1 = cv2.GaussianBlur(gray, (5,5), 1)
g2 = cv2.GaussianBlur(gray, (5,5), 2)
dog = g1 - g2
results.append(("DoG", "Feature detector", "Blobs/features", "Yes", "No", "Feature detection",
time.time()-start))

# -------------------------
# 🔹 HOUGH
# -------------------------
start = time.time()
edges = cv2.Canny(gray, 100, 200)
lines = cv2.HoughLines(edges, 1, np.pi/180, 100)
results.append(("Hough", "Line detector", "Lines", "No", "No", "Line detection", time.time()-
start))

# -------------------------
# 🔹 HARRIS
# -------------------------
start = time.time()
gray_float = np.float32(gray)
harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)
results.append(("Harris", "Corner detector", "Corners", "No", "Partial", "Corner detection",
time.time()-start))

# -------------------------
# 🔹 SIFT
# -------------------------
start = time.time()
```

```
sift = cv2.SIFT_create()
kp_sift, des_sift = sift.detectAndCompute(gray, None)
results.append(("SIFT", "Descriptor", "Keypoints", "Yes", "Yes", "Matching", time.time()-start))

# -------------------------
# 🔹 SURF (may fail)
# -------------------------
try:
    start = time.time()
    surf = cv2.xfeatures2d.SURF_create(400)
    kp_surf, des_surf = surf.detectAndCompute(gray, None)
    results.append(("SURF", "Descriptor", "Keypoints", "Yes", "Yes", "Matching", time.time()-
start))
except:
    results.append(("SURF", "Descriptor", "Not Available", "Yes", "Yes", "Matching", 0))

# -------------------------
# 🔹 HOG
# -------------------------
start = time.time()
features, hog_img = hog(gray, visualize=True)
results.append(("HOG", "Descriptor", "Shape/gradient", "Limited", "Limited", "Object detection",
time.time()-start))

# -------------------------
# 🔹 GLOH (Approx)
# -------------------------
start = time.time()
Gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0)
Gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1)
mag = np.sqrt(Gx**2 + Gy**2)
results.append(("GLOH", "Descriptor", "Local features", "Yes", "Yes", "Feature matching",
time.time()-start))

# -------------------------
# 🔹 Print Table
# -------------------------
print("\nComparison Table:\n")
print("{:<10} {:<15} {:<20} {:<10} {:<10} {:<20} {:<10}".format(
    "Technique","Type","Detects","Scale","Rotation","Application","Time"))

for r in results:
    print("{:<10} {:<15} {:<20} {:<10} {:<10} {:<20} {:.5f}".format(*r))
```

```
Comparison Table:

Technique  Type            Detects              Scale      Rotation   Application          Time
Canny      Edge detector   Edges                No         No         Edge detection
0.00141
LoG        Edge detector   Edges                Limited    No         Edge detection
0.00148
DoG        Feature detector Blobs/features       Yes        No         Feature detection
0.00102
Hough      Line detector   Lines                No         No         Line detection
0.00645
Harris     Corner detector Corners              No         Partial    Corner detection
0.00200
```

SIFT	Descriptor	Keypoints	Yes	Yes	Matching
0.02894					
SURF	Descriptor	Not Available	Yes	Yes	Matching
0.00000					
HOG	Descriptor	Shape/gradient	Limited	Limited	Object detection
0.34402					
GLOH	Descriptor	Local features	Yes	Yes	Feature matching
0.00196					

P3

1. Implement Gaussian and Laplacian image pyramids to analyze vehicles of different sizes in a traffic image.

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Read the traffic image
img = cv2.imread("traffic.jpg")

# Check whether the image was loaded
if img is None:
    print("Error: Image not found!")
    exit()

# Convert BGR to RGB for correct display in Matplotlib
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# Step 2: Create the Gaussian pyramid
gaussian = [img]

for i in range(3):
    img = cv2.pyrDown(img)
    gaussian.append(img)

# Step 3: Create the Laplacian pyramid
laplacian = []

for i in range(len(gaussian) - 1):
    # Get the current image
    current = gaussian[i]

    # Get the next smaller image
    smaller = gaussian[i + 1]

    # Expand the smaller image to the current image size
    expanded = cv2.pyrUp(
        smaller,
        dstsize=(current.shape[1], current.shape[0])
    )

    # Subtract to obtain image details
    lap = cv2.subtract(current, expanded)

    laplacian.append(lap)
```

```
# Step 4: Display the Gaussian pyramid
plt.figure(figsize=(12, 8))

for i, image in enumerate(gaussian):
    plt.subplot(2, 2, i + 1)
    plt.imshow(image)
    plt.title("Gaussian Level " + str(i))
    plt.axis("off")
```

```
plt.tight_layout()
plt.show()

# Step 5: Display the Laplacian pyramid
plt.figure(figsize=(12, 8))

for i, image in enumerate(laplacian):
    plt.subplot(2, 2, i + 1)
    plt.imshow(image)
    plt.title("Laplacian Level " + str(i))
    plt.axis("off")

plt.tight_layout()
plt.show()
```





2. Implement an image-pyramid-based approach to detect faces appearing at different scales in a photograph.

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Load the photograph
img = cv2.imread("people2.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 2: Load the pre-trained face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)

# Step 3: Create a copy for drawing rectangles
result = img.copy()

# Step 4: Set the initial image scale
scale = 1.0
```

```
# Step 5: Detect faces at different image scales
while True:

    # Resize the original grayscale image
    resized = cv2.resize(gray,None,fx=scale,fy=scale)

    # Detect faces in the resized image
    faces = face_cascade.detectMultiScale(resized,scaleFactor=1.1,minNeighbors=15,minSize=(30,
30))

    # Draw rectangles on the original image
    for (x, y, w, h) in faces:

        # Convert coordinates to original image size
        x = int(x / scale)
        y = int(y / scale)
        w = int(w / scale)
        h = int(h / scale)

        cv2.rectangle(result,(x, y),(x + w, y + h),(0, 255, 0),2)

    # Stop when the image becomes too small
    if resized.shape[1] < 100 or resized.shape[0] < 100:
        break

    # Reduce the image size for the next level
    scale = scale * 0.8
```

```
# Step 6: Display the final result
result = cv2.cvtColor(result, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 7))
plt.imshow(result)
plt.title("Face Detection Using Image Pyramid")
plt.axis("off")
plt.show()
```



3. Implement multi-scale image processing to detect a person appearing at different distances from a surveillance camera.

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Load the surveillance image
img = cv2.imread("crowd.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Create a copy for displaying results
result = img.copy()
```

```
# Step 3: Load the pre-trained HOG person detector
hog = cv2.HOGDescriptor()

hog.setSVMDetector(
    cv2.HOGDescriptor_getDefaultPeopleDetector()
)

# Step 4: Set image scale
scale = 1.0

# Step 5: Detect people at different scales
while True:

    # Resize the image
    resized = cv2.resize(img,None,fx=scale,fy=scale)
```

```
    # Detect people in the resized image
    boxes, weights = hog.detectMultiScale(resized,winStride=(8,8),padding=(8,8),scale=1.05)

    # Step 6: Convert coordinates to original image size
    for (x, y, w, h) in boxes:

        x = int(x / scale)
        y = int(y / scale)
        w = int(w / scale)
        h = int(h / scale)

        # Draw a rectangle around the person
        cv2.rectangle(result,(x, y),(x + w, y + h),(0, 255, 0),2)

        # Stop when the resized image becomes too small
        if resized.shape[1] < 160 or resized.shape[0] < 160:
            break

        # Reduce the image size for the next level
        scale = scale * 0.8
```

```
# Step 7: Display the final result
result = cv2.cvtColor(result,cv2.COLOR_BGR2RGB)

plt.figure(figsize=(10, 7))
plt.imshow(result)
plt.title("Multi-Scale Person Detection")
plt.axis("off")
plt.show()
```

4. Generate different-resolution versions of a scanned document using Gaussian pyramids and analyze the effect of resolution on character readability.

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Load the scanned document
img = cv2.imread("document.png")

if img is None:
    print("Error: Image not found!")
    exit()

# Convert the image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 2: Create the Gaussian pyramid
pyramid = [gray]

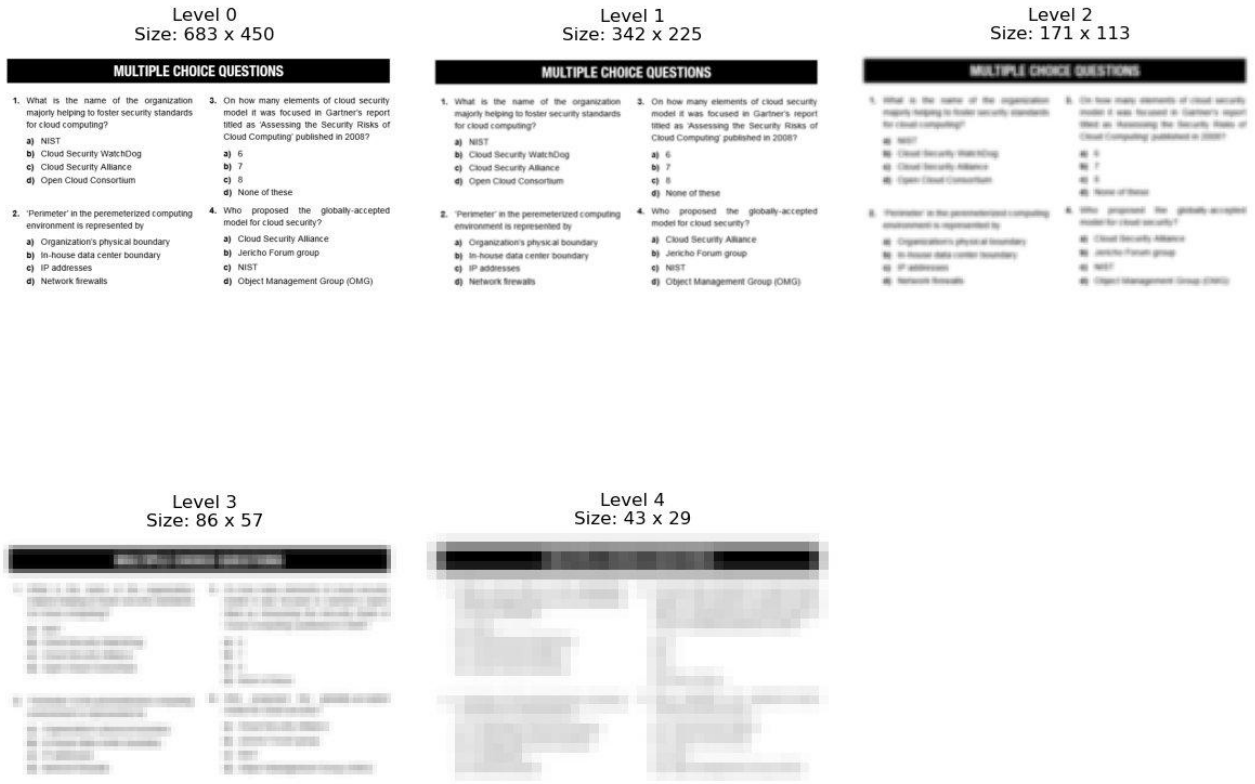
for i in range(4):
    # Reduce the image size by half
    gray = cv2.pyrDown(gray)
    pyramid.append(gray)
```

```
# Step 3: Display all resolution levels
plt.figure(figsize=(12, 10))
```

```
for i, image in enumerate(pyramid):
    plt.subplot(2, 3, i + 1)
    plt.imshow(image, cmap="gray")
    # Display image size
    height, width = image.shape

    plt.title("Level " + str(i) + "\nSize: " + str(width) + " x " + str(height))
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# # Step 4: Save each resolution version
# for i, image in enumerate(pyramid):
#     filename = "document_level_" + str(i) + ".jpg"
#     cv2.imwrite(filename, image)

# print("All resolution images saved successfully!")
```

5. Implement Gaussian derivative filters to detect road boundaries and lane markings in a road image

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Read the road image
img = cv2.imread("road.jpg")
```

```
if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 3: Apply Gaussian blur to remove noise
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Step 4: Apply Gaussian derivative filters using Sobel
# Detect horizontal brightness changes (vertical edges)
sobel_x = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)

# Detect vertical brightness changes (horizontal edges)
sobel_y = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)

# Step 5: Combine the edge information
edges = cv2.magnitude(sobel_x, sobel_y)

# Convert edges to 8-bit image
edges = cv2.convertScaleAbs(edges)

# Step 6: Apply thresholding
_, binary = cv2.threshold(edges, 50, 255, cv2.THRESH_BINARY)
```

```
# Step 7: Display results
plt.figure(figsize=(12, 8))

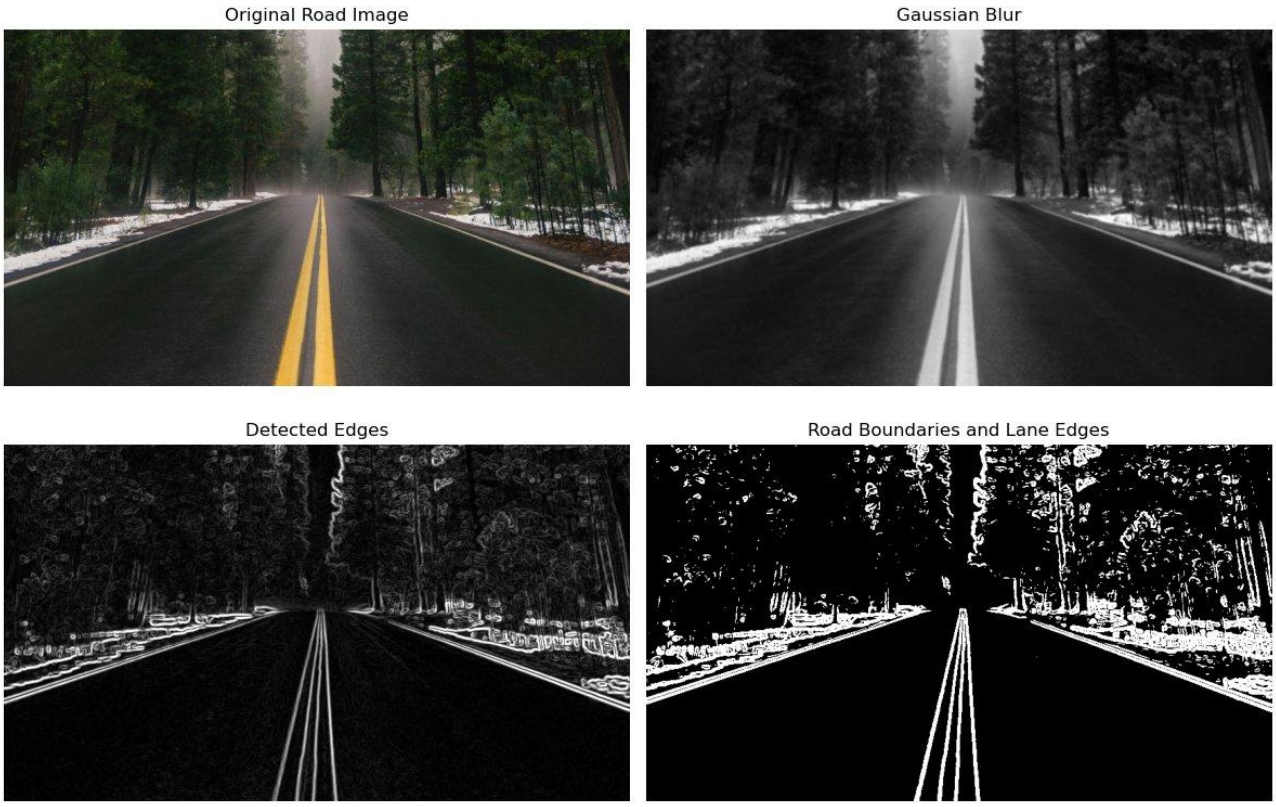
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Road Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(edges, cmap="gray")
plt.title("Detected Edges")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(binary, cmap="gray")
plt.title("Road Boundaries and Lane Edges")
plt.axis("off")

plt.tight_layout()
plt.show()
```



6. Implement first-order and second-order Gaussian derivative filters to detect boundaries and defects on a manufactured product.

```
import cv2
import matplotlib.pyplot as plt

# Step 1: Load the product image
img = cv2.imread("product3.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 3: Apply Gaussian blur
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Step 4: First-order derivative using Sobel
first_order = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
first_order = cv2.convertScaleAbs(first_order)

# Step 5: Second-order derivative using Laplacian
second_order = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)
second_order = cv2.convertScaleAbs(second_order)
```

```
# Step 6: Display all images
plt.figure(figsize=(10, 8))
```

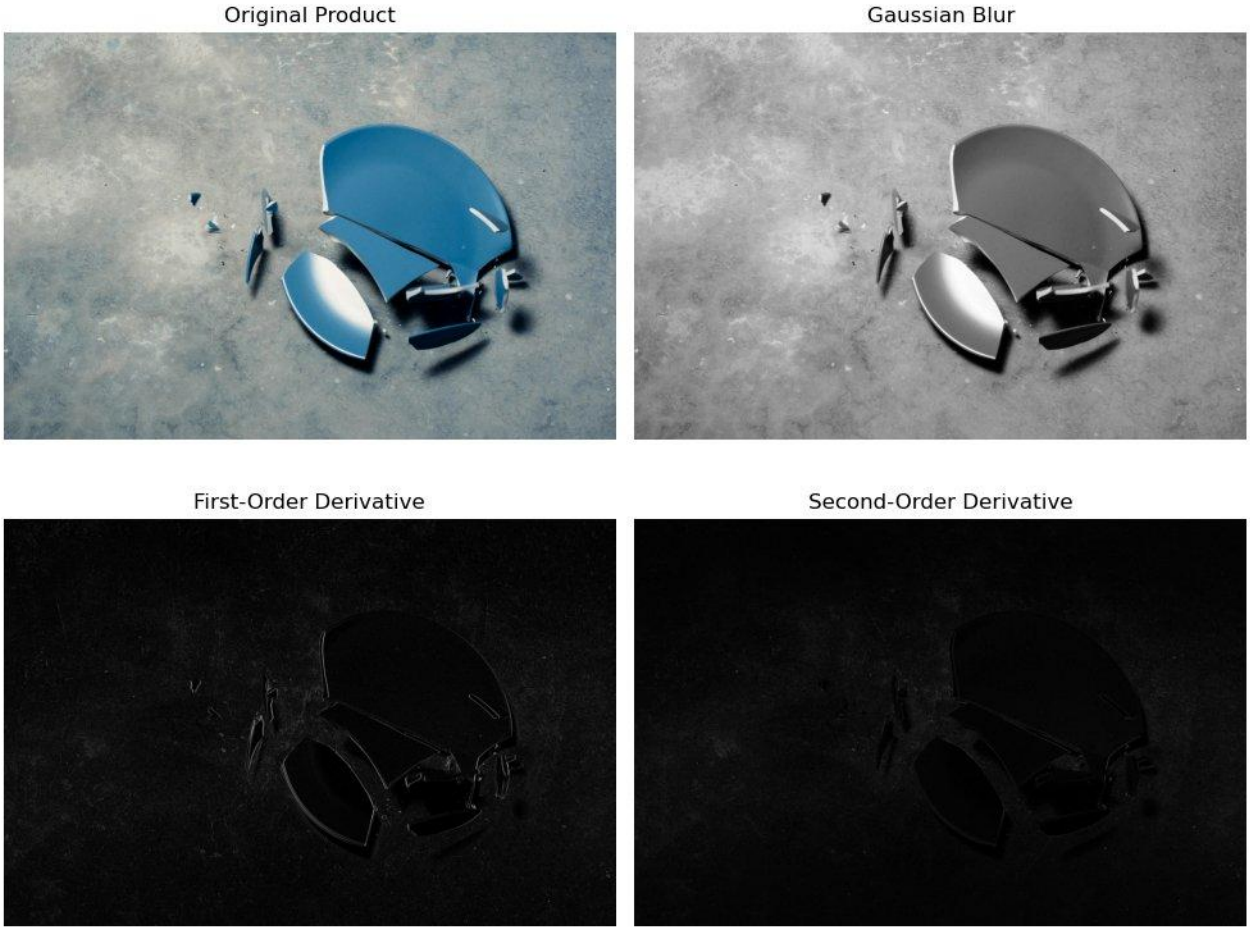
```
plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Product")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(blur, cmap="gray")
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(first_order, cmap="gray")
plt.title("First-Order Derivative")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(second_order, cmap="gray")
plt.title("Second-Order Derivative")
plt.axis("off")

plt.tight_layout()
plt.show()
```



7. Implement Gabor filters with different orientations and frequencies to identify defective regions in a fabric image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load the fabric image
img = cv2.imread("fabric.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Convert image to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 2: Create Gabor filters
orientations = [0, 45, 90, 135]
frequencies = [0.1, 0.2]

# Display original image
plt.figure(figsize=(12, 10))

plt.subplot(3, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Fabric")
plt.axis("off")

# Step 3: Apply filters
plot_no = 2

for angle in orientations:
    for freq in frequencies:

        # Convert angle to radians
        theta = np.deg2rad(angle)

        # Create Gabor kernel
        kernel = cv2.getGaborKernel((10,10),4.0,theta,1.0 / freq,0.5,0,ktype=cv2.CV_32F)

        # Apply filter to fabric image
        filtered = cv2.filter2D(gray, cv2.CV_32F, kernel)

        # Convert result to displayable image
        filtered = cv2.convertScaleAbs(filtered)

        # Display filtered image
        plt.subplot(3, 3, plot_no)
        plt.imshow(filtered, cmap="gray")
        plt.title(str(angle) + " deg, f=" + str(freq))
        plt.axis("off")

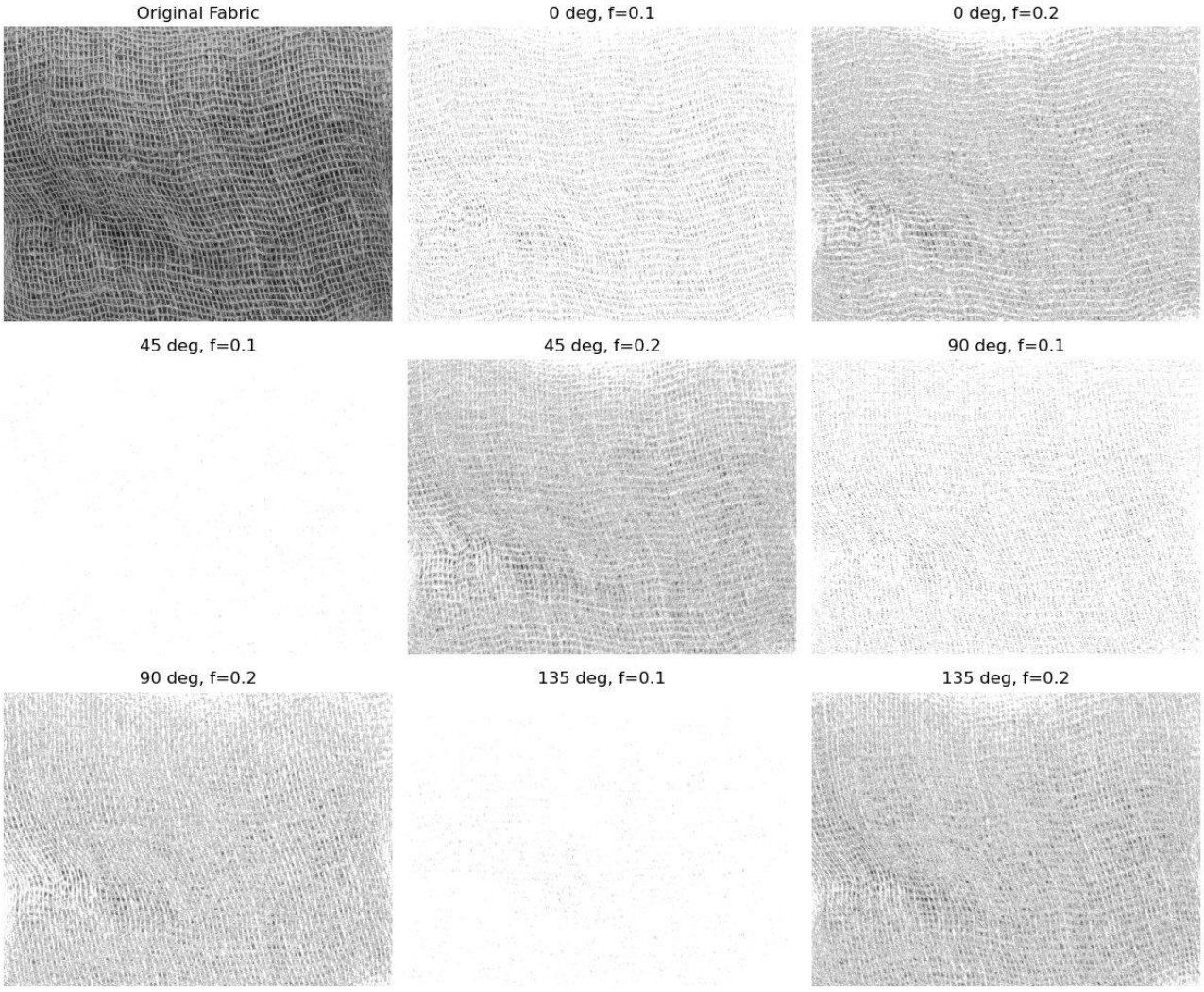
        plot_no += 1

        # Stop after filling 9 plots
        if plot_no > 9:
```

```
            break

    if plot_no > 9:
        break

plt.tight_layout()
plt.show()
```



8. Apply Gabor filters to enhance fingerprint ridge patterns and visualize the resulting texture information.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load fingerprint image
img = cv2.imread("fingerprint.png")

if img is None:
    print("Error: Image not found!")
    exit()
```

```
# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 3: Resize image
gray = cv2.resize(gray, (300, 300))

# Step 4: Apply Gaussian blur
blur = cv2.GaussianBlur(gray, (3, 3), 0)

# Step 5: Create a Gabor filter
theta = np.deg2rad(45)  # Ridge direction
frequency = 0.1

kernel = cv2.getGaborKernel(
    (21,21),       # Kernel size
    4.0,            # Sigma
    theta,          # Orientation
    1 / frequency,  # Wavelength
    5,            # Aspect ratio
    0,              # Phase offset
    ktype=cv2.CV_32F
)

# Step 6: Apply Gabor filter
filtered = cv2.filter2D(blur, cv2.CV_32F, kernel)

# Step 7: Normalize result for display
filtered = cv2.normalize(filtered, None, 0, 255, cv2.NORM_MINMAX)
filtered = filtered.astype(np.uint8)
```

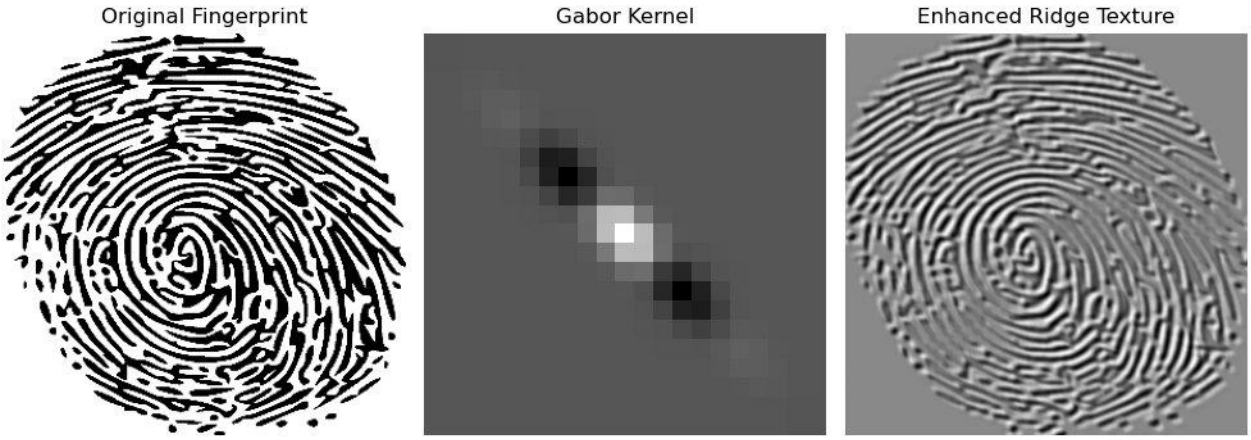
```
# Step 8: Display images
plt.figure(figsize=(10, 5))

plt.subplot(1, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Fingerprint")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(kernel, cmap="gray")
plt.title("Gabor Kernel")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(filtered, cmap="gray")
plt.title("Enhanced Ridge Texture")
plt.axis("off")

plt.tight_layout()
plt.show()
```



9. Implement Gabor-filter-based texture analysis to distinguish between smooth and rough or cracked road surfaces

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load road image
img = cv2.imread("road3.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 3: Resize image
gray = cv2.resize(gray, (400, 300))

# Step 4: Create Gabor filter settings
orientations = [0, 45, 90, 135]
responses = []

# Step 5: Apply Gabor filters
for angle in orientations:

    theta = np.deg2rad(angle)

    kernel = cv2.getGaborKernel(
        (10,10),  # Kernel size
        4.0,       # Sigma
        theta,     # Orientation
        10.0,      # Wavelength
        0.5,       # Gamma
        0,         # Phase
        ktype=cv2.CV_32F
    )

    # Apply filter
    filtered = cv2.filter2D(gray, cv2.CV_32F, kernel)
```

```
    # Convert response to absolute values
    filtered = np.abs(filtered)

    responses.append(filtered)

# Step 6: Combine all filter responses
texture = np.mean(responses, axis=0)

# Step 7: Calculate texture score
score = np.mean(texture)

print("Texture Score:", round(float(score), 2))
```

```
Texture Score: 2074.75
```

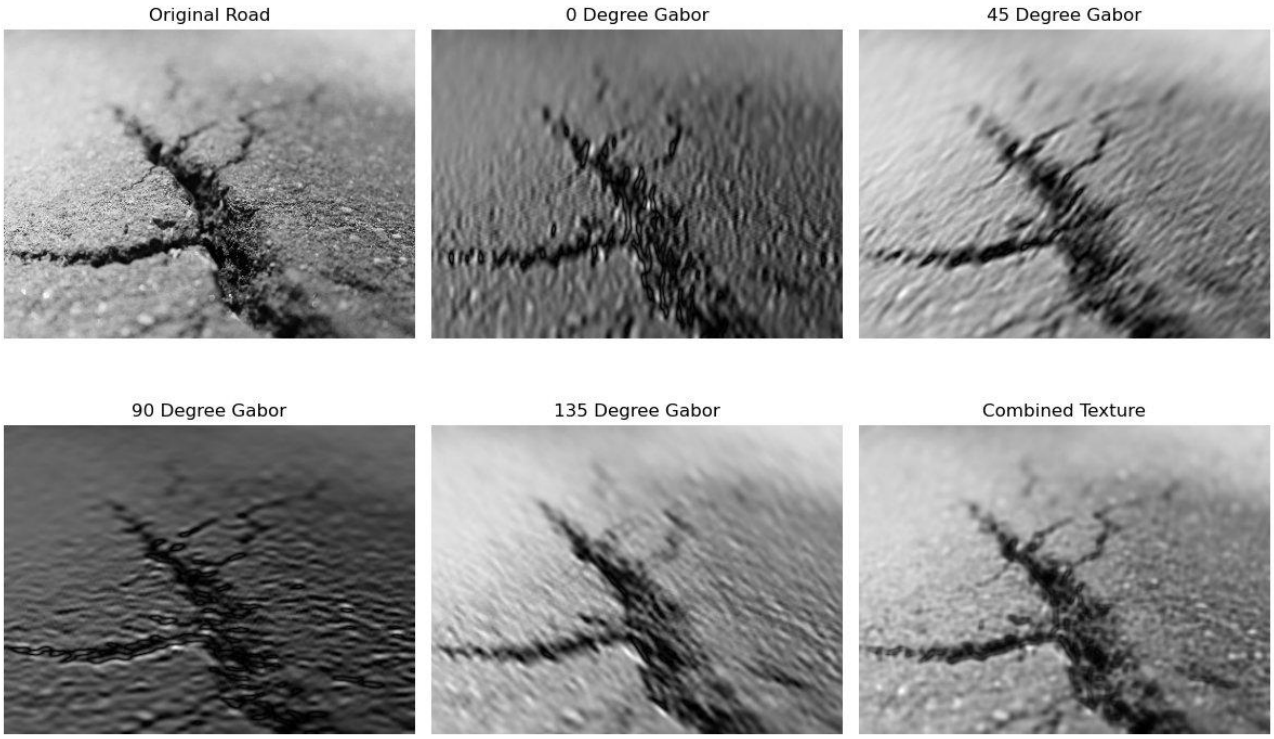
```
# Step 8: Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Road")
plt.axis("off")

for i in range(4):
    plt.subplot(2, 3, i + 2)
    plt.imshow(responses[i], cmap="gray")
    plt.title(str(orientations[i]) + " Degree Gabor")
    plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(texture, cmap="gray")
plt.title("Combined Texture")
plt.axis("off")

plt.tight_layout()
plt.show()
```



10. Extract texture features using Gabor filters and classify images of different types of wood.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Function to extract Gabor texture features
def extract_features(image_path):

    # Read image
    img = cv2.imread(image_path)

    if img is None:
        print("Image not found:", image_path)
        return None

    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Resize image
    gray = cv2.resize(gray, (200, 200))

    features = []

    # Apply Gabor filters at different angles
    for angle in [0, 45, 90, 135]:

        theta = np.deg2rad(angle)

        kernel = cv2.getGaborKernel((21, 21),4.0,theta,10.0,0.5,0,ktype=cv2.CV_32F)
```

```
        # Apply filter
        filtered = cv2.filter2D(gray, cv2.CV_32F, kernel)

        # Calculate texture features
        features.append(np.mean(np.abs(filtered)))
        features.append(np.std(filtered))

    return features
```

```
# Step 1: List wood images
images = ["wood1.jpg","wood2.jpg","wood3.jpg"]

# Step 2: Extract and display features
for image_path in images:

    features = extract_features(image_path)

    if features is None:
        continue

    print("\nImage:", image_path)
    print("Gabor Features:", np.round(features, 2))

    # Step 3: Simple rule-based classification
    texture_score = np.mean(features[::2])

    print("Texture Score:", np.round(texture_score, 2))

    if texture_score < 500:
        wood_type = "Class A"
    elif texture_score < 1500:
        wood_type = "Class B"
    else:
        wood_type = "Class C"

    print("Predicted Wood Type:", wood_type)
```

```
Image: wood1.jpg
Gabor Features: [1486.25 1729.62 1522.81  556.86  777.67  416.99 1520.56  555.07]
Texture Score: 1326.82
Predicted Wood Type: Class B

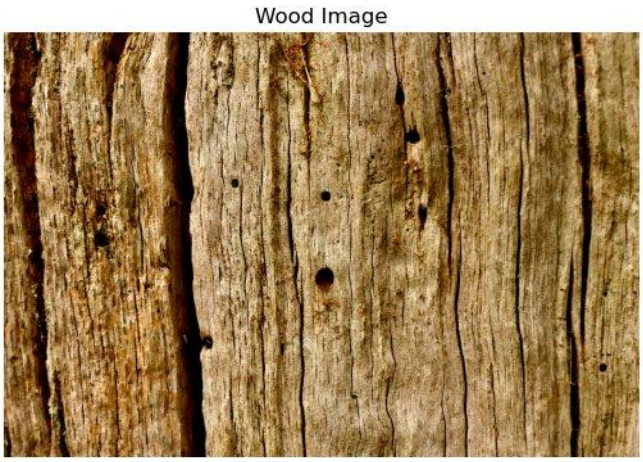
Image: wood2.jpg
Gabor Features: [ 957.38  639.82 1785.58  544.13  892.92  356.19 1763.92  486.7 ]
Texture Score: 1349.95
Predicted Wood Type: Class B

Image: wood3.jpg
Gabor Features: [ 457.91  365.21  892.23  642.76  852.56 1028.    852.23  485.67]
Texture Score: 763.73
Predicted Wood Type: Class B
```

```
# Step 4: Display one example image
img = cv2.imread(images[0])
```

```
if img is not None:
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    plt.imshow(img)
    plt.title("Wood Image")
    plt.axis("off")
    plt.show()
```



11. Use Gabor filters to distinguish between healthy and diseased regions of a plant leaf based on texture characteristics.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np

# 1. Load the leaf image in grayscale
# Replace 'leaf.jpg' with the path to your image
image = cv2.imread('leaf.jpg', cv2.IMREAD_GRAYSCALE)

if image is None:
  print('Error: Image not found. Please check the file path.')
else:
  # Resize for consistent processing
  image = cv2.resize(image, (400, 400))

  # 2. Define Gabor filter parameters
  ksize = 40  # Size of the filter kernel
  sigma = 4.0  # Standard deviation of the Gaussian envelope
  theta = np.pi / 4  # Orientation angle (e.g., 45 degrees)
  lambd = 10.0  # Wavelength of the sinusoidal factor
  gamma = 2  # Spatial aspect ratio (ellipticity)
  psi = 0.0  # Phase offset

  # Create the Gabor kernel
  gabor_kernel = cv2.getGaborKernel(
      (ksize, ksize), sigma, theta, lambd, gamma, psi, ktype=cv2.CV_32F
  )

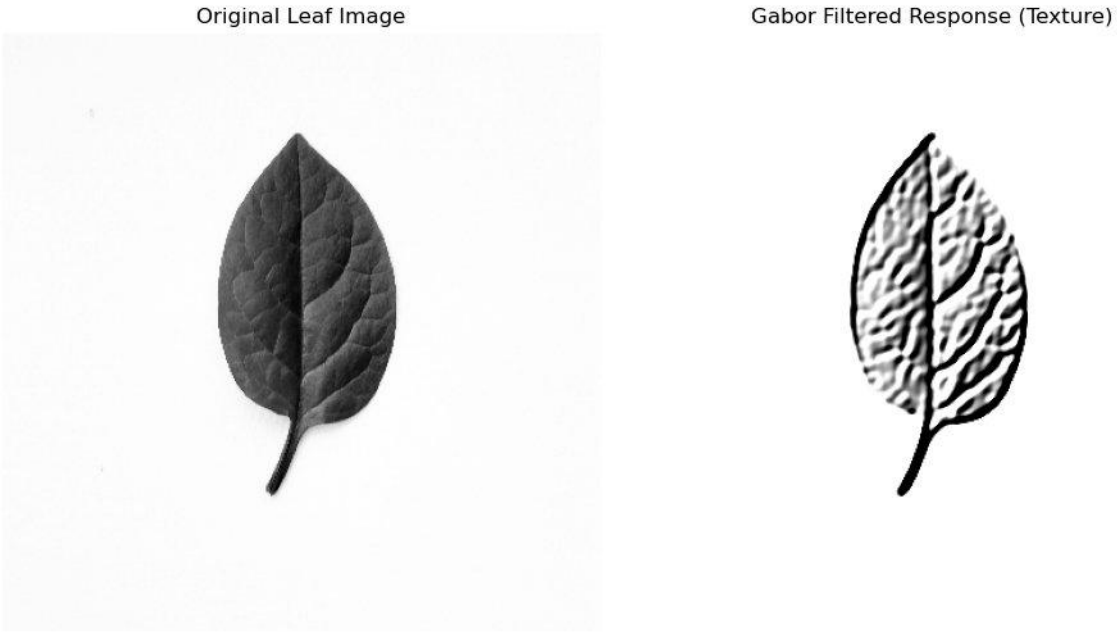
  # 3. Filter the image to extract texture features
  filtered_image = cv2.filter2D(image, cv2.CV_8UC3, gabor_kernel)
```

```
# 4. Visualize the original and filtered textures using Matplotlib
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Leaf Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(filtered_image, cmap='gray')
plt.title('Gabor Filtered Response (Texture)')
plt.axis('off')

plt.tight_layout()
plt.show()
```



12. Apply Gabor filters to identify and differentiate surface textures in images of tiles, metal, or other manufactured materials.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Read the material image
img = cv2.imread("metal.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Step 2: Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Step 3: Resize image
gray = cv2.resize(gray, (300, 300))

# Step 4: Apply Gabor filters at different angles
angles = [0, 45, 90, 135]
responses = []

for angle in angles:

    # Convert angle to radians
    theta = np.deg2rad(angle)

    # Create Gabor filter
    kernel = cv2.getGaborKernel((40,40),4.0,theta,10.0,0.5,0,ktype=cv2.CV_32F)

    # Apply filter
    filtered = cv2.filter2D(gray, cv2.CV_32F, kernel)

    # Convert response to visible image
    filtered = cv2.convertScaleAbs(filtered)

    responses.append(filtered)

# Step 5: Display results
plt.figure(figsize=(10, 7))

plt.subplot(2, 3, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original Material")
plt.axis("off")

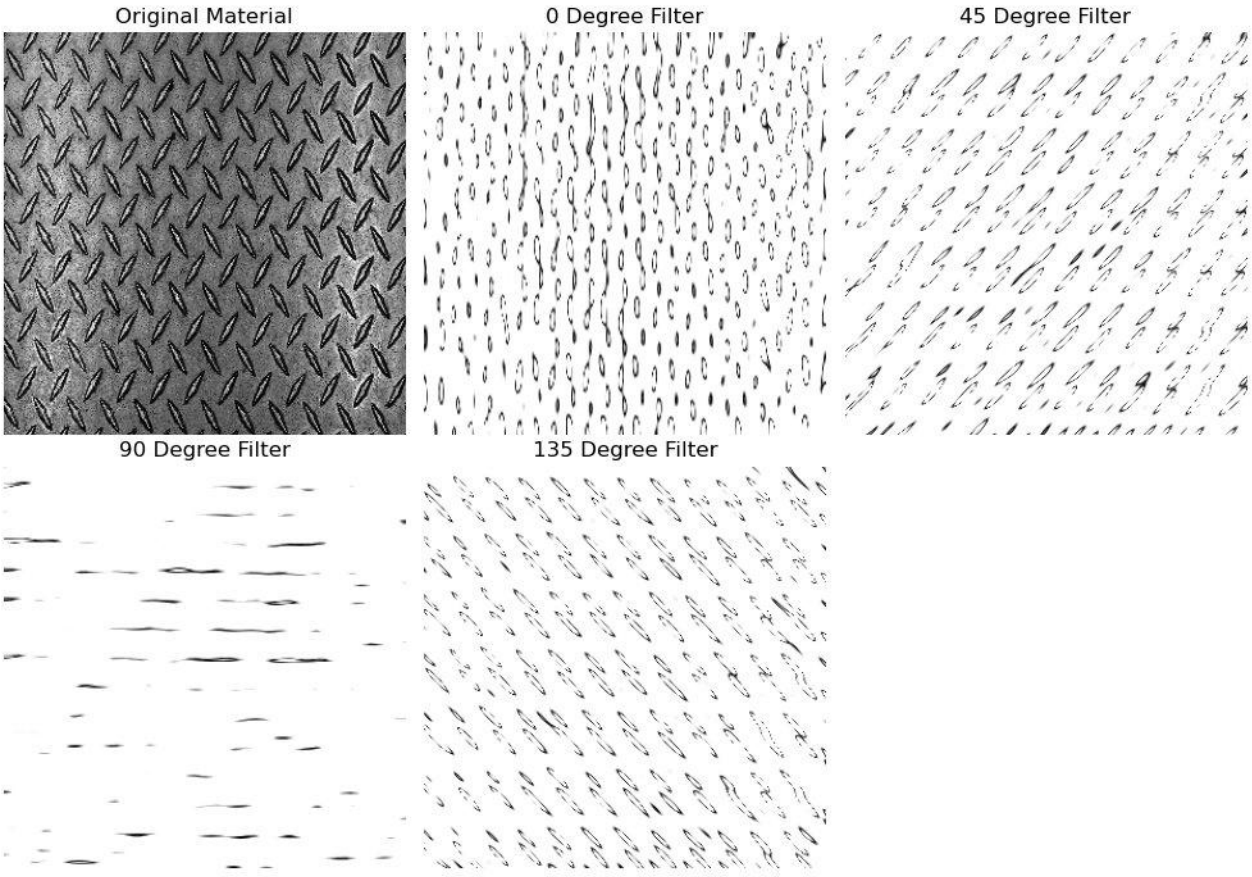
for i in range(4):
    plt.subplot(2, 3, i + 2)
    plt.imshow(responses[i], cmap="gray")
    plt.title(str(angles[i]) + " Degree Filter")
    plt.axis("off")

# Step 6: Calculate texture score
score = np.mean(responses)

print("Texture Score:", round(float(score), 2))

plt.tight_layout()
plt.show()
```

```
Texture Score: 244.39
```



13. Implement two-dimensional Discrete Wavelet Transform (DWT) for image compression and reconstruct the image using selected wavelet coefficients.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
import pywt

# 1. Load image in grayscale and resize for clean wavelet levels
# Replace 'image.jpg' with your target image path
image = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
image = cv2.resize(image, (512, 512))
```

```
# 2. Apply 2D Discrete Wavelet Transform (DWT) using the 'haar' wavelet
# Returns approximation (LL) and details (LH: horizontal, HL: vertical, HH: diagonal)
coeffs2 = pywt.dwt2(image, 'haar')
LL, (LH, HL, HH) = coeffs2

# 3. Compress by thresholding (zeroing out weak coefficients to save space)
threshold = 25
LH_thresh = pywt.threshold(LH, threshold, mode='soft')
HL_thresh = pywt.threshold(HL, threshold, mode='soft')
HH_thresh = pywt.threshold(HH, threshold, mode='soft')

# Combine the thresholded coefficients back
coeffs_compressed = (LL, (LH_thresh, HL_thresh, HH_thresh))

# 4. Reconstruct the image from the selected/compressed coefficients
```

```
reconstructed_image = pywt.idwt2(coeffs_compressed, 'haar')
```

```
# 5. Visualize Original vs. Reconstructed Compressed Image
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(reconstructed_image, cmap='gray')
plt.title('Reconstructed (Compressed) Image')
plt.axis('off')

plt.tight_layout()
plt.show()
```



14. Implement DWT-based image denoising on a noisy medical image and compare the original, noisy, and denoised images.

```
import cv2
import pywt
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("medical.jpg",0)

if img is None:
    print("Image not found!")
    exit()
```

```
# Add stronger Gaussian noise
```

```
np.random.seed(10)
noise = np.random.normal(0, 25, img.shape)
noisy = np.clip(img.astype(float) + noise, 0, 255).astype(np.uint8)

# Apply DWT
LL, (LH, HL, HH) = pywt.dwt2(noisy, "haar")

# Remove noise using thresholding
LH = pywt.threshold(LH, 35, mode="soft")
HL = pywt.threshold(HL, 35, mode="soft")
HH = pywt.threshold(HH, 35, mode="soft")

# Reconstruct image
denoised = pywt.idwt2((LL, (LH, HL, HH)), "haar")
denoised = np.clip(denoised, 0, 255).astype(np.uint8)
```

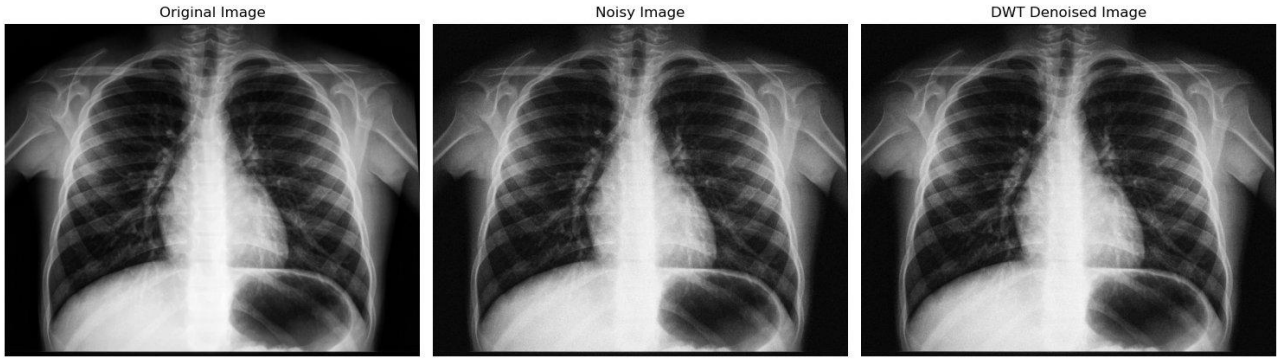
```
# Display images
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(noisy, cmap="gray", vmin=0, vmax=255)
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(denoised, cmap="gray", vmin=0, vmax=255)
plt.title("DWT Denoised Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
# Make sure the original/noisy images match the denoised image's shape
h, w = denoised.shape[:2]
img_resized = cv2.resize(img, (w, h))
noisy_resized = cv2.resize(noisy, (w, h))

# Now calculate the standard deviation of the differences safely
print('Noisy image standard deviation:',np.std(noisy_resized.astype(float) -
img_resized.astype(float)))
```

```
print('Denoised image standard deviation:',np.std(denoised.astype(float) -
img_resized.astype(float)))
```

```
Noisy image standard deviation: 15.095075925789017
Denoised image standard deviation: 12.801370436482742
```

15. Apply DWT-based denoising to an old or degraded photograph while preserving important image details.

```
import cv2
import pywt
import numpy as np
import matplotlib.pyplot as plt

# Read old photograph
img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found!")
    exit()
```

```
# Add noise to simulate a degraded photograph
noise = np.random.normal(0,25, img.shape)
noisy = np.clip(img.astype(float) + noise, 0, 255).astype(np.uint8)

# Apply DWT
LL, (LH, HL, HH) = pywt.dwt2(noisy, "haar")

# Apply soft thresholding to remove noise
LH = pywt.threshold(LH, 10, mode="soft")
HL = pywt.threshold(HL, 10, mode="soft")
HH = pywt.threshold(HH, 10, mode="soft")

# Reconstruct image
denoised = pywt.idwt2((LL, (LH, HL, HH)), "haar")
denoised = np.clip(denoised, 0, 255).astype(np.uint8)
```

```
# Compare images
plt.figure(figsize=(12, 5))

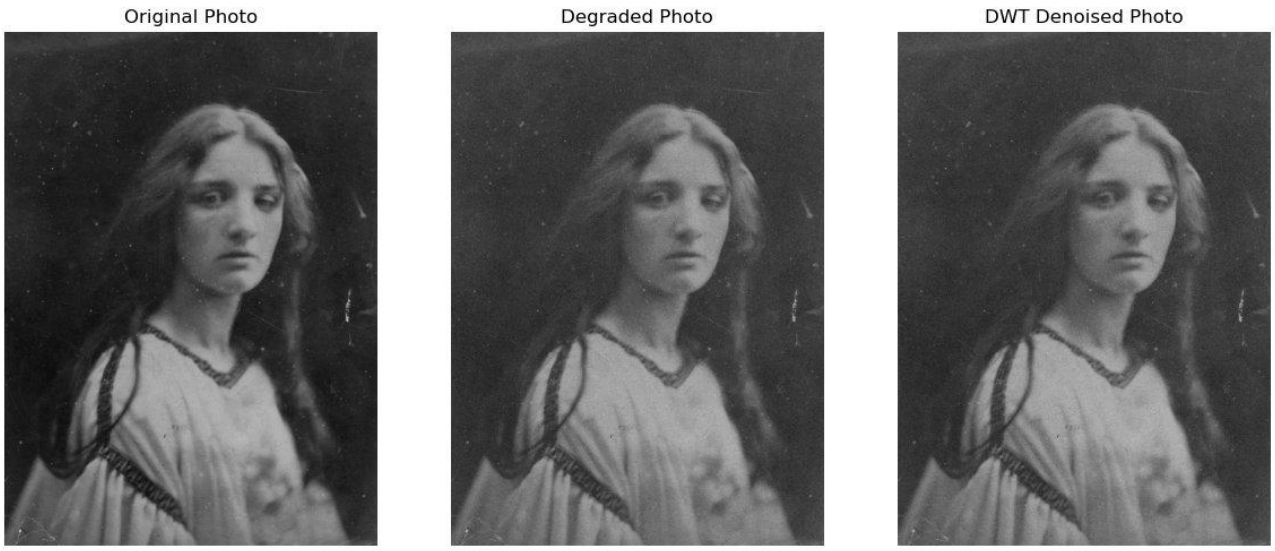
plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Photo")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(noisy, cmap="gray")
plt.title("Degraded Photo")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(denoised, cmap="gray")
plt.title("DWT Denoised Photo")
plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```



16. Implement DWT-based digital watermarking by embedding a watermark into an image and recovering the embedded watermark.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
import pywt

# 1. Load Host and Watermark images in grayscale
host = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
watermark = cv2.imread('watermark.jpg', cv2.IMREAD_GRAYSCALE)

# Resize to required dimensions (watermark must be half the host dimensions for DWT)
host = cv2.resize(host, (512, 512))
watermark = cv2.resize(watermark, (256, 256))
_, watermark = cv2.threshold(watermark, 127, 1, cv2.THRESH_BINARY)

# 2. Apply Wavelet Transform to Host Image
LL, (LH, HL, HH) = pywt.dwt2(host, 'haar')

# 3. Embed Watermark into the HL frequency band
alpha = 20.0  # Strength of the watermark
HL_watermarked = HL + (alpha * watermark.astype(np.float32))

# 4. Reconstruct the Watermarked Image
watermarked_image = pywt.idwt2((LL, (LH, HL_watermarked, HH)), 'haar')
watermarked_image = np.clip(watermarked_image, 0, 255).astype(np.uint8)

# 5. Extract the Watermark back out
extracted = (HL_watermarked - HL) / alpha
extracted = np.clip(extracted * 255, 0, 255).astype(np.uint8)
```

```
# 6. Show Results
plt.figure(figsize=(10, 4))

plt.subplot(1, 3, 1)
```

```
plt.imshow(watermark * 255, cmap='gray')
plt.title('Original Watermark')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(watermarked_image, cmap='gray')
plt.title('Watermarked Image')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(extracted, cmap='gray')
plt.title('Extracted Watermark')
plt.axis('off')

plt.tight_layout()
plt.show()
```



17. Analyze the quality of two images using their DWT coefficients and compare the amount of detail information present in each image.

```
import cv2
import pywt
import numpy as np
import matplotlib.pyplot as plt

# Read two images in grayscale
img1 = cv2.imread("product.jpg", 0)
img2 = cv2.imread("product2.jpg", 0)

if img1 is None or img2 is None:
    print("Image not found!")
    exit()
```

```
# Resize images to the same size
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

# Apply DWT
LL1, (LH1, HL1, HH1) = pywt.dwt2(img1, "haar")
LL2, (LH2, HL2, HH2) = pywt.dwt2(img2, "haar")

# Calculate detail energy
```

```
energy1 = np.mean(LH1**2 + HL1**2 + HH1**2)
energy2 = np.mean(LH2**2 + HL2**2 + HH2**2)

print("Image 1 Detail Energy:", round(energy1, 2))
print("Image 2 Detail Energy:", round(energy2, 2))
```

```
Image 1 Detail Energy: 26.4
Image 2 Detail Energy: 47.24
```

```
# Display images
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(img1, cmap="gray")
plt.title("Image 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(img2, cmap="gray")
plt.title("Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()

# Compare detail information
if energy1 > energy2:
    print("Image 1 contains more high-frequency detail energy.")
elif energy2 > energy1:
    print("Image 2 contains more high-frequency detail energy.")
else:
    print("Both images have equal detail energy.")
```




```
Image 2 contains more high-frequency detail energy.
```

18. Implement the Region Growing segmentation technique to segment a selected region from a medical image using suitable seed points.

```
import cv2
```

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Load your image in grayscale
img = cv2.imread('med2.jpg', cv2.IMREAD_GRAYSCALE)

img = cv2.resize(img,(400,400))
```

```
# 2. Pick a seed point (X, Y) inside the region you want to segment
h, w = img.shape
seed = (200,200)
threshold = 30  # Brightness tolerance

# 3. Simple Region Growing
mask = np.zeros((h, w), dtype=np.uint8)
visited = np.zeros((h, w), dtype=bool)

seed_val = int(img[seed[1], seed[0]])
queue = [seed]

while queue:
    x, y = queue.pop(0)

    if visited[y, x]:
        continue
    visited[y, x] = True

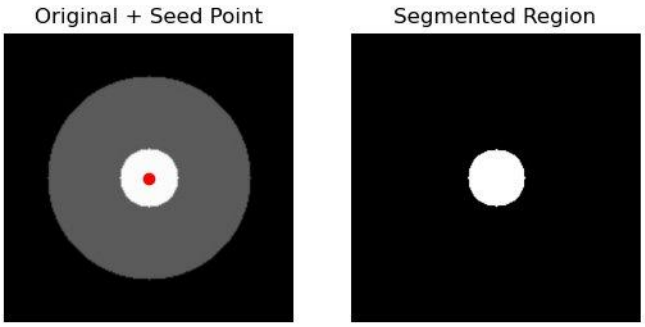
    # Check brightness similarity
    if abs(int(img[y, x]) - seed_val) <= threshold:
        mask[y, x] = 255

        # Add 4 neighbors (left, right, up, down)
        for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < w and 0 <= ny < h and not visited[ny, nx]:
                queue.append((nx, ny))
```

```
# 4. Display original image and result side-by-side
plt.subplot(1, 2, 1)
plt.imshow(img, cmap='gray')
plt.plot(seed[0], seed[1], 'ro')  # Red dot showing seed point
plt.title("Original + Seed Point")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(mask, cmap='gray')
plt.title("Segmented Region")
plt.axis("off")

plt.show()
```



19. Implement Region Growing to separate water, vegetation, and land regions from a satellite image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load satellite image (RGB)
# Replace 'satellite.jpg' with your image path
img = cv2.imread('satelite.png')
img = cv2.resize(img,(400,400))
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Convert to HSV color space (HSV works best for color-based region growing)
img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

```
# 2. Multi-Region Growing Function (using Color Similarity)
def region_growing_color(hsv_img, seed, color_threshold=(15, 40, 40)):
    """
    Grows a region starting from a seed point based on HSV color similarity.
    color_threshold: (H_threshold, S_threshold, V_threshold)
    """
    h, w, _ = hsv_img.shape
    mask = np.zeros((h, w), dtype=np.uint8)
    visited = np.zeros((h, w), dtype=bool)

    seed_val = hsv_img[seed[1], seed[0]].astype(int)
    queue = [seed]

    while queue:
        x, y = queue.pop(0)

        if visited[y, x]:
            continue
        visited[y, x] = True

        # Calculate HSV difference from seed pixel
        curr_val = hsv_img[y, x].astype(int)
        h_diff = abs(curr_val[0] - seed_val[0])
        s_diff = abs(curr_val[1] - seed_val[1])
        v_diff = abs(curr_val[2] - seed_val[2])

        # Check if difference is within color thresholds
        if h_diff <= color_threshold[0] and s_diff <= color_threshold[1] and v_diff <=
color_threshold[2]:
            mask[y, x] = 255
```

```
            # 4-connected neighbors
            for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nx, ny = x + dx, y + dy
                if 0 <= nx < w and 0 <= ny < h and not visited[ny, nx]:
                    queue.append((nx, ny))

    return mask

# 3. Define Seed Points (x, y) for each class
# Note: Adjust these (X, Y) coordinates to match your image feature locations!
seed_water = (75, 180)       # Seed on water body
seed_vegetation = (180,200)  # Seed on green forest/fields
seed_land = (250, 380)       # Seed on bare ground/built-up land

# 4. Grow Regions for each class
mask_water = region_growing_color(img_hsv, seed_water, color_threshold=(15, 60, 60))
mask_veg = region_growing_color(img_hsv, seed_vegetation, color_threshold=(20, 50, 50))
mask_land = region_growing_color(img_hsv, seed_land, color_threshold=(20, 50, 50))

# 5. Create Colored Overlay Map
overlay = np.zeros_like(img_rgb)
overlay[mask_water == 255] = [0, 100, 255]    # Blue for Water
overlay[mask_veg == 255] = [0, 200, 0]        # Green for Vegetation
overlay[mask_land == 255] = [210, 180, 140]   # Brown/Tan for Land
```

```
# 6. Display Results
plt.figure(figsize=(14, 8))

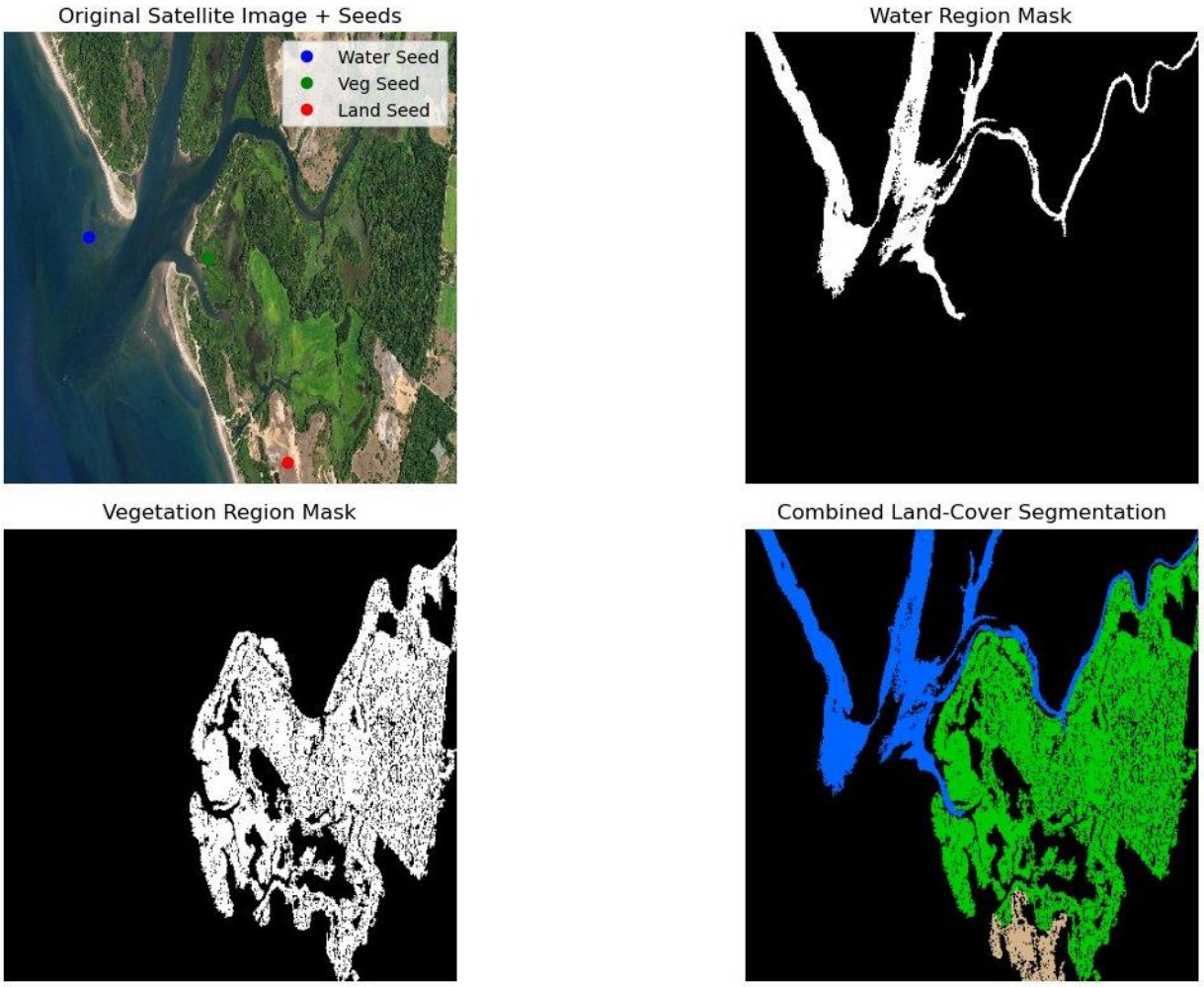
plt.subplot(2, 2, 1)
plt.imshow(img_rgb)
plt.plot(seed_water[0], seed_water[1], 'bo', label='Water Seed')
plt.plot(seed_vegetation[0], seed_vegetation[1], 'go', label='Veg Seed')
plt.plot(seed_land[0], seed_land[1], 'ro', label='Land Seed')
plt.title("Original Satellite Image + Seeds")
plt.legend(loc='upper right')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(mask_water, cmap='gray')
plt.title("Water Region Mask")
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(mask_veg, cmap='gray')
plt.title("Vegetation Region Mask")
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(overlay)
plt.title("Combined Land-Cover Segmentation")
plt.axis('off')

plt.tight_layout()
plt.show()
```



20. Implement edge-based image segmentation using the Canny edge detector to identify road boundaries in a road image.

```
import cv2
import matplotlib.pyplot as plt

# 1. Load your road image
img = cv2.imread('road_edge.png')

# 2. Convert to grayscale & blur to reduce noise
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

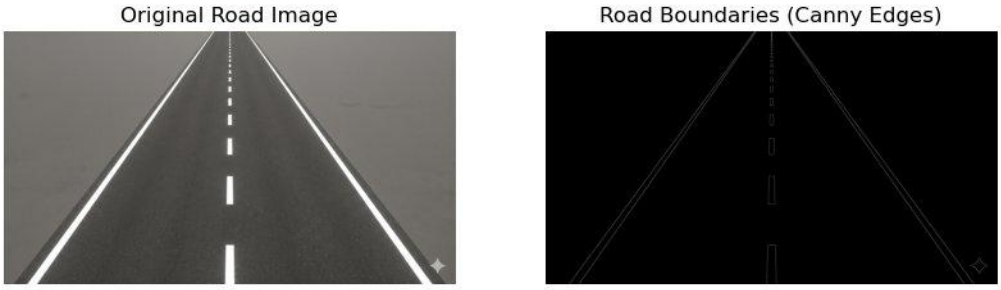
# 3. Detect edges using Canny
edges = cv2.Canny(blur, 150, 200)

# 4. Display original and detected road boundaries
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Road Image")
plt.axis('off')
```

```
plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title("Road Boundaries (Canny Edges)")
plt.axis('off')

plt.show()
```



21. Implement edge-based segmentation to separate a foreground object from its background in a real-world photograph

```
import cv2
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("leaf2.jpg")

if img is None:
    print("Image not found!")
    exit()

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Reduce noise
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# Detect edges
edges = cv2.Canny(blur, 50, 150)

# Find contours
contours, _ = cv2.findContours(
    edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

# Draw detected object boundaries
result = img.copy()
cv2.drawContours(result, contours, -1, (0, 255, 0), 2)

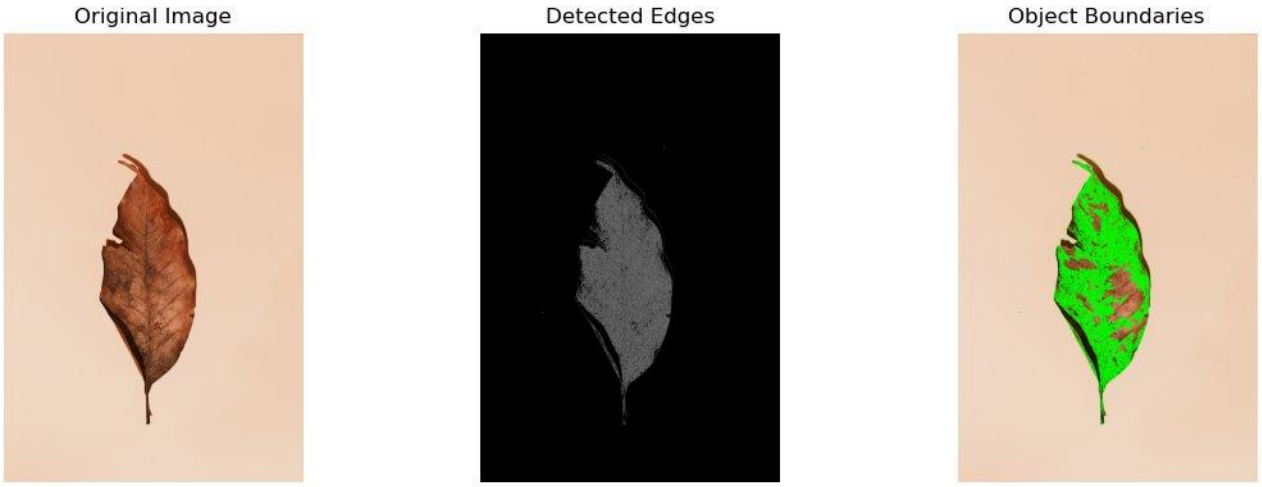
# Display results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
```

```
plt.subplot(1, 3, 2)
plt.imshow(edges, cmap="gray")
plt.title("Detected Edges")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Object Boundaries")
plt.axis("off")

plt.tight_layout()
plt.show()
```




```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load image & convert to grayscale
img = cv2.imread('leaf2.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 2. Detect edges using Canny
edges = cv2.Canny(gray, 50, 150)

# 3. Connect broken edges to form a closed shape
kernel = np.ones((5, 5), np.uint8)
closed_edges = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel)

# 4. Find the object boundary and fill it to make a mask
contours, _ = cv2.findContours(closed_edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
mask = np.zeros_like(gray)

if contours:
    largest_contour = max(contours, key=cv2.contourArea)
    cv2.drawContours(mask, [largest_contour], -1, 255, thickness=cv2.FILLED)

# 5. Extract foreground object
foreground = cv2.bitwise_and(img, img, mask=mask)
```

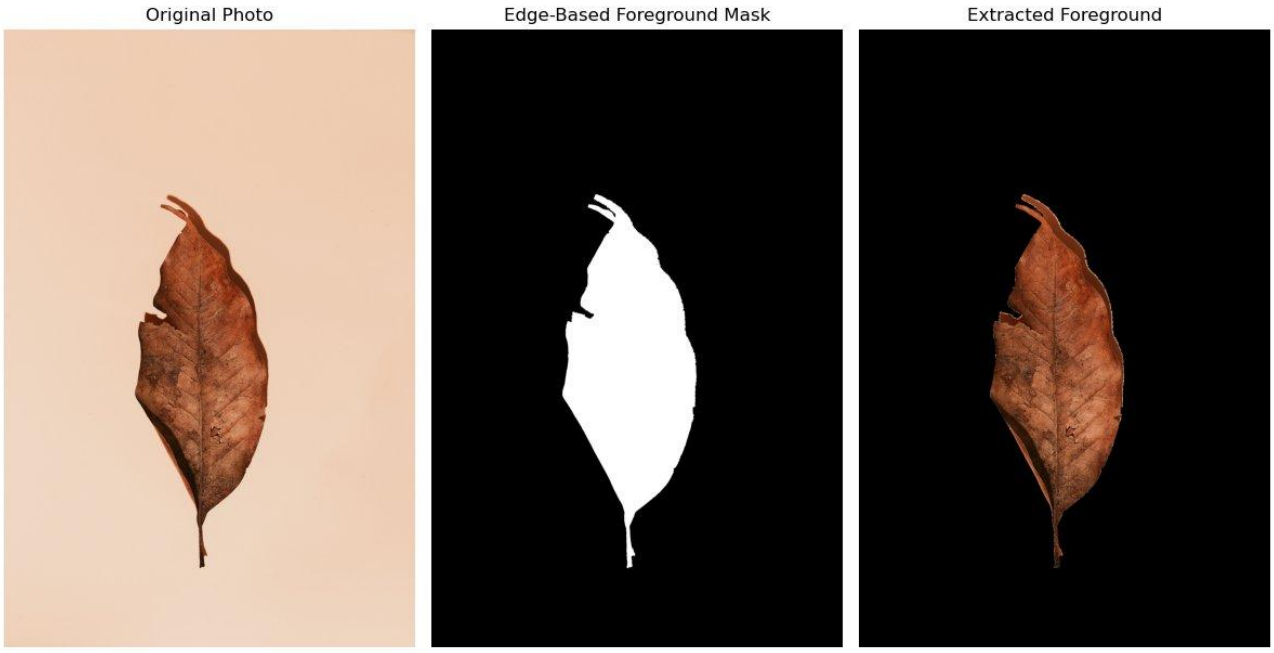
```
# 6. Display Results
plt.figure(figsize=(12, 6))

plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Photo")
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(mask, cmap='gray')
plt.title("Edge-Based Foreground Mask")
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(foreground, cv2.COLOR_BGR2RGB))
plt.title("Extracted Foreground")
plt.axis('off')

plt.tight_layout()
plt.show()
```



22. Implement Graph-Cut-based image segmentation to separate a region of interest, such as an organ, from the background in a medical image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. Load your medical image
img = cv2.imread('medical3.png')
img = cv2.resize(img,(400,400))
```

```
# 2. Define a bounding box around the organ/region of interest
# Format: (start_x, start_y, width, height)
# Change these values according to where your organ is located in the image
```

```
rect = (50,50, 200,300)

# 3. Create mask and temporary models required by GrabCut
mask = np.zeros(img.shape[:2], np.uint8)
bgd_model = np.zeros((1, 65), np.float64)
fgd_model = np.zeros((1, 65), np.float64)

# 4. Apply Graph-Cut (GrabCut) algorithm
# Runs for 5 iterations using the bounding box setup
cv2.grabCut(img, mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT)

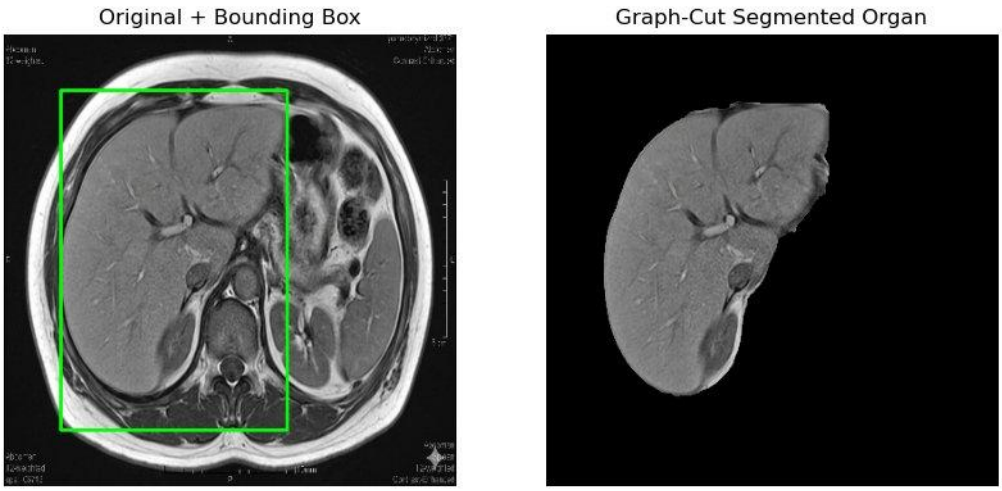
# 5. Extract foreground (Values 1 and 3 in the mask belong to foreground)
output_mask = np.where((mask == 1) | (mask == 3), 1, 0).astype('uint8')
foreground = img * output_mask[:, :, np.newaxis]
```

```
# 6. Display original image with rectangle and extracted region
plt.figure(figsize=(10, 5))

# Plot 1: Bounding Box Selection
img_with_rect = img.copy()
cv2.rectangle(img_with_rect, (rect[0], rect[1]), (rect[0]+rect[2], rect[1]+rect[3]), (0, 255, 0),
2)
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img_with_rect, cv2.COLOR_BGR2RGB))
plt.title("Original + Bounding Box")
plt.axis('off')

# Plot 2: Extracted Organ / Foreground
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(foreground, cv2.COLOR_BGR2RGB))
plt.title("Graph-Cut Segmented Organ")
plt.axis('off')

plt.show()
```



23. Implement Graph Cut to remove the background from a photograph and extract the required foreground object.

```
import cv2
import numpy as np
```

```
import matplotlib.pyplot as plt

# 1. Load your image
# Replace 'your_image.jpg' with your actual image path
img = cv2.imread('bike.png')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
# 2. Automatically set a bounding box around the center (or manually edit coordinates)
# Format: (start_x, start_y, width, height)
h, w = img.shape[:2]
rect = (int(w * 0.1), int(h * 0.1), int(w * 0.8), int(h * 0.8))

# 3. Initialize Graph-Cut structures
mask = np.zeros((h, w), np.uint8)
bgd_model = np.zeros((1, 65), np.float64)
fgd_model = np.zeros((1, 65), np.float64)

# 4. Run Graph-Cut (GrabCut) for 5 iterations
cv2.grabCut(img, mask, rect, bgd_model, fgd_model, 5, cv2.GC_INIT_WITH_RECT)

# 5. Extract Foreground (Values 1 and 3 belong to the object)
output_mask = np.where((mask == 1) | (mask == 3), 1, 0).astype('uint8')
foreground = img_rgb * output_mask[:, :, np.newaxis]
```

```
# 6. Display Results
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(foreground)
plt.title("Extracted Object (Background Removed)")
plt.axis('off')

plt.show()
```





24. Implement Mean-Shift segmentation to divide a natural scene image into meaningful regions based on color and spatial information.

```
import cv2
import matplotlib.pyplot as plt

# Read image
```

```
img = cv2.imread("nature.png")
img = cv2.resize(img,(400,400))

if img is None:
    print("Image not found!")
    exit()
```

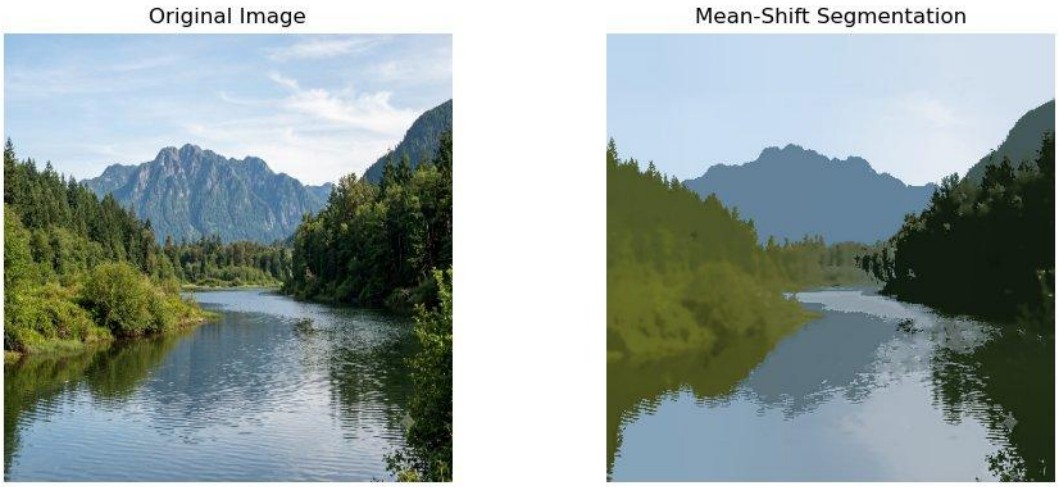
```
# Apply Mean-Shift segmentation
result = cv2.pyrMeanShiftFiltering(img,sp=50,sr=50,maxLevel=1)
```

```
# Display images
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Mean-Shift Segmentation")
plt.axis("off")

plt.tight_layout()
plt.show()
```



25. Implement Markov Random Field (MRF)-based image segmentation to classify pixels into different regions in a noisy image and compare the result with basic segmentation.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
img = cv2.imread("nature.png", 0)

if img is None:
    print("Image not found!")
    exit()
```

```
noisy = img
```

```
# Basic segmentation
_, basic_seg = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# MRF-like segmentation using smoothing
smooth = cv2.medianBlur(noisy, 5)
_, mrf_result = cv2.threshold(smooth, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

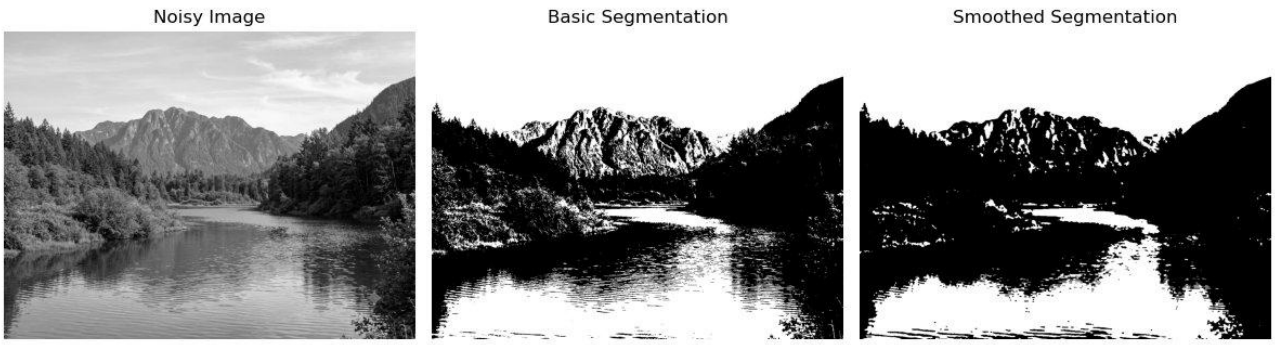
```
# Display results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(noisy, cmap="gray")
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(basic, cmap="gray")
plt.title("Basic Segmentation")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(mrf_result, cmap="gray")
plt.title("Smoothed Segmentation")
plt.axis("off")

plt.tight_layout()
plt.show()
```



26. Implement texture-based image segmentation using Gabor-filter or DWT-based features to separate regions with different textures, such as grass, road, and building surfaces.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image
img = cv2.imread("road2.jpg", 0)

if img is None:
```

```
    print("Image not found!")
    exit()
img  = cv2.resize(img,(600,400))
```

```
# Apply Gabor filter
kernel = cv2.getGaborKernel((5,5), 10, 180, 5, 1, 0)
filtered = cv2.filter2D(img, cv2.CV_32F, kernel)

# Normalize filtered image
filtered = cv2.normalize(filtered, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)

# Segment into two texture regions
_, segmented = cv2.threshold(filtered, 127, 255, cv2.THRESH_BINARY)
```

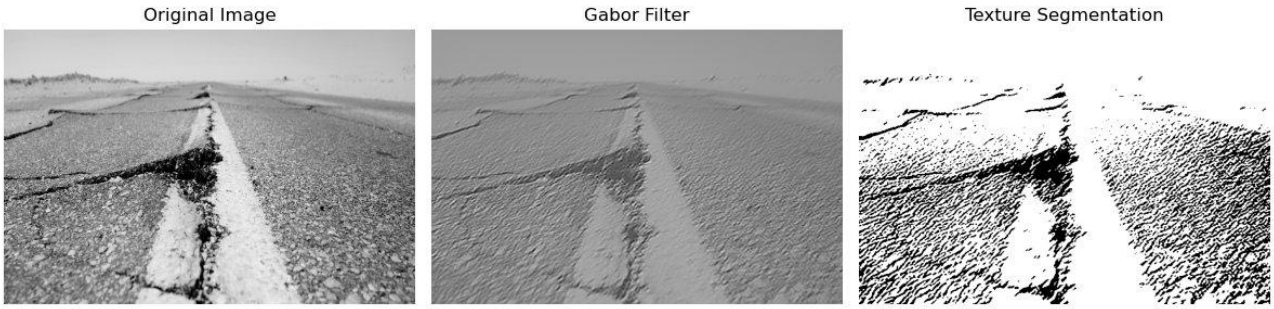
```
# Display results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(filtered, cmap="gray")
plt.title("Gabor Filter")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(segmented, cmap="gray")
plt.title("Texture Segmentation")
plt.axis("off")

plt.tight_layout()
plt.show()
```



27. Implement an object detection system to detect and count cars, motorcycles, and buses in a traffic image or video.

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

# Read traffic image
img = cv2.imread("traffic2.png")
```

```
if img is None:
    print("Image not found!")
    exit()
```

```
# Load pre-trained YOLO model
model = YOLO()

# Detect objects
results = model(img)

# Vehicle classes in COCO dataset
vehicles = {2: "Car",3: "Motorcycle",5: "Bus"}
counts = {"Car": 0, "Motorcycle": 0, "Bus": 0}

# Draw boxes and count vehicles
for box in results[0].boxes:
    class_id = int(box.cls[0])

    if class_id in vehicles:
        name = vehicles[class_id]
        counts[name] += 1

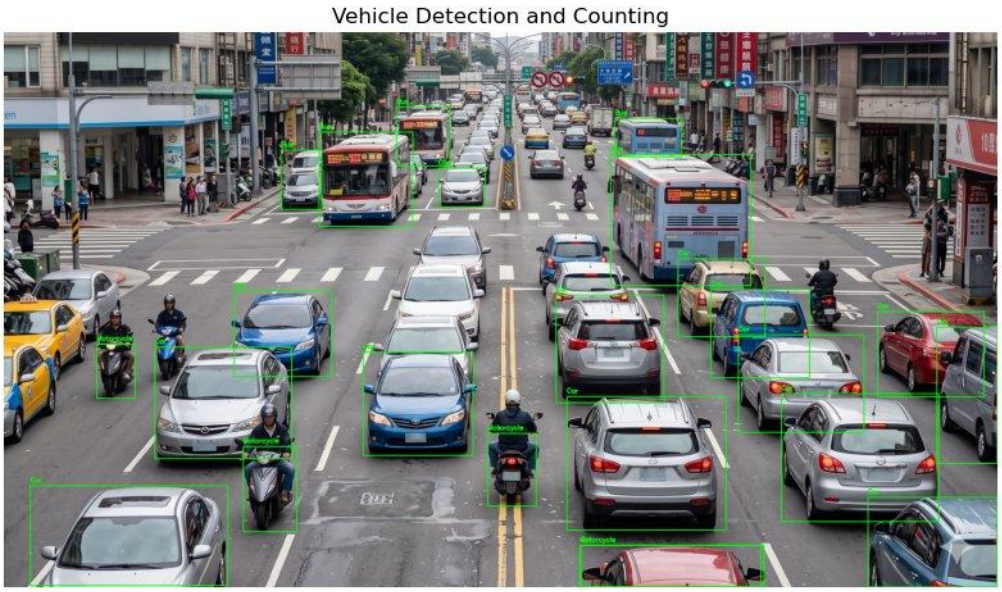
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(img, name, (x1, y1 - 10),cv2.FONT_HERSHEY_SIMPLEX, 0.6,(0, 255, 0), 2)
```

```
0: 384x640 5 persons, 15 cars, 4 motorcycles, 5 buss, 1 truck, 192.0ms
Speed: 5.9ms preprocess, 192.0ms inference, 3.3ms postprocess per image at shape (1, 3, 384, 640)
```

```
# Display vehicle counts
print("Cars:", counts["Car"])
print("Motorcycles:", counts["Motorcycle"])
print("Buses:", counts["Bus"])

# Display result
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Vehicle Detection and Counting")
plt.axis("off")
plt.show()
```

```
Cars: 15
Motorcycles: 4
Buses: 5
```



28. Develop an object detection and recognition system to identify garbage and classify it into categories such as plastic, paper, metal, glass, and organic waste.

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

# Load pre-trained YOLO model
model = YOLO("yolov8n.pt")

# Read garbage image
img = cv2.imread("garbage.png")

if img is None:
    print("Image not found!")
    exit()

# Detect objects
results = model(img)

# Display detected objects
for box in results[0].boxes:
    class_id = int(box.cls[0])
    name = model.names[class_id]

    x1, y1, x2, y2 = map(int, box.xyxy[0])

    cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(img, name, (x1, y1 - 10),cv2.FONT_HERSHEY_SIMPLEX, 0.6,(0,0,255), 2)

# Show result
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Garbage Object Detection")
plt.axis("off")
plt.show()
```

```
0: 384x640 1 bottle, 1 cup, 1 bowl, 3 bananas, 1 orange, 98.8ms
Speed: 4.9ms preprocess, 98.8ms inference, 2.4ms postprocess per image at shape (1, 3, 384, 640)
```



29. Develop an object detection and recognition system to identify different products on a store shelf and display their corresponding class labels.

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt

# Load YOLO model
model = YOLO("yolov8n.pt")

# Read shelf image
img = cv2.imread("shelf.png")

if img is None:
    print("Image not found!")
    exit()

# Detect objects
results = model(img)

# Draw bounding boxes and labels
for box in results[0].boxes:
    class_id = int(box.cls[0])
    label = model.names[class_id]

    x1, y1, x2, y2 = map(int, box.xyxy[0])

    cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(img, label, (x1, y1 - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                (0, 255, 0), 2)
```

```
# Display result
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Store Shelf Product Detection")
plt.axis("off")
plt.show()
```

```
0: 384x640 14 bottles, 13 potted plants, 1 book, 1 vase, 90.6ms
Speed: 4.6ms preprocess, 90.6ms inference, 2.9ms postprocess per image at shape (1, 3, 384, 640)
```



30. Develop a real-time object detection and recognition system for smart-city surveillance that detects predefined objects such as people and vehicles, draws bounding boxes around them, and displays their confidence scores.

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
SURVEILLANCE_CLASSES = {0: "Person", 2: "Car", 3: "Motorcycle", 5: "Bus", 7: "Truck"}

# Process a single frame/image directly for testing
img = cv2.imread('traffic2.png') # or read a frame from cv2.VideoCapture
results = model(img)

for box in results[0].boxes:
    class_id = int(box.cls[0])
    conf = float(box.conf[0])

    if class_id in SURVEILLANCE_CLASSES and conf > 0.4:
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        label_text = f"{SURVEILLANCE_CLASSES[class_id]} {conf*100:.1f}%"

        cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.putText(img, label_text, (x1, y1 - 10),
```

```
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

# Display safely in Notebooks
plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.show()
```

```
0: 384x640 6 persons, 24 cars, 5 motorcycles, 5 buss, 1 truck, 131.1ms
Speed: 6.2ms preprocess, 131.1ms inference, 2.3ms postprocess per image at shape (1, 3, 384, 640)
```



```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
SURVEILLANCE_CLASSES = {0: "Person", 2: "Car", 3: "Motorcycle", 5: "Bus", 7: "Truck"}

# Read first valid frame from your video file
cap = cv2.VideoCapture('cctv.mp4')
ret, frame = cap.read()
cap.release()

if ret:
    results = model(frame)

    for box in results[0].boxes:
        class_id = int(box.cls[0])
        conf = float(box.conf[0])

        if class_id in SURVEILLANCE_CLASSES and conf > 0.4:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            label_text = f"{SURVEILLANCE_CLASSES[class_id]} {conf * 100:.1f}%"

            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(frame, label_text, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
```

```
    plt.figure(figsize=(10, 6))
    plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    plt.title("Surveillance Detection Result")
    plt.axis('off')
    plt.show()
```

```
0: 640x384 2 persons, 3 cars, 2 motorcycles, 2 trucks, 74.2ms
Speed: 5.2ms preprocess, 74.2ms inference, 1.7ms postprocess per image at shape (1, 3, 640, 384)
```




```
import cv2
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
cap = cv2.VideoCapture("cctv.mp4")

classes = ["person", "car", "motorcycle", "bus", "truck", "bicycle"]

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Detect objects
    results = model(frame, verbose=False)[0]

    for box in results.boxes:
        label = model.names[int(box.cls[0])]
        conf = float(box.conf[0])

        if label in classes and conf > 0.4:
            x1, y1, x2, y2 = map(int, box.xyxy[0])

            # Draw box and label
```

```
            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(frame, f"{label}: {conf:.2f}", (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    # Resize and display
    cv2.imshow("CCTV Detection", cv2.resize(frame, (800, 700)))
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

P4

CV practical assignment 4

Question 1

```
import cv2

# Initialize video capture (0 for webcam, or pass a video file path e.g. "corridor.mp4")
video_path = 0
cap = cv2.VideoCapture("person_walking_in_bedroom.mp4")

if not cap.isOpened():
    print("Error: Could not open video source.")
    exit()

# Read the first frame
ret, frame1 = cap.read()
ret, frame2 = cap.read()

# Minimum contour area to trigger motion detection (filters out small noise)
MIN_CONTOUR_AREA = 1000

while cap.isOpened() and ret:
    # 1. Compute absolute difference between consecutive frames
    diff = cv2.absdiff(frame1, frame2)

    # 2. Convert to grayscale
    gray = cv2.cvtColor(diff, cv2.COLOR_BGR2GRAY)

    # 3. Apply Gaussian Blur to smooth out high-frequency noise
    blur = cv2.GaussianBlur(gray, (5, 5), 0)

    # 4. Threshold to get a binary image (motion = white, background = black)
    _, thresh = cv2.threshold(blur, 20, 255, cv2.THRESH_BINARY)

    # 5. Dilate thresholded image to fill in gaps inside contours
    dilated = cv2.dilate(thresh, None, iterations=3)

    # 6. Find contours of the motion regions
    contours, _ = cv2.findContours(dilated, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    motion_detected = False

    for contour in contours:
        # Ignore small movements (e.g., small camera jitter or shadows)
        if cv2.contourArea(contour) < MIN_CONTOUR_AREA:
            continue

        motion_detected = True

        # Get bounding box coordinates for the moving region
        (x, y, w, h) = cv2.boundingRect(contour)
```

```
        # Draw bounding box around detected motion
        cv2.rectangle(frame1, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # Display status on the video feed
    if motion_detected:
        cv2.putText(
            frame1,
            "Status: Motion Detected",
            (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX,
            1,
            (0, 0, 255),
            2,
        )
    else:
        cv2.putText(
            frame1,
            "Status: Clear",
            (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX,
            1,
            (0, 255, 0),
            2,
        )

    # Display frames
    cv2.imshow("Corridor CCTV Feed", frame1)
    cv2.imshow("Motion Threshold", dilated)

    # Update frames for the next iteration
    frame1 = frame2
    ret, frame2 = cap.read()

    # Press 'q' or 'ESC' to exit
    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

Question 2

```
import cv2

# Initialize video stream (use 0 for webcam or replace with path to video file, e.g.,
'parking.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video source.")
    exit()

# Initialize MOG2 Background Subtractor
# history: number of frames used to build the background model
```

```
# varThreshold: threshold on the squared Mahalanobis distance (controls sensitivity)
# detectShadows: True marks shadows in gray (127), which helps exclude them from contours
bg_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=50, detectShadows=True
)

# Morphological kernel to clean noise and bridge gaps in vehicle silhouettes
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

# Minimum contour area to trigger vehicle detection (filters out small movement like
leaves/birds)
MIN_VEHICLE_AREA = 3000

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # 1. Apply background subtraction
    fg_mask = bg_subtractor.apply(frame)

    # 2. Threshold to remove detected shadows (shadows are marked as 127, motion as 255)
    _, thresh = cv2.threshold(fg_mask, 250, 255, cv2.THRESH_BINARY)

    # 3. Morphological operations: Open to remove small speckles, Close to merge vehicle regions
    clean_mask = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
    clean_mask = cv2.morphologyEx(clean_mask, cv2.MORPH_CLOSE, kernel)

    # 4. Find contours of detected moving regions
    contours, _ = cv2.findContours(
        clean_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    vehicle_count = 0

    for contour in contours:
        # Ignore motion smaller than typical vehicle threshold
        if cv2.contourArea(contour) < MIN_VEHICLE_AREA:
            continue

        vehicle_count += 1

        # Get bounding box coordinates for moving vehicle
        x, y, w, h = cv2.boundingRect(contour)

        # Draw bounding box and label
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
        cv2.putText(
            frame,
            "Vehicle",
            (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,
            (0, 255, 0),
            2,
        )

    # Display real-time status overlay
    status_text = f"Moving Vehicles Detected: {vehicle_count}"
```

```
    cv2.putText(
        frame,
        status_text,
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 0, 255) if vehicle_count > 0 else (0, 255, 0),
        2,
    )

    # Render video windows
    cv2.imshow("Parking Security Camera", frame)
    cv2.imshow("Foreground Mask (Cleaned)", clean_mask)

    # Press 'q' or 'ESC' to exit
    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

```
# question 3
```

```
import cv2

# Initialize video source (0 for live webcam, or path to video file e.g., 'surveillance.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video feed.")
    exit()

# Initialize MOG2 Subtractor
# history=500: baseline frame history length
# varThreshold=16: variance threshold for pixel classification (lower = higher sensitivity)
# detectShadows=True: distinguishes true motion (255) from cast shadows (127)
bg_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=500, varThreshold=16, detectShadows=True
)

# Structuring element for noise filtering & merging segmented regions
kernel_open = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
kernel_close = cv2.getStructuringElement(cv2.MORPH_RECT, (7, 7))

# Minimum contour area (pixels) to qualify as an object
MIN_OBJECT_AREA = 800

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # 1. Obtain foreground mask from MOG2 model
    fg_mask = bg_subtractor.apply(frame)

    # 2. Isolate true motion pixels (255) from shadows (127)
```

```
    _, binary_mask = cv2.threshold(fg_mask, 250, 255, cv2.THRESH_BINARY)

    # 3. Morphological Operations
    # OPEN removes small noise dots; CLOSE fills gaps within individual objects
    mask_cleaned = cv2.morphologyEx(binary_mask, cv2.MORPH_OPEN, kernel_open)
    mask_cleaned = cv2.morphologyEx(mask_cleaned, cv2.MORPH_CLOSE, kernel_close)

    # 4. Find external contours representing independent moving objects
    contours, _ = cv2.findContours(
        mask_cleaned, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    detected_objects_count = 0

    for contour in contours:
        # Filter out minor perturbations (leaves, camera shake, small animals)
        if cv2.contourArea(contour) < MIN_OBJECT_AREA:
            continue

        detected_objects_count += 1

        # Calculate bounding box & object centroid
        x, y, w, h = cv2.boundingRect(contour)
        cx, cy = x + w // 2, y + h // 2

        # Draw bounding rectangle around each detected object
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

        # Plot centroid point
        cv2.circle(frame, (cx, cy), 4, (0, 0, 255), -1)

        # Label each detected instance with ID and dimensions
        label = f"Obj #{detected_objects_count} [{w}x{h}]"
        cv2.putText(
            frame,
            label,
            (x, y - 8),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.5,
            (0, 255, 0),
            1,
            cv2.LINE_AA,
        )

    # Top overlay bar showing total live detections
    cv2.rectangle(frame, (0, 0), (320, 40), (0, 0, 0), -1)
    status_msg = f"Active Targets: {detected_objects_count}"
    cv2.putText(
        frame,
        status_msg,
        (10, 26),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (0, 255, 255),
        2,
        cv2.LINE_AA,
    )

    # Display real-time output streams
```

```
    cv2.imshow("Multi-Object Detection Feed", frame)
    cv2.imshow("MOG2 Mask (Cleaned)", mask_cleaned)

    # Key exit listener ('q' or 'ESC')
    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

```
# question 4
```

```
import cv2

# Initialize video capture (0 for live camera, or path to video file e.g., 'surveillance.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video source.")
    exit()

# Read the first frame
ret, frame = cap.read()
if not ret:
    print("Error: Failed to read video feed.")
    cap.release()
    exit()

# Initialize OpenCV Object Tracker
# CSRT is highly accurate and handles rotation/occlusion well
# Note: For OpenCV versions < 4.5.0, use cv2.TrackerCSRT_create()
tracker = cv2.TrackerCSRT_create()

# Prompt user to select target object using bounding box
print("=== CONTROLS ===")
print("1. Drag mouse to draw a box around the object to track.")
print("2. Press ENTER or SPACE to confirm selection.")
print("3. Press 'c' to cancel selection.")
print("==================")

# Open ROI (Region of Interest) selector window
bbox = cv2.selectROI("Select Object to Track", frame, fromCenter=False, showCrosshair=True)
cv2.destroyWindow("Select Object to Track")

# Initialize the tracker with the selected bounding box and frame
if bbox[2] > 0 and bbox[3] > 0:  # Check if a valid ROI was selected
    tracker.init(frame, bbox)
else:
    print("No valid object selected. Exiting.")
    cap.release()
    exit()

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break
```

```
    # Update tracker position for the current frame
    success, bbox = tracker.update(frame)

    if success:
        # Tracking success: Draw bounding box
        x, y, w, h = [int(v) for v in bbox]
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

        # Draw center point
        cx, cy = x + w // 2, y + h // 2
        cv2.circle(frame, (cx, cy), 4, (0, 0, 255), -1)

        # Status text
        cv2.putText(frame, "Tracking Active", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
        cv2.putText(frame, f"Pos: ({cx}, {cy})", (10, 60),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 0), 1)
    else:
        # Tracking failure: Object lost or out of frame
        cv2.putText(frame, "Tracking Lost!", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

    # Display real-time output
    cv2.imshow("Object Tracking Feed", frame)

    # Controls: Press 'r' to re-select object, 'q' or ESC to quit
    key = cv2.waitKey(30) & 0xFF
    if key == ord('q') or key == 27:
        break
    elif key == ord('r'):
        # Re-select object interactively during runtime
        new_bbox = cv2.selectROI("Select Object to Track", frame, fromCenter=False)
        cv2.destroyWindow("Select Object to Track")
        if new_bbox[2] > 0 and new_bbox[3] > 0:
            tracker = cv2.TrackerCSRT_create()
            tracker.init(frame, new_bbox)

cap.release()
cv2.destroyAllWindows()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[1], line 21
     16     exit()
     18 # Initialize OpenCV Object Tracker
     19 # CSRT is highly accurate and handles rotation/occlusion well
     20 # Note: For OpenCV versions < 4.5.0, use cv2.TrackerCSRT_create()
---> 21 tracker = cv2.TrackerCSRT_create()
     23 # Prompt user to select target object using bounding box
     24 print("=== CONTROLS ===")

AttributeError: module 'cv2' has no attribute 'TrackerCSRT_create'
```

```
# question 5
```

```
import numpy as np
import cv2
```

```
# Initialize video stream
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

ret, frame = cap.read()
if not ret:
    print("Error: Cannot read video feed.")
    cap.release()
    exit()

# MANUALLY define bounding box coordinates: (x, y, width, height)
# Adjust these 4 values to fit your package location in the first frame
track_window = (200, 150, 100, 100)

x, y, w, h = track_window

# 1. Crop ROI and convert to HSV
roi = frame[y : y + h, x : x + w]
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# 2. Filter out low light/glare
mask = cv2.inRange(
    hsv_roi, np.array((0.0, 60.0, 32.0)), np.array((180.0, 255.0, 255.0))
)

# 3. Calculate 1D Hue histogram of target package
roi_hist = cv2.calcHist([hsv_roi], [0], mask, [180], [0, 180])
cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)

# Termination criteria: 10 iterations OR move < 1px
term_crit = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    dst = cv2.calcBackProject([hsv], [0], roi_hist, [0, 180], 1)

    # Apply Mean Shift
    ret, track_window = cv2.meanShift(dst, track_window, term_crit)

    # Draw result
    x, y, w, h = track_window
    cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

    # In Jupyter, use matplotlib or cv2_imshow instead of cv2.imshow
    cv2.imshow("Conveyor Belt Feed", frame)

    if cv2.waitKey(30) & 0xFF in (ord("q"), 27):
        break

cap.release()
cv2.destroyAllWindows()
```

```
# question 6
```

```
import numpy as np
import cv2

# Initialize video stream (0 for live camera or path to video file e.g., 'sports.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video source.")
    exit()

# Read initial frame
ret, frame = cap.read()
if not ret or frame is None:
    print("Error: Cannot read initial frame.")
    cap.release()
    exit()

# --- TARGET SELECTION ---
# Option A: Interactive selection (Works in standard local Python scripts)
cv2.namedWindow("Select Player", cv2.WINDOW_NORMAL)
track_window = cv2.selectROI("Select Player", frame, fromCenter=False)
cv2.destroyWindow("Select Player")

x, y, w, h = track_window

# Fallback: Hardcode initial ROI (x, y, w, h) if running in Jupyter/headless environments
if w == 0 or h == 0:
    track_window = (200, 100, 80, 150)
    x, y, w, h = track_window

# 1. Crop player ROI and convert to HSV color space
roi = frame[y : y + h, x : x + w]
hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)

# 2. Mask out non-informative pixels (extreme shadows / glare)
mask = cv2.inRange(
    hsv_roi, np.array((0.0, 60.0, 32.0)), np.array((180.0, 255.0, 255.0))
)

# 3. Calculate and normalize the 1D Hue histogram of the player target
roi_hist = cv2.calcHist([hsv_roi], [0], mask, [180], [0, 180])
cv2.normalize(roi_hist, roi_hist, 0, 255, cv2.NORM_MINMAX)

# Termination criteria: Stop iteration when window moves < 1px or reaches 10 iterations
term_crit = (cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 1)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Convert frame to HSV and project color histogram model
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    dst = cv2.calcBackProject([hsv], [0], roi_hist, [0, 180], 1)

    # Apply CamShift to adapt window location, size, and orientation
```

```
    ret_camshift, track_window = cv2.CamShift(dst, track_window, term_crit)

    # CamShift returns a RotatedRect structure: ((center_x, center_y), (width, height), angle)
    pts = cv2.boxPoints(ret_camshift)
    pts = np.int32(pts)

    # Draw the adaptive, rotated tracking ellipse / bounding box around the player
    cv2.polylines(frame, [pts], True, (0, 255, 0), 2)

    # Extract orientation angle and bounding box dimensions
    (cx, cy), (box_w, box_h), angle = ret_camshift
    cv2.putText(
        frame,
        f"CamShift Tracking | Size: {int(box_w)}x{int(box_h)} | Angle: {int(angle)}deg",
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (0, 255, 255),
        2,
    )

    # Display video feed and back-projection heatmap
    cv2.imshow("CamShift Player Tracking", frame)
    cv2.imshow("Back-Projection Map", dst)

    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

```
# question 7
```

```
import numpy as np
import cv2

# Initialize video stream (0 for live camera or path to video file e.g., 'player_run.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open video feed.")
    exit()

# Parameters for Shi-Tomasi Corner Detection (Features to Track)
feature_params = dict(
    maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize=7
)

# Parameters for Lucas-Kanade Optical Flow
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03),
)

# Read the initial frame
```

```
ret, old_frame = cap.read()
if not ret or old_frame is None:
    print("Error: Failed to read frame.")
    cap.release()
    exit()

old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

# Select initial feature points to track on the player
# In practice, pass an ROI mask over the player to track features only on them
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

# Create a mask image for drawing optical flow tracking lines
mask = np.zeros_like(old_frame)



def get_direction_label(dx, dy, threshold=0.5):
    """Convert average displacement vector (dx, dy) into directional text."""
    magnitude = np.sqrt(dx**2 + dy**2)
    if magnitude < threshold:
        return "Stationary"

    # Calculate angle in degrees (-180 to 180)
    angle = np.arctan2(-dy, dx) * 180 / np.pi  # Invert dy due to image coords

    if -22.5 <= angle < 22.5:
        return "Right"
    elif 22.5 <= angle < 67.5:
        return "Up-Right"
    elif 67.5 <= angle < 112.5:
        return "Up"
    elif 112.5 <= angle < 157.5:
        return "Up-Left"
    elif angle >= 157.5 or angle < -157.5:
        return "Left"
    elif -157.5 <= angle < -112.5:
        return "Down-Left"
    elif -112.5 <= angle < -67.5:
        return "Down"
    elif -67.5 <= angle < -22.5:
        return "Down-Right"



while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    if p0 is not None and len(p0) > 0:
        # Calculate Lucas-Kanade Optical Flow: p1 = new points, st = status, err = error
        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray, frame_gray, p0, None, **lk_params
        )

        # Select good points (status == 1 means feature was found in new frame)
        if p1 is not None:
            good_new = p1[st == 1]
```

```
            good_old = p0[st == 1]

            displacements_x = []
            displacements_y = []

            # Draw tracking trajectories and compute displacement
            for i, (new, old) in enumerate(zip(good_new, good_old)):
                a, b = new.ravel()
                c, d = old.ravel()

                dx = a - c
                dy = b - d
                displacements_x.append(dx)
                displacements_y.append(dy)

                # Draw motion line and current point
                mask = cv2.line(
                    mask,
                    (int(a), int(b)),
                    (int(c), int(d)),
                    (0, 255, 0),
                    2,
                )
                frame = cv2.circle(
                    frame, (int(a), int(b)), 4, (0, 0, 255), -1
                )

            # Determine global average direction of all valid points
            if displacements_x:
                avg_dx = np.mean(displacements_x)
                avg_dy = np.mean(displacements_y)
                direction = get_direction_label(avg_dx, avg_dy)

                # Display direction and movement vector on output
                cv2.putText(
                    frame,
                    f"Movement Direction: {direction}",
                    (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.8,
                    (0, 255, 255),
                    2,
                )
                cv2.putText(
                    frame,
                    f"Vector (dx, dy): ({avg_dx:.2f}, {avg_dy:.2f})",
                    (10, 60),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.5,
                    (255, 255, 255),
                    1,
                )

            # Overlay trajectory mask on current frame
            output = cv2.add(frame, mask)

            # Update previous frame and previous points for next iteration
            old_gray = frame_gray.copy()
            p0 = good_new.reshape(-1, 1, 2)
        else:
```

```
            output = frame
    else:
        # Re-detect feature points if all existing points are lost
        p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
        output = frame

    # Refresh feature points every 60 frames to keep tracking fresh
    if cv2.waitKey(30) & 0xFF == ord("r"):
        p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
        mask = np.zeros_like(frame)

    cv2.imshow("Player Motion Direction (Lucas-Kanade)", output)

    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

```
# question 8
```

```
import cv2

# Initialize video feed (0 for live security camera, or path to video file e.g., 'entrance.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open entrance video feed.")
    exit()

# 1. Initialize HOG Person Detector
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())

# List to hold active trackers for each detected person
trackers = []



def is_new_person(new_box, existing_trackers, iou_threshold=0.3):
    """Checks if a newly detected bounding box overlaps with an already tracked person."""
    nx, ny, nw, nh = new_box

    for tracker in existing_trackers:
        success, box = tracker.update(frame)
        if success:
            tx, ty, tw, th = [int(v) for v in box]

            # Compute Intersection over Union (IoU)
            x1 = max(nx, tx)
            y1 = max(ny, ty)
            x2 = min(nx + nw, tx + tw)
            y2 = min(ny + nh, ty + th)

            inter_area = max(0, x2 - x1) * max(0, y2 - y1)
            new_area = nw * nh
```

```
            tracked_area = tw * th
            union_area = new_area + tracked_area - inter_area

            if union_area > 0 and (inter_area / union_area) > iou_threshold:
                return False
    return True



frame_count = 0
# Re-detect every N frames to catch new people entering while tracking existing ones
REDETECT_INTERVAL = 15

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    # Resize frame for faster processing speed
    frame = cv2.resize(frame, (800, 600))
    active_trackers = []

    # --- STEP 1: Update Existing Trackers ---
    for tracker in trackers:
        success, bbox = tracker.update(frame)
        if success:
            active_trackers.append(tracker)
            x, y, w, h = [int(v) for v in bbox]

            # Draw bounding box and label for tracked person
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.putText(
                frame,
                "Person",
                (x, y - 8),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.5,
                (0, 255, 0),
                2,
            )

    trackers = active_trackers

    # --- STEP 2: Detect New Entrants (Periodically) ---
    if frame_count % REDETECT_INTERVAL == 0 or len(trackers) == 0:
        # Detect people in frame using HOG + SVM
        # winStride and padding balance detection speed and accuracy
        boxes, weights = hog.detectMultiScale(
            frame, winStride=(8, 8), padding=(8, 8), scale=1.05
        )

        for i, (x, y, w, h) in enumerate(boxes):
            # Confidence threshold to eliminate false positives
            if weights[i] > 0.6:
                new_box = (x, y, w, h)

                # Instantiate tracker if person is not already tracked
                if is_new_person(new_box, trackers):
                    new_tracker = cv2.TrackerCSRT_create()
```

```
                    new_tracker.init(frame, new_box)
                    trackers.append(new_tracker)

    # --- STEP 3: Display System Overlay ---
    cv2.putText(
        frame,
        f"Entrance Monitor | Active People: {len(trackers)}",
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (0, 255, 255),
        2,
    )

    cv2.imshow("Building Security Entrance Feed", frame)

    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[10], line 94
     92 # Instantiate tracker if person is not already tracked
     93 if is_new_person(new_box, trackers):
---> 94     new_tracker = cv2.TrackerCSRT_create()
     95     new_tracker.init(frame, new_box)
     96     trackers.append(new_tracker)

AttributeError: module 'cv2' has no attribute 'TrackerCSRT_create'
```

```
# question 9
```

```
import collections
import time
import cv2
import mediapipe as mp
import numpy as np

# Initialize MediaPipe Pose Model
mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(
    static_image_mode=False,
    model_complexity=1,
    smooth_landmarks=True,
    min_detection_confidence=0.5,
    min_tracking_confidence=0.5,
)

# Initialize Video Feed (0 for live camera or path to file e.g., 'exercise.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)
```

```
if not cap.isOpened():
    print("Error: Could not open video feed.")
    exit()

# Buffer to store recent hip positions for velocity calculation (last 10 frames)
POSITION_BUFFER_SIZE = 10
hip_positions = collections.deque(maxlen=POSITION_BUFFER_SIZE)
prev_time = time.time()



def calculate_angle(a, b, c):
    """Calculates the 2D angle (in degrees) between three landmark joints."""
    a = np.array(a)  # First joint (e.g., Hip)
    b = np.array(b)  # Mid joint (e.g., Knee)
    c = np.array(c)  # End joint (e.g., Ankle)

    radians = np.arctan2(c[1] - b[1], c[0] - b[0]) - np.arctan2(
        a[1] - b[1], a[0] - b[0]
    )
    angle = np.abs(radians * 180.0 / np.pi)

    if angle > 180.0:
        angle = 360 - angle

    return angle



def classify_activity(hip_speed, left_knee_angle, right_knee_angle):
    """Classifies human activity based on joint velocity and knee flexion angles."""
    # Thresholds (tuned for normalized screen coordinates per second)
    RUNNING_SPEED_THRESHOLD = 0.25
    WALKING_SPEED_THRESHOLD = 0.08

    # Average knee angle (lower angle = more bent leg)
    avg_knee_angle = (left_knee_angle + right_knee_angle) / 2.0

    if hip_speed > RUNNING_SPEED_THRESHOLD:
        return "Running", (0, 0, 255)  # Red
    elif hip_speed > WALKING_SPEED_THRESHOLD:
        return "Walking", (0, 255, 255)  # Yellow
    else:
        return "Standing", (0, 255, 0)  # Green



while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    curr_time = time.time()
    dt = curr_time - prev_time
    prev_time = curr_time

    # Convert BGR image to RGB for MediaPipe
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = pose.process(rgb_frame)

    activity = "Detecting..."
```

```
    color = (255, 255, 255)

    if results.pose_landmarks:
        landmarks = results.pose_landmarks.landmark

        # Extract relevant landmark coordinates (Normalized x, y)
        left_hip = [
            landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x,
            landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y,
        ]
        right_hip = [
            landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value].x,
            landmarks[mp_pose.PoseLandmark.RIGHT_HIP.value].y,
        ]

        left_knee = [
            landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].x,
            landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].y,
        ]
        right_knee = [
            landmarks[mp_pose.PoseLandmark.RIGHT_KNEE.value].x,
            landmarks[mp_pose.PoseLandmark.RIGHT_KNEE.value].y,
        ]

        left_ankle = [
            landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x,
            landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y,
        ]
        right_ankle = [
            landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE.value].x,
            landmarks[mp_pose.PoseLandmark.RIGHT_ANKLE.value].y,
        ]

        # Calculate midpoint of hips to track core body position
        mid_hip = [(left_hip[0] + right_hip[0]) / 2, (left_hip[1] + right_hip[1]) / 2]
        hip_positions.append((mid_hip, curr_time))

        # Calculate Hip Velocity (Distance moved / time elapsed)
        hip_speed = 0.0
        if len(hip_positions) >= 2:
            (pos_old, time_old) = hip_positions[0]
            (pos_new, time_new) = hip_positions[-1]
            time_diff = time_new - time_old

            if time_diff > 0:
                dist = np.sqrt(
                    (pos_new[0] - pos_old[0]) ** 2
                    + (pos_new[1] - pos_old[1]) ** 2
                )
                hip_speed = dist / time_diff

        # Calculate joint angles
        left_knee_angle = calculate_angle(left_hip, left_knee, left_ankle)
        right_knee_angle = calculate_angle(right_hip, right_knee, right_ankle)

        # Classify current frame's activity
        activity, color = classify_activity(
            hip_speed, left_knee_angle, right_knee_angle
        )
```

```
        # Draw Pose Skeleton overlay
        mp_drawing.draw_landmarks(
            frame,
            results.pose_landmarks,
            mp_pose.POSE_CONNECTIONS,
            mp_drawing.DrawingSpec(
                color=(245, 117, 66), thickness=2, circle_radius=2
            ),
            mp_drawing.DrawingSpec(
                color=(245, 66, 230), thickness=2, circle_radius=2
            ),
        )

    # UI Overlay
    cv2.rectangle(frame, (0, 0), (350, 60), (0, 0, 0), -1)
    cv2.putText(
        frame,
        f"Activity: {activity}",
        (10, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        color,
        2,
        cv2.LINE_AA,
    )

    cv2.imshow("Human Activity Recognition", frame)

    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

```
# question 10
```

```
from collections import deque
import cv2
import numpy as np

# Initialize video feed (0 for live CCTV feed, or path to video file e.g., 'lab_cctv.mp4')
video_path = "person_walking_in_bedroom.mp4"
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    print("Error: Could not open CCTV video feed.")
    exit()

# Parameters for Farneback Dense Optical Flow
farneback_params = dict(
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0,
)
```

```
# Rolling history buffer for dynamic baseline statistical calculation
HISTORY_WINDOW = 100  # Number of frames to compute running mean/std
motion_magnitude_history = deque(maxlen=HISTORY_WINDOW)

# Anomaly threshold: Number of standard deviations above mean to trigger alert
Z_SCORE_THRESHOLD = 3.0
MIN_BASELINE_FRAMES = 30  # Minimum frames required to establish baseline

# Read initial frame
ret, prev_frame = cap.read()
if not ret or prev_frame is None:
    print("Error: Failed to read video feed.")
    cap.release()
    exit()

prev_gray = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    curr_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 1. Compute Dense Optical Flow using Farneback algorithm
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray, curr_gray, None, **farneback_params
    )

    # 2. Calculate magnitude and angle of motion vectors for all pixels
    magnitude, angle = cv2.cartToPolar(flow[..., 0], flow[..., 1])

    # Filter out minor background sensor noise
    motion_mask = magnitude > 1.5
    active_motion_magnitude = magnitude[motion_mask]

    # Calculate average motion intensity in current frame
    if len(active_motion_magnitude) > 0:
        current_mean_motion = np.mean(active_motion_magnitude)
    else:
        current_mean_motion = 0.0

    # 3. Anomaly Detection Logic via Moving Z-Score
    is_anomaly = False
    z_score = 0.0

    if len(motion_magnitude_history) >= MIN_BASELINE_FRAMES:
        baseline_mean = np.mean(motion_magnitude_history)
        baseline_std = np.std(motion_magnitude_history) + 1e-5  # Prevent div-by-zero

        # Z-Score: measures how many standard deviations the current frame deviates from baseline
        z_score = (current_mean_motion - baseline_mean) / baseline_std

        if z_score > Z_SCORE_THRESHOLD:
            is_anomaly = True

    # Update dynamic baseline history
```

```
    motion_magnitude_history.append(current_mean_motion)

    # 4. Generate Visualization Overlay
    # Map motion vectors to HSV color wheel (Hue = direction, Value = speed)
    hsv = np.zeros_like(frame)
    hsv[..., 1] = 255
    hsv[..., 0] = angle * 180 / np.pi / 2
    hsv[..., 2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
    flow_rgb = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

    # Draw Status and Alert Overlay
    if is_anomaly:
        # Red Border Alert for unusual movement
        cv2.rectangle(
            frame, (0, 0), (frame.shape[1], frame.shape[0]), (0, 0, 255), 10
        )
        cv2.rectangle(frame, (0, 0), (450, 60), (0, 0, 255), -1)
        cv2.putText(
            frame,
            "ALERT: UNUSUAL MOVEMENT DETECTED!",
            (10, 38),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (255, 255, 255),
            2,
            cv2.LINE_AA,
        )
    else:
        # Green status bar for normal movement
        cv2.rectangle(frame, (0, 0), (320, 50), (0, 0, 0), -1)
        cv2.putText(
            frame,
            "Status: Normal Activity",
            (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.7,
            (0, 255, 0),
            2,
            cv2.LINE_AA,
        )

    # Display Z-Score and Flow metrics
    cv2.putText(
        frame,
        f"Motion Index: {current_mean_motion:.2f} | Z-Score: {z_score:.2f}",
        (10, frame.shape[0] - 15),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (255, 255, 255),
        1,
        cv2.LINE_AA,
    )

    # Render display windows
    cv2.imshow("Restricted Lab CCTV Monitor", frame)
    cv2.imshow("Optical Flow Heatmap", flow_rgb)

    prev_gray = curr_gray.copy()

    key = cv2.waitKey(30) & 0xFF
    if key == ord("q") or key == 27:
```

```
        break

cap.release()
cv2.destroyAllWindows()
```